

Lesson 3: Decision Trees and Ensembles of Trees



3.0 Lesson Overview



Lesson Overview

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



3.1 Building a Default Decision Tree Model



Basics of Decision Trees



Copyright © SAS Institute Inc. All rights reserved.

Basics of Decision Trees



Essential Discovery Tasks

- ✓ Select an algorithm.
- Improve the model.
- Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.
- Build ensemble models.



For the first task in the Discovery phase of the analytics lifecycle, we've selected the *decision tree* algorithm. In this lesson, you learn to build individual decision trees as well as tree-based ensemble models. To start, you learn the basics about decision trees.

Analysis Goal: Predict the outcome based on the location in the plot.

IF $X_2 < 0.63$ THEN branch left.
ELSE IF $X_2 \geq 0.63$ THEN
branch right.

rules

root node

$X_2 < 0.63$

yes

no

interior node

$X_1 < 0.52$

$X_1 < 0.51$

yes

no

yes

no

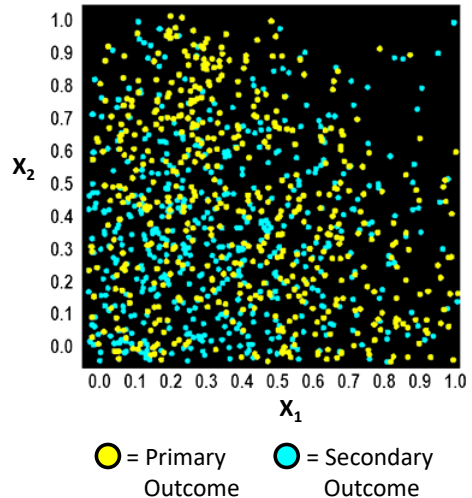
leaf node



40% 60%



70% 55%



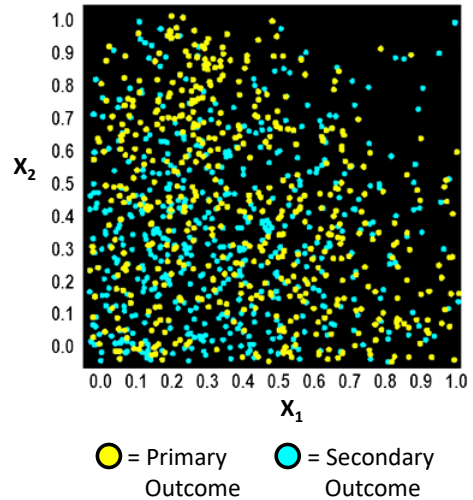
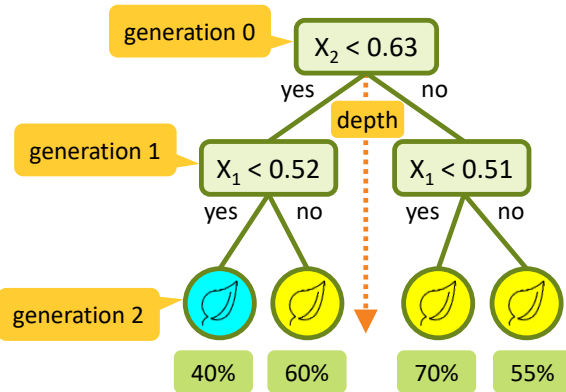
Let's look at a simple example. In this plot, the input X_1 is on the horizontal axis and the input X_2 is on the vertical axis. The values of the binary target are plotted according to the corresponding values of the two inputs. The target outcome is represented by a color: yellow is the primary outcome and blue is the secondary outcome. The analysis goal is to predict the outcome based on the location in the plot.

To predict cases (that is, to score data), decision trees use rules that specify a decision based on the values of the input variables. The rules are expressed in Boolean logic, which means that they are if-then-else statements. The rules are arranged hierarchically in a tree-like structure with nodes connected by lines. The nodes represent the rules, and the lines order the rules.

The first rule, at the base (that is, the top) of the tree, is named the *root node*. For example, the rule in the root node shown here is: If the value of X_2 is less than 0.63, branch left; else if the value of X_2 is greater than or equal to 0.63, branch right.

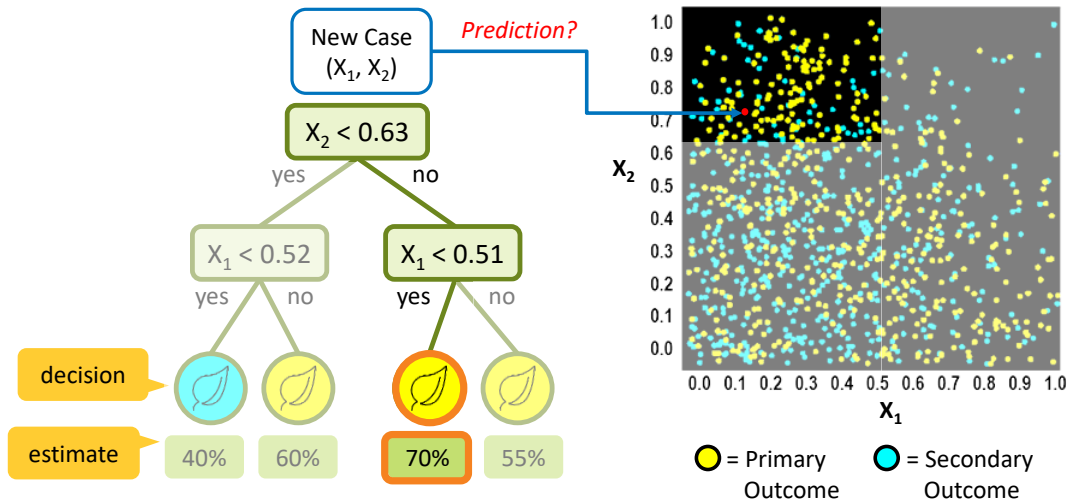
Subsequent rules are named *interior nodes*. Nodes with only one connection are *leaf nodes*.

Analysis Goal: Predict the outcome based on the location in the plot.



The *depth* of a tree specifies the number of generations of nodes. The root node is generation 0. The children of the root node are the first generation, and so on. The tree shown here stops at the second generation.

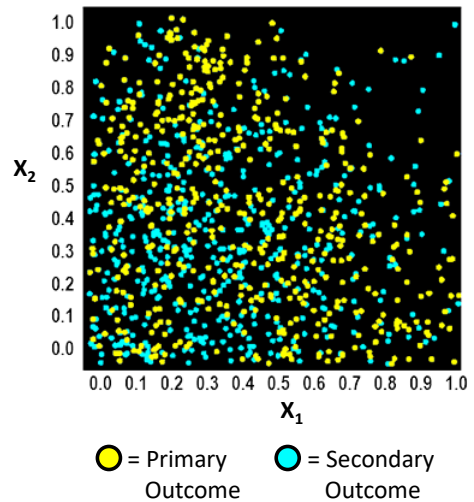
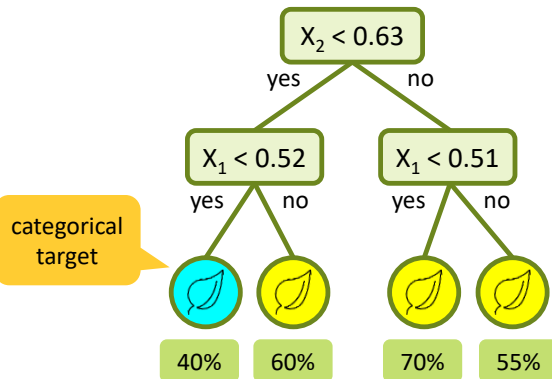
Analysis Goal: Predict the outcome based on the location in the plot.



To score a new case, we examine the input values and apply the rules defined by the decision tree. The input values of a new case eventually lead to a single leaf in the tree. A tree leaf provides a prediction that typically includes a decision (for example, to classify the outcome as yellow) and an estimate (for example, the primary-target proportion of 70%).

Analysis Goal: Predict the outcome based on the location in the plot.

✓
An interval target can also be used.



In this example, the target is a categorical target. However, as you'll see later, decision trees can also be used for interval targets.

Question: Decision Tree Nodes



Copyright © SAS Institute Inc. All rights reserved.

Question: Decision Tree Nodes

Which component of a decision tree provides the predictions?

- a. child nodes
- b. interior nodes
- c. leaf nodes
- d. nodes in generation 0
- e. root nodes

Answer: Decision Tree Nodes

Which component of a decision tree provides the predictions?

- a. child nodes
- b. interior nodes
- ☒ c. leaf nodes
- d. nodes in generation 0
- e. root nodes

CA: c

FB: A tree's leaf nodes provide the predictions. Leaf nodes are child nodes, but not all child nodes are leaves. Generation 0 is the root level of the tree, which has one root node. The root node does not make predictions. The interior nodes are all the nodes between the root node and the leaves.

Demo: Building a Decision Tree Model Using the Default Settings

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Supervised Learning Node Results Window

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Score Code in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Interactively Edit a Decision Tree

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



3.2 Modifying the Model: Tree Structure

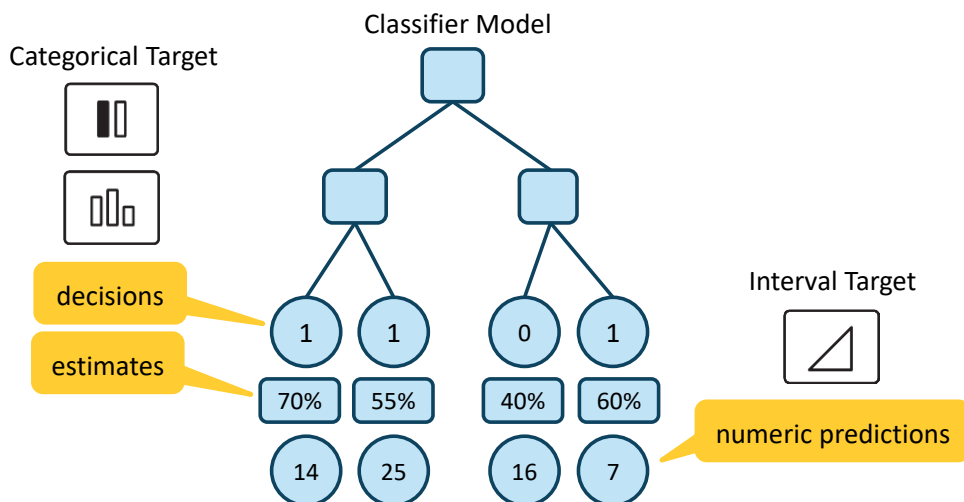


Decision Trees for Categorical Targets: Classification Trees

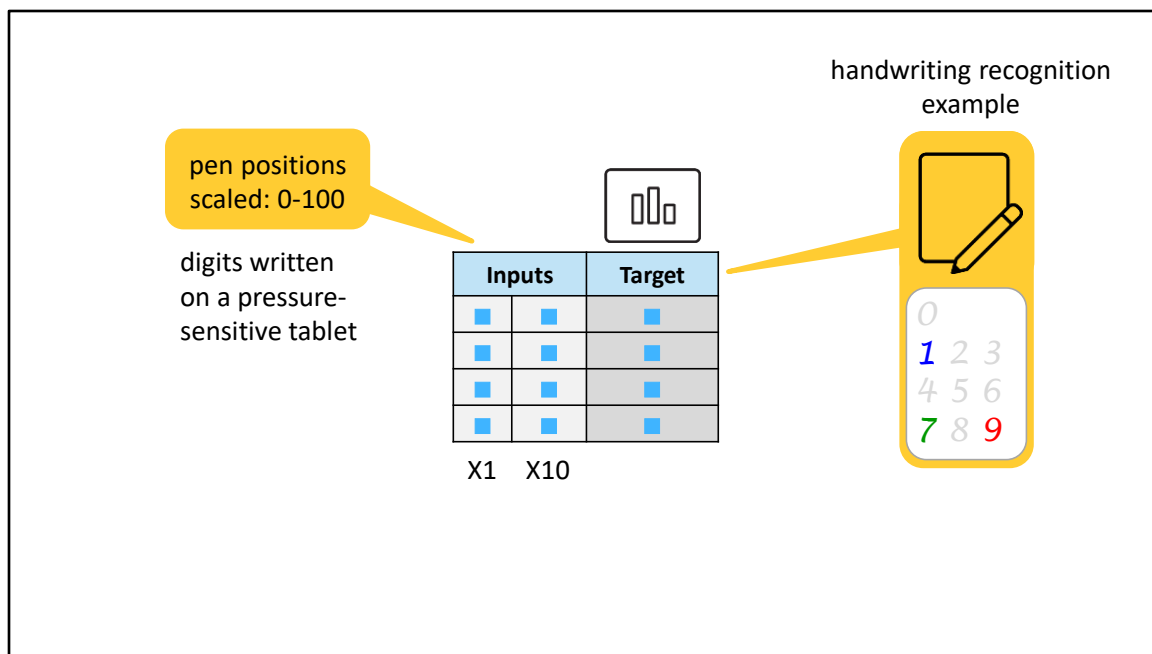


Copyright © SAS Institute Inc. All rights reserved.

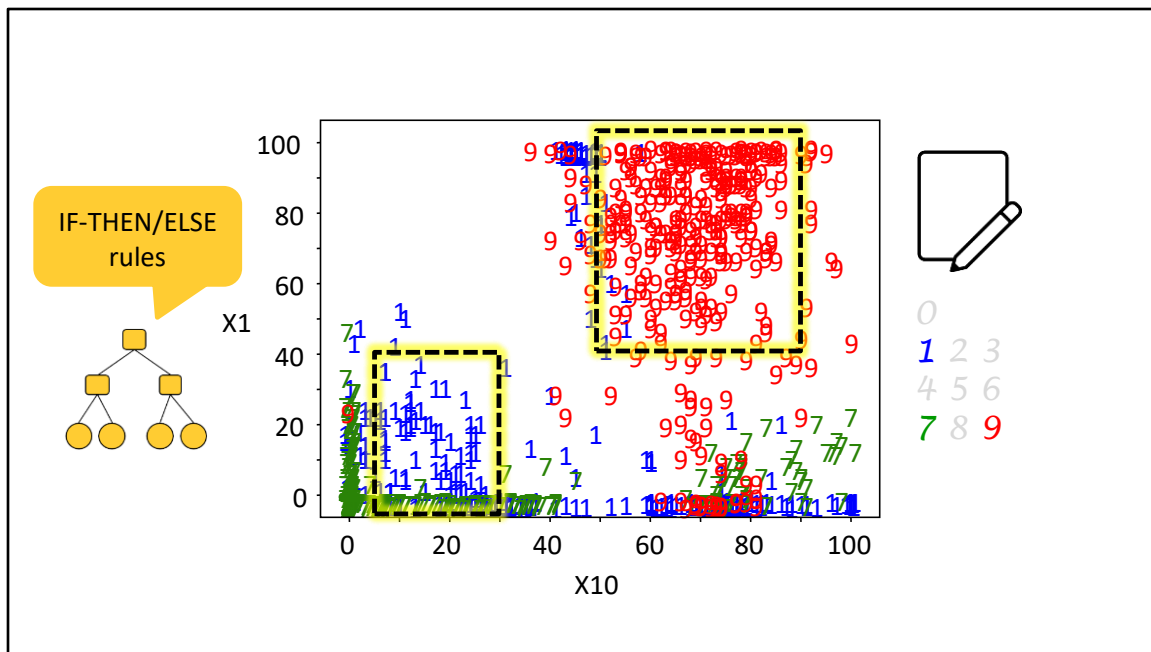
Decision Trees for Categorical Targets: Classification Trees



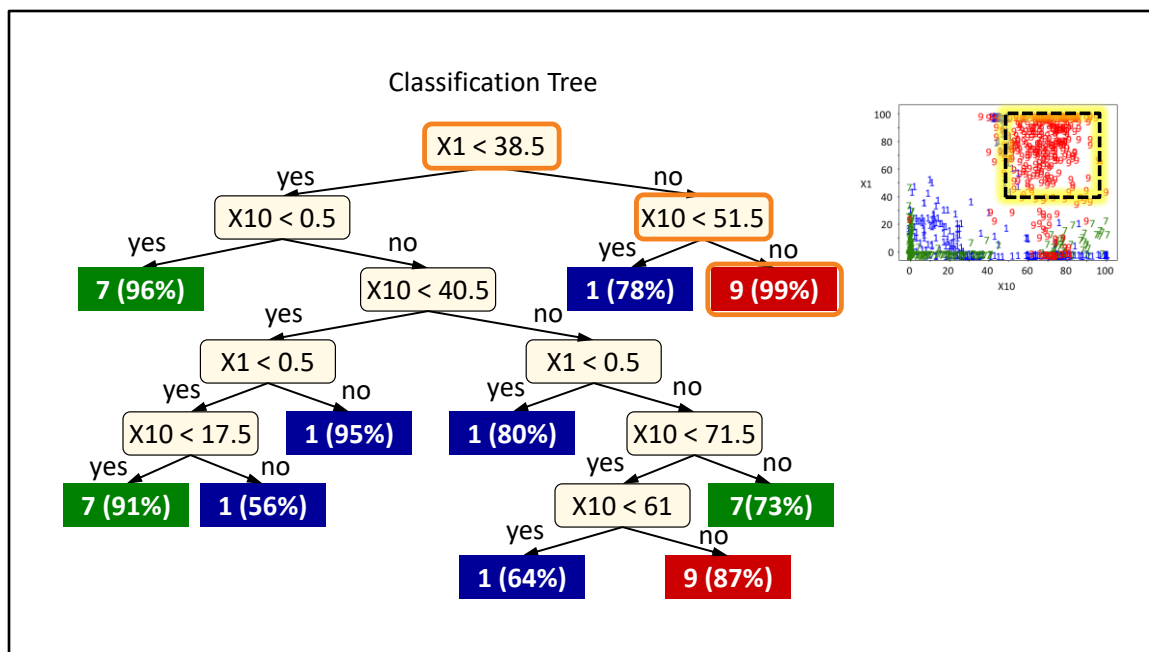
Decision trees were originally developed to be pure classifier models (in other words, to make *decisions* for a categorical target). They have since been expanded to provide estimates such as probabilities for categorical targets and numeric predictions for interval targets.



Handwriting recognition is a classic application for a classifier model. In the example shown here, the categorical target is a hand-written digit, from 0-9. The digits were written on a pressure-sensitive tablet. The input variables are measurements of the positions of the pen, and the inputs are scaled between 0 and 100. In this example, the goal is to predict whether the written digit is a 1, a 7, or a 9. The analysis data set contains a subset of cases: only the cases in which the target is one of the specified digits. The analysis data set contains two of the original 16 inputs: X1 and X10.

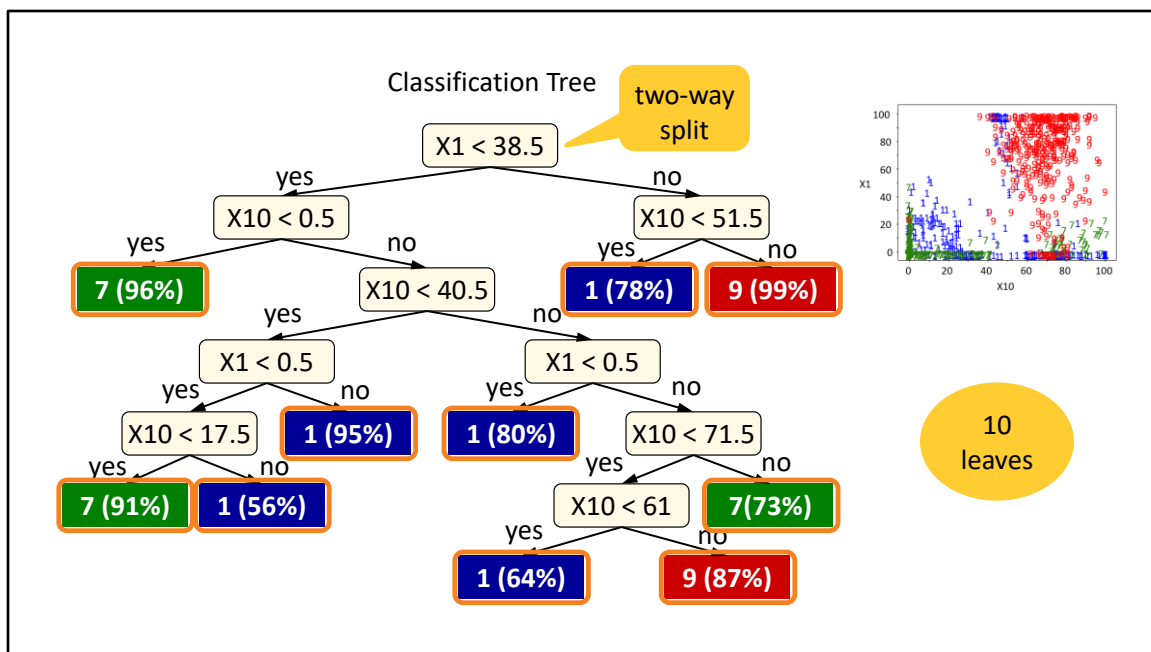


Notice that, in general, specific digits tend to group together based on specific values, or ranges, of the inputs. There are many 9s when X1 is approximately greater than 40, and X10 is between 55 and 90. There are many 1s when X1 is between 5 and 40 and X10 is between 5 and 30. A decision tree is based on if-then-else rules based on the values of the inputs. If X1 takes certain values and X10 takes certain values, then the outcome is likely to be a certain digit.



Here is a decision tree built against the subset of the handwriting data. Do you remember that group of 9s in the upper right quadrant of the data? In the tree, we can get to the leaf node for that group in two steps. Starting at the root node, if X_1 is *not* less than 38.5 *and* X_{10} is *not* less than 51.5, then there is a 99% chance that the written digit is a 9. In other words, the model found a subset of the data where 99% of the target values correspond to the written digit 9.

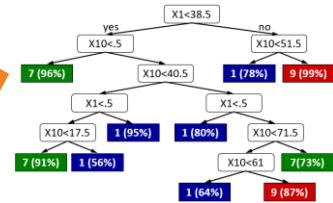
This type of decision tree is called a *classification* tree because it is built against a categorical target.



For this model, each branch of the tree is based on a two-way split, but decision trees can also have multi-way splits.

This tree provides 10 predictions (that is, there are 10 leaves). As shown, each estimate prediction represents the percentage of all data points in that leaf node that are the specified digit.

Leaf	Pr(1 x)	Pr(7 x)	Pr(9 x)	Decision
1	.03	.96	.01	7
2	.09	.91	.00	7
3	.56	.44	.00	1
4	.95	.05	.00	1
5	.80	.10	.10	1
6	.64	.09	.27	1
7	.00	.13	.87	9
8	.10	.73	.17	7
9	.78	.01	.21	1
10	.01	.00	.99	9



- This defines several multivariate step functions.
- Each function corresponds to the posterior probability of a target class.

This table shows the ten leaves of the tree and the posterior probability for each numeric digit. The posterior probability is the probability generated by a predictive model.

The last column in the table is the decision prediction. The decision for each leaf is the digit with the largest posterior probability for that leaf. Leaf 1 discovered a subset of the data where 3% of the values are 1, 96% of the values are 7, and only 1% are 9.

You can think of a classification tree as defining several multivariate step functions. Each function corresponds to the posterior probability of a target class.

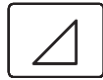
Decision Trees for Interval Targets: Regression Trees



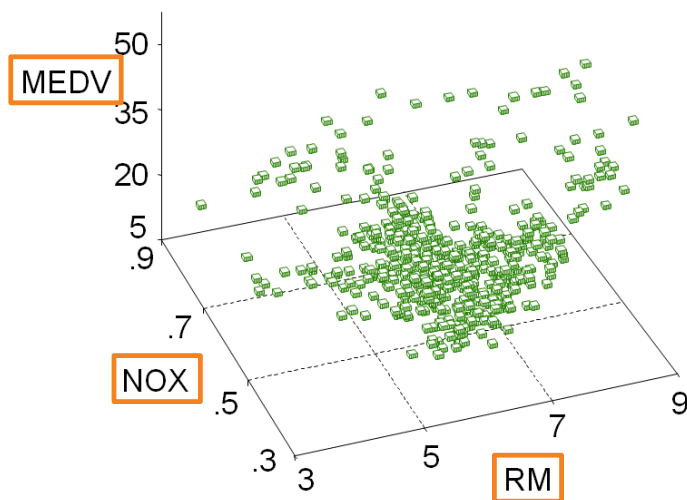
Copyright © SAS Institute Inc. All rights reserved.

Decision Trees for Interval Targets: Regression Trees

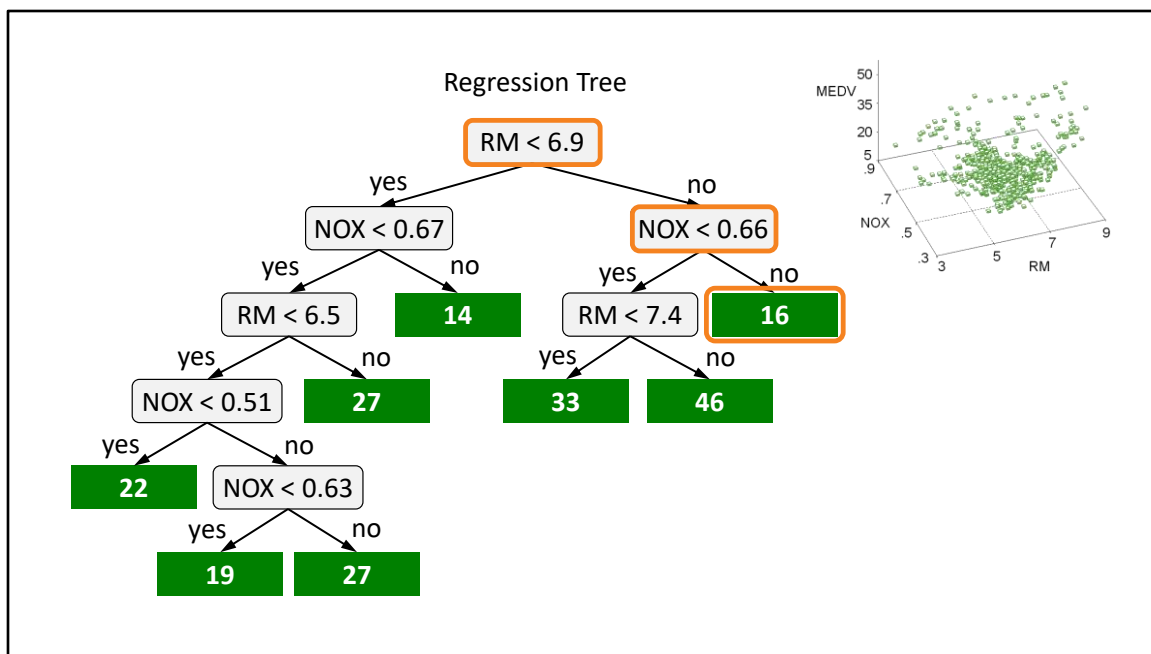
Interval Target



any numeric
value within a
certain range



Let's look at the use of a decision tree for an interval target. An interval target can have any numeric value, within a certain range, including decimal values. In the data set shown here, the target (MEDV) represents median home values in the Boston area (in units of 1,000 dollars US currency). The two input variables are RM (number of rooms) and NOX (nitrogen oxide concentration in the air).



A decision tree with an interval target is called a *regression* tree. The leaves represent the average of the median home values in that subset of the data, defined by the path that leads to the particular leaf. Just as in the classification tree shown earlier, the branches to the left indicate a "yes" response to the rule, and branches to the right indicate a "no" response.

For example, a house that does not have fewer than 6.9 rooms (that is, the house has 7 rooms or more) and does not have a nitrogen oxide concentration less than 0.66 (that is, the house has a nitrogen oxide concentration greater than or equal to 0.66) has a predicted median home value of 16,000 dollars.

This regression tree has eight leaves and thus can provide eight numeric predictions.

Leaves = Boolean Rules

Leaf	RM	NOX	Predicted MEDV
1	< 6.5	< .51	22
2	< 6.5	[.51, .63)	19
3	< 6.5	[.63, .67)	27
4	[6.5, 6.9)	< .67	27
5	< 6.9	$\geq$.67	14
6	[6.9, 7.4)	< .66	33
7	$\geq$ 7.4	< .66	46
8	$\geq$ 6.9	$\geq$.66	16

if $RM \in \{values\}$
and $NOX \in \{values\}$,
then $MEDV=value$.

Here is the regression tree for the Boston housing data, shown as a table. Unlike the table for the classification tree that you saw earlier, this table also shows the prediction rules. These rules are often referred to as English rules. Remember that the rules are based on Boolean logic (that is, they consist of a series of if-then-else statements).

For example, the rule for the first leaf is: if the number of rooms is less than 6.5 and the nitrogen oxide concentration is less than 0.51, then the predicted median home value is 22,000 dollars.

Question: Decision Tree Splits



Copyright © SAS Institute Inc. All rights reserved.

Question: Decision Tree Splits

A decision tree can have only two-way splits.

- a. True
- b. False

Answer: Decision Tree Splits

A decision tree can have only two-way splits.

- a. True
- ☒ b. False

CA: b

FB: A decision tree can have multi-way splits.

Improving the Decision Tree Model



Copyright © SAS Institute Inc. All rights reserved.

Improving the Decision Tree Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- Improve the model.
- Optimize the complexity of
- Regularize and tune the hyperparameters of the model.
- Build ensemble models.

structure

recursive
partitioning
method

After building an initial tree, the next essential discovery task is to improve the model. With decision trees, there are many settings that we can consider modifying. Some settings affect the structure of the tree, such as depth and leaf size. Other settings relate to the recursive partitioning method that is used to grow a tree, as well as any associated parameters.

Demo: Modifying the Structure Parameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Decision Trees



Copyright © SAS Institute Inc. All rights reserved.

Question: Decision Trees

Which of the following statements is true regarding decision trees?

- a. To predict cases, decision trees use rules that involve the values or categories of the input variables.
- b. Decision trees can handle only categorical targets.
- c. Each predictor variable can appear only in a single split in the tree.
- d. A decision tree for a categorical target can provide decisions but not estimates.

Answer: Decision Trees

Which of the following statements is true regarding decision trees?

- ☒ a. To predict cases, decision trees use rules that involve the values or categories of the input variables.
- b. Decision trees can handle only categorical targets.
- c. Each predictor variable can appear only in a single split in the tree.
- d. A decision tree for a categorical target can provide decisions but not estimates.

CA: a

FB: To predict cases, decision trees use Boolean (if-then-else) rules that involve the values or categories of the input variables.

3.3 Modifying the Model: Recursive Partitioning



Recursive Partitioning



Copyright © SAS Institute Inc. All rights reserved.

Recursive Partitioning

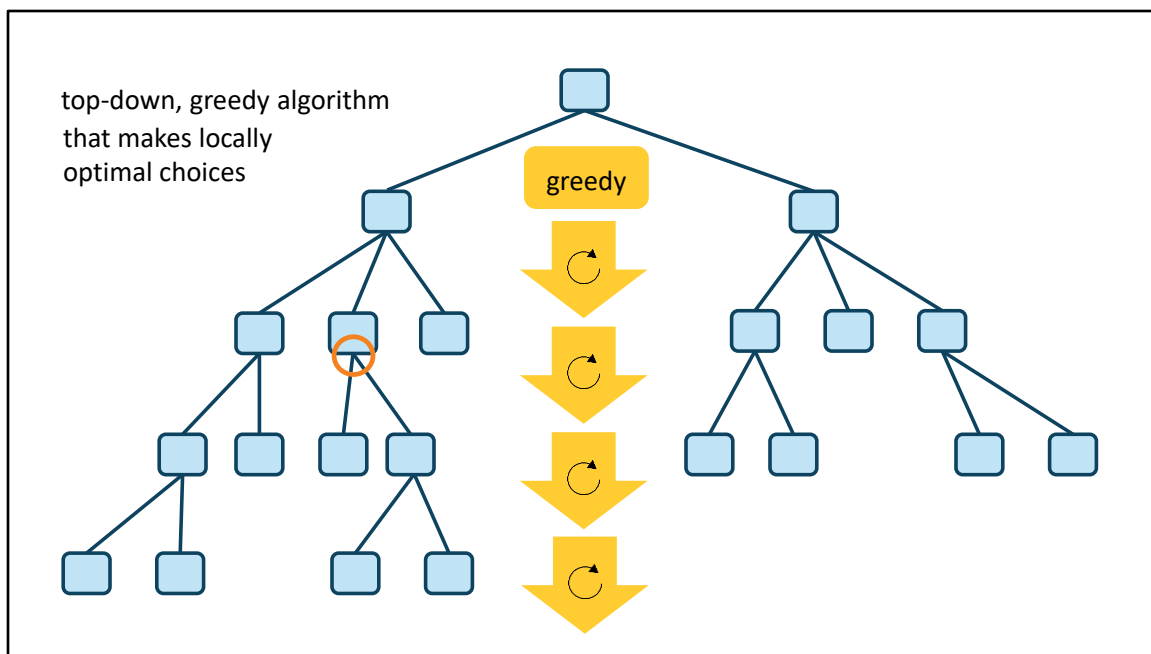


Essential Discovery Tasks

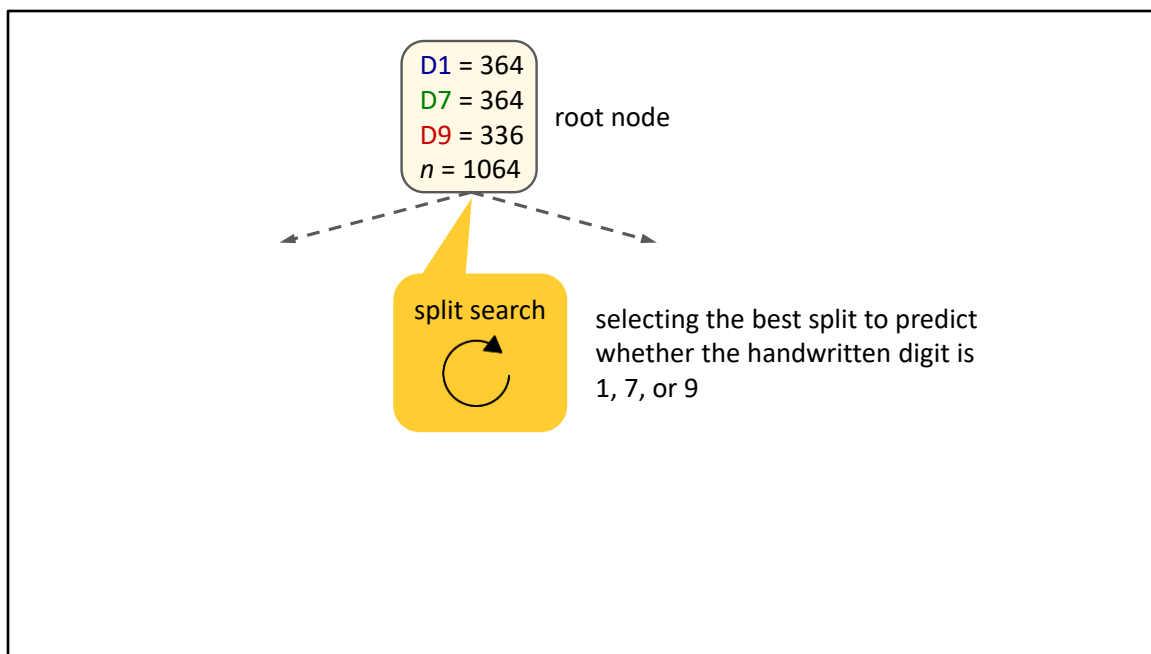
- ✓ Select an algorithm.
- Improve the model.
- Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.
- Build ensemble models.

recursive
partitioning method

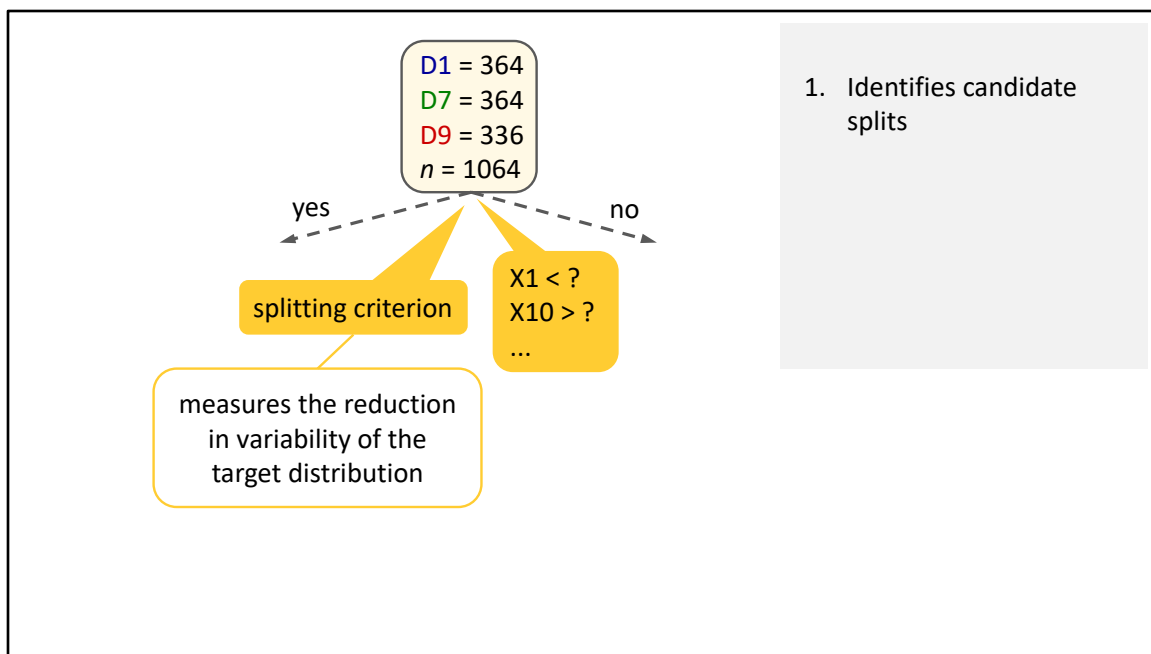
Another way to improve a decision tree model is to modify its growth by adjusting the recursive partitioning parameters that define the splits. Recursive partitioning is the standard method used to grow decision trees.



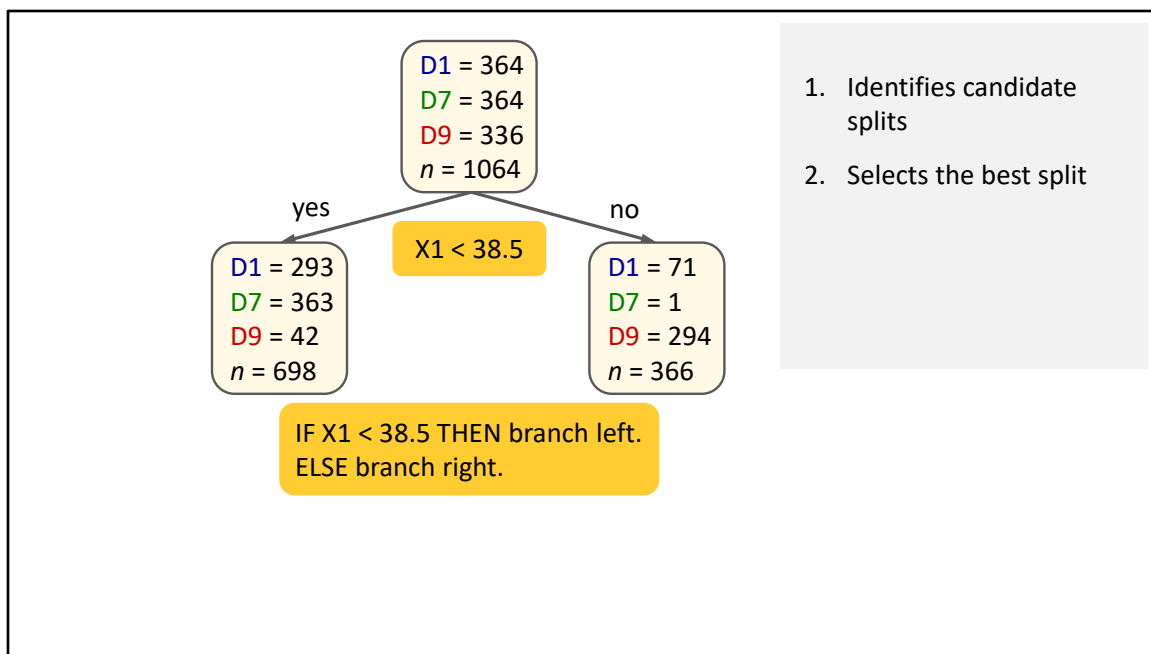
Recursive partitioning is a top-down, greedy algorithm. A *greedy* algorithm makes locally optimal choices at each step.



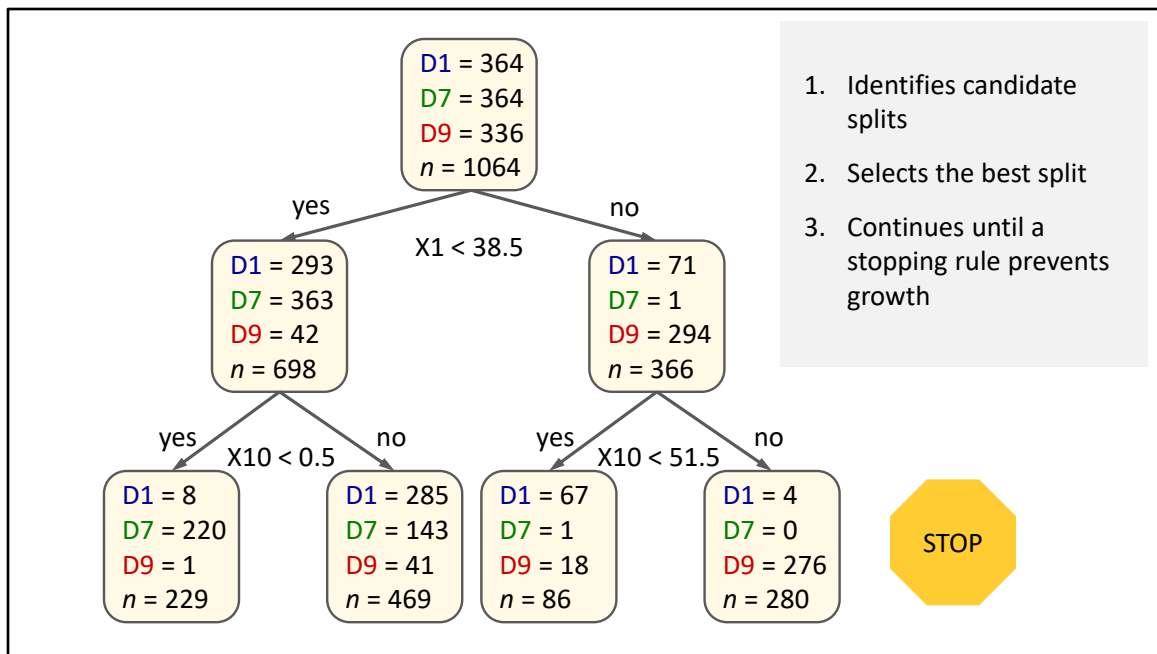
Here is the root node of a classification tree that was shown earlier. The analysis goal is to predict whether the handwritten digit is 1, 7, or 9. The root node represents the number of cases for each digit. Starting at the root node, recursive partitioning uses an iterative process to select the best split for the node. This process is called a split search.



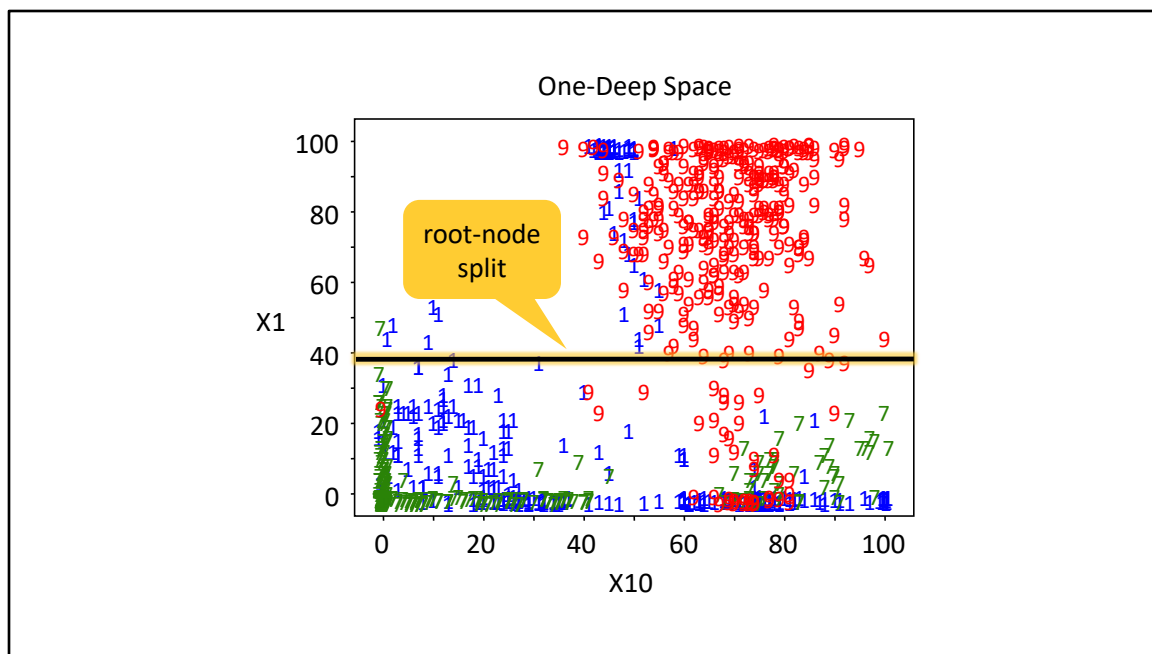
First, recursive partitioning identifies the best of all possible splits for each input variable based on the value of a splitting criterion. These are the candidate splits. The splitting criterion measures the reduction in variability of the target distribution in the child nodes. The goal is to reduce variability and thus increase purity in the child nodes.



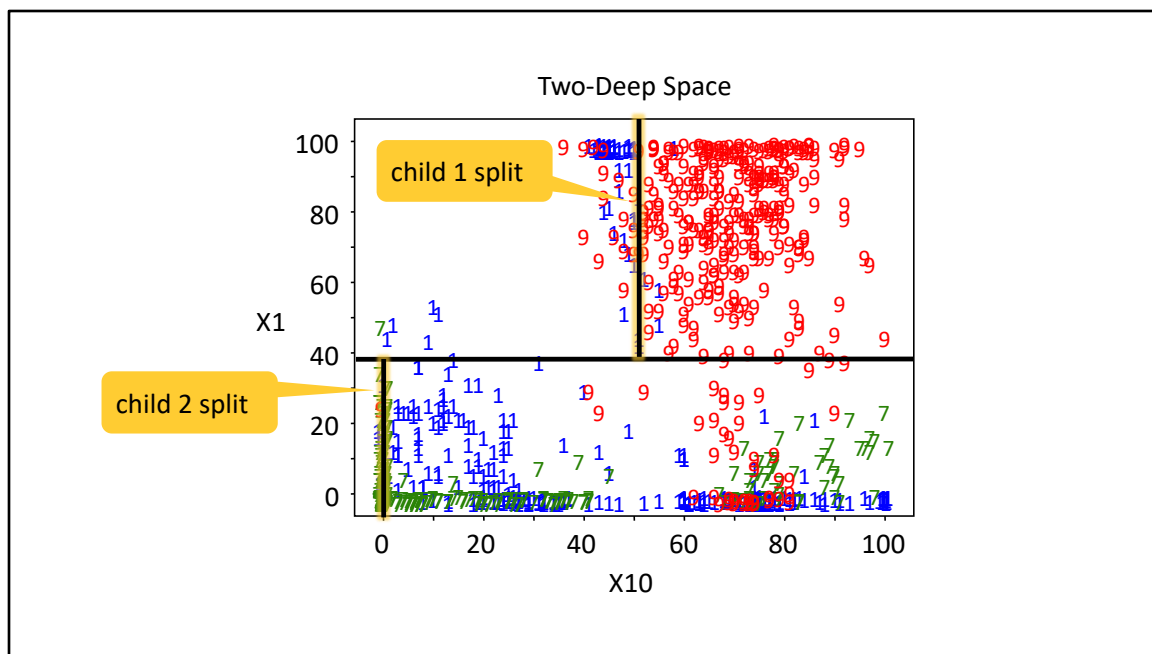
The process then selects the overall best split (that is, a split that is based on a single input). This split is expressed as an IF-THEN-ELSE rule. The cases in the root node are then partitioned according to the selected split. In this tree, the optimal split is to branch left when $X1$ is less than 38.5, and branch right when $X1$ is greater than or equal to 38.5. The 1064 observations in the root node have been partitioned such that 698 follow the rule to the left branch, and 366 follow the rule to the right branch.



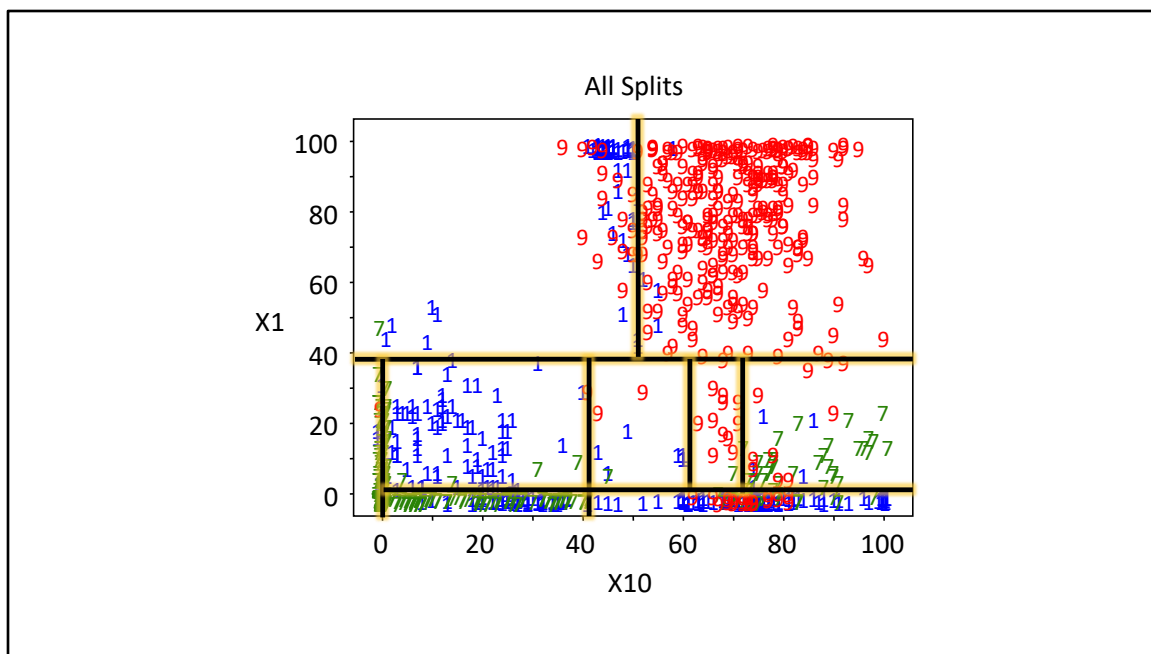
The split search is now repeated in each child node. The split selection at a node depends entirely on the cases in that local region of the input space. As the split search continues deeper into the tree, the data become more fragmented. The splitting criterion and other stopping rules determine the end of the process and the final depth of the tree.



Let's see what the split search process looks like in this plot of the training data. The root-node split corresponds to a partition of the input space where the boundary is perpendicular to one input dimension. The resulting tree has a depth of one (or is a one-deep space).



After the second iteration of the split search, the resulting tree has a depth of two. It is now a two-deep space. The resulting splits are shown in the plot.

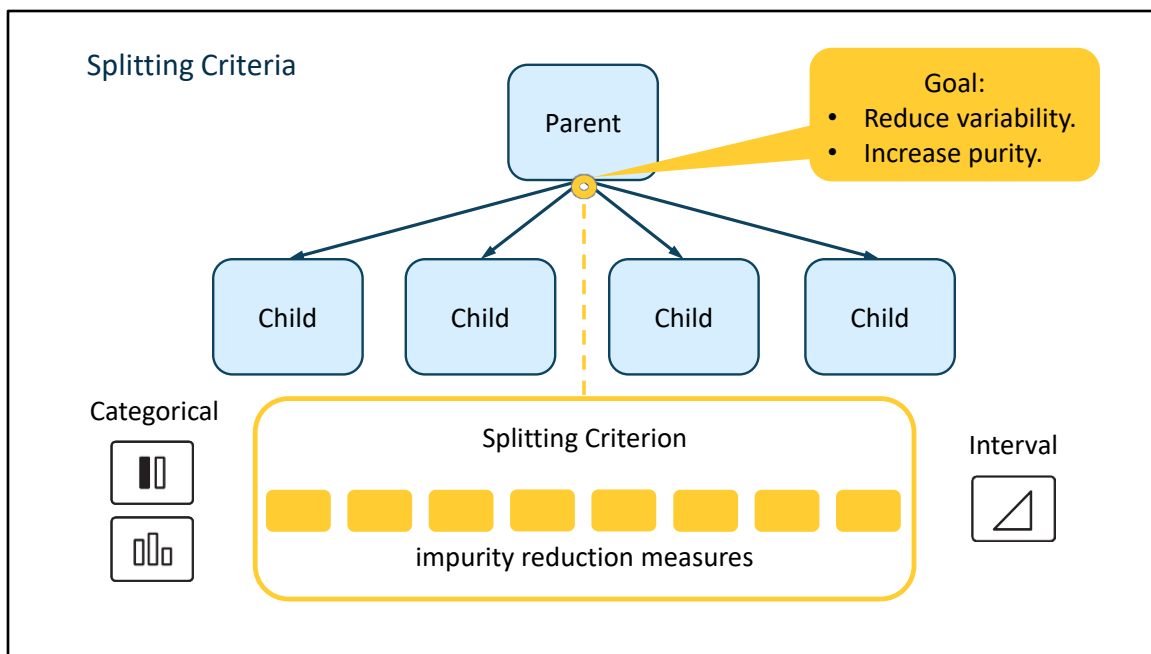


Finally, the plot shows all the splits that occurred before the recursive partitioning process stopped.

Splitting Criteria

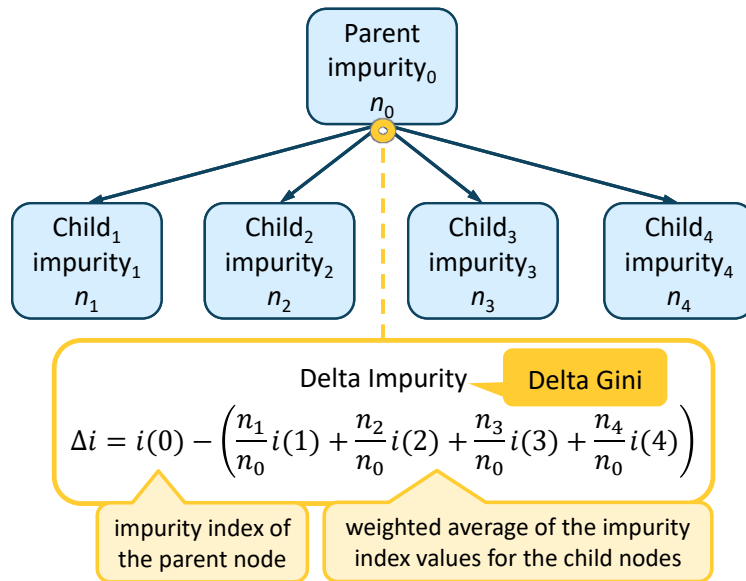


Copyright © SAS Institute Inc. All rights reserved.



The goal of splitting is always to reduce the variability of the target distribution and thus increase purity in the child nodes. A splitting criterion measures this reduction. Splitting criteria can be based on a variety of impurity reduction measures. Most of these measures apply only to categorical targets (including binary targets) or only to interval targets.

Categorical



Let's look at an example of a splitting criterion commonly used for each type of target. For categorical data, commonly-used splitting criteria measure the reduction in *node impurity* (that is, the reduction of within-node variability) induced by the split.

For these splitting criteria, the delta impurity criterion measures the difference between the impurity index of the parent node and a weighted average of the impurity index values for the child nodes. For example, the delta *Gini* splitting criterion measures the difference between the Gini index value of the parent node and a weighted average of the Gini index values of the child nodes.

Categorical



Node

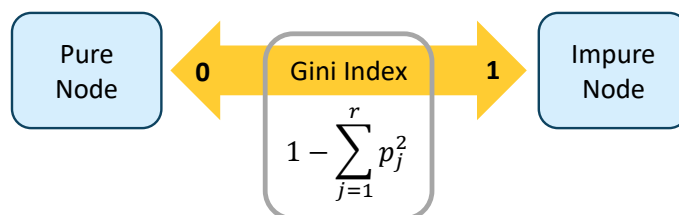
Gini Index

$$1 - \sum_{j=1}^r p_j^2$$

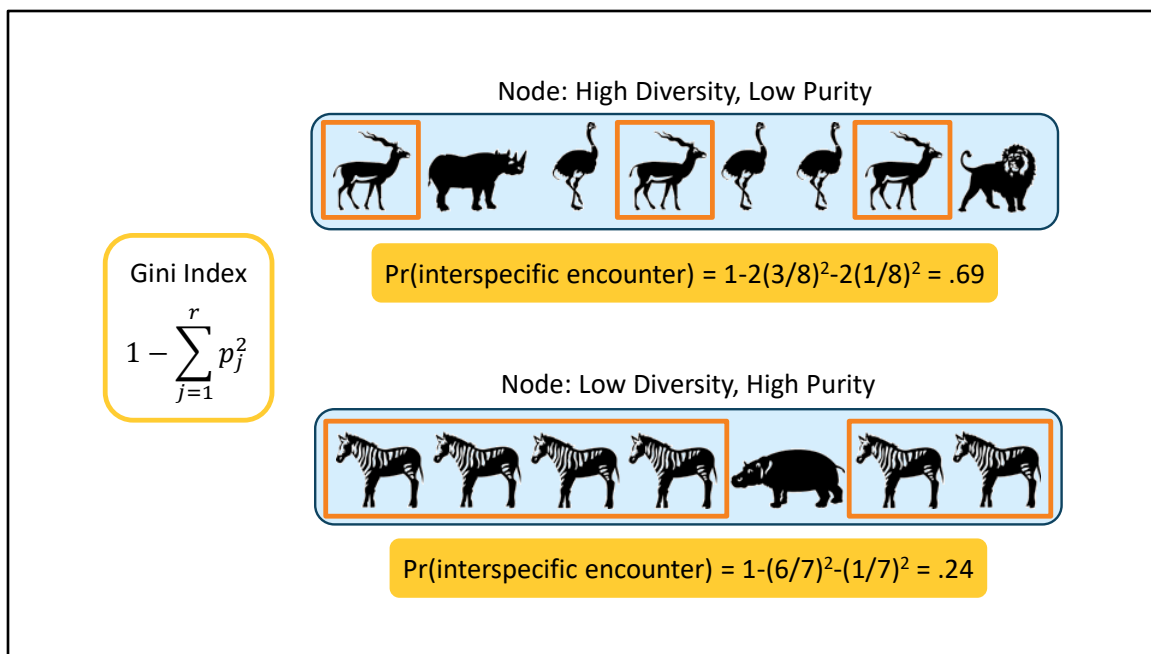
the probability that any two elements of a group, chosen at random, with replacement, are different

The Gini index equals 1 minus the sum of the probabilities for each individual target class appearing in the node. The Gini index can be interpreted as the probability that any two elements of a group, chosen at random (with replacement), are different.

Categorical

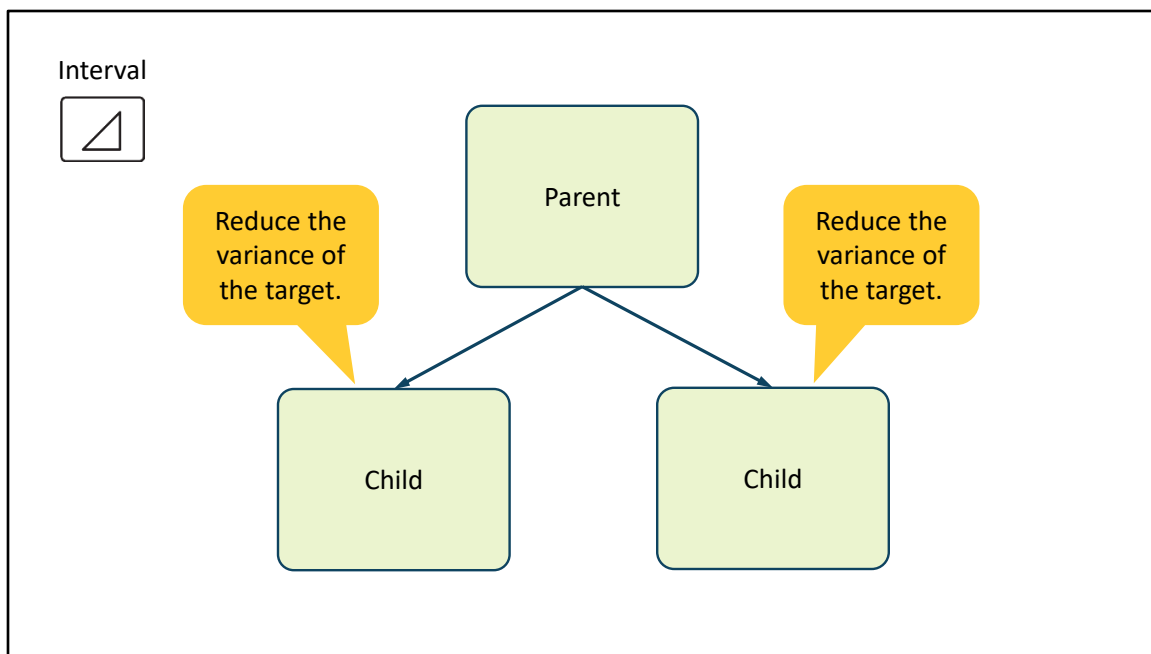


A pure node has a Gini index of 0. As the number of evenly distributed classes increases, the Gini index approaches 1.



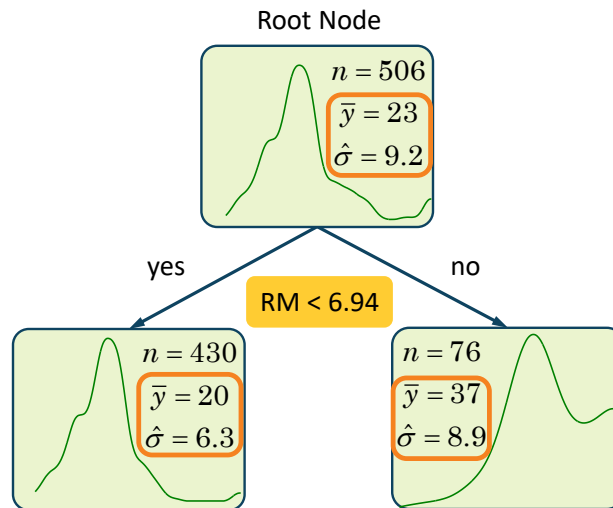
This diagram shows two groups of animals. Each group represents a node. The target classes that occur in each group are shown. In this case, the target is the species of animal. The first group has eight animals, which represent four species. In this group, the highest number of animals of any one species is three. This group has high diversity and low purity, and its Gini index value is 0.69. This means that, if we randomly select two animals from this group, there is a 69% chance that the animals will be different from each other. In other words, this is the chance of an interspecific encounter.

The second group has seven animals, which represent only two species: six zebras and one hippopotamus. This group has low diversity and high purity, and its Gini index value is 0.24. This means that, if we randomly select two animals from this group, there is only a 24% chance of an interspecific encounter.



When the target has an interval measurement level, splitting criteria can be based on reducing the variance of the target in the child nodes.

Interval



This example is based on the regression tree for the Boston housing data that was shown earlier in the lesson. The variance reduction in the first split of the tree is shown here. The root node represents all cases where the overall average median home value is 23,000 dollars with a standard deviation of 9,200 dollars. The split is made on RM (the number of rooms per house). All houses that have RM less than 6.94 have an average median home value of 20,000 dollars with a standard deviation of 6,300 dollars. All houses that do not meet this rule have an average median home value of 37,000 dollars and a standard deviation of 8,900 dollars. This makes sense because houses with fewer rooms typically have a lower value than houses with more rooms.

Notice that the variability in median home value in each child node is smaller than the variability in the parent node.

Impurity Reduction Measures for Categorical and Interval Targets

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Splitting Criteria in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Leaf Purity



Copyright © SAS Institute Inc. All rights reserved.

Question: Leaf Purity

Suppose you build a decision tree to predict whether a customer makes a purchase on the internet (yes or no). A leaf has perfect purity when it contains which of the following?

- a. all events
- b. all nonevents
- c. either all events or all nonevents
- d. an equal number of events and nonevents

Answer: Leaf Purity

Suppose you build a decision tree to predict whether a customer makes a purchase on the internet (yes or no). A leaf has perfect purity when it contains which of the following?

- a. all events
- b. all nonevents
- ☒ c. either all events or all nonevents
- d. an equal number of events and nonevents

CA: c

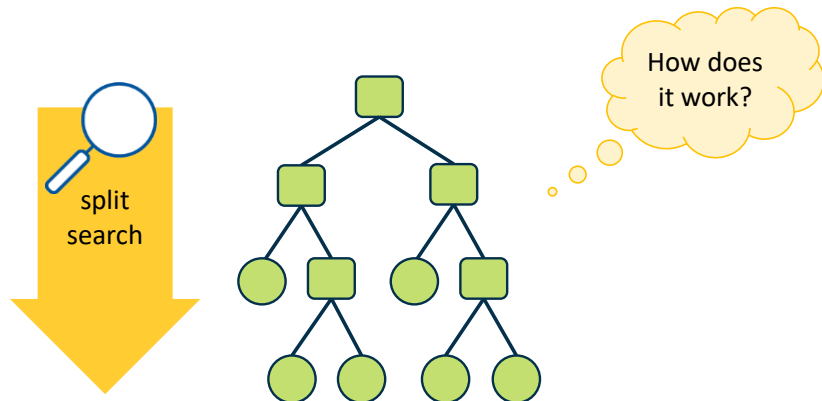
FB: Given a binary target, a leaf has perfect purity when all its cases have the same target outcome, but it does not matter whether they are events or nonevents. If multiple cases in the training data set have the same values of all the input variables but different target values, it is not possible to achieve perfect purity in the leaf that contains those cases.

Split Search

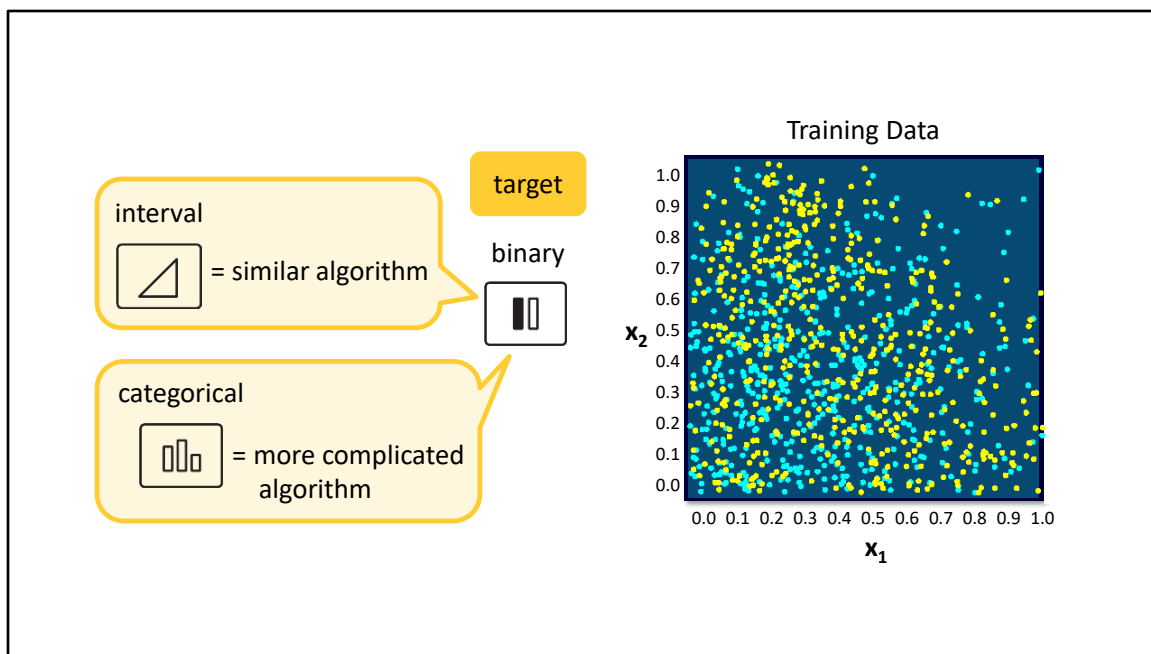


Copyright © SAS Institute Inc. All rights reserved.

Split Search

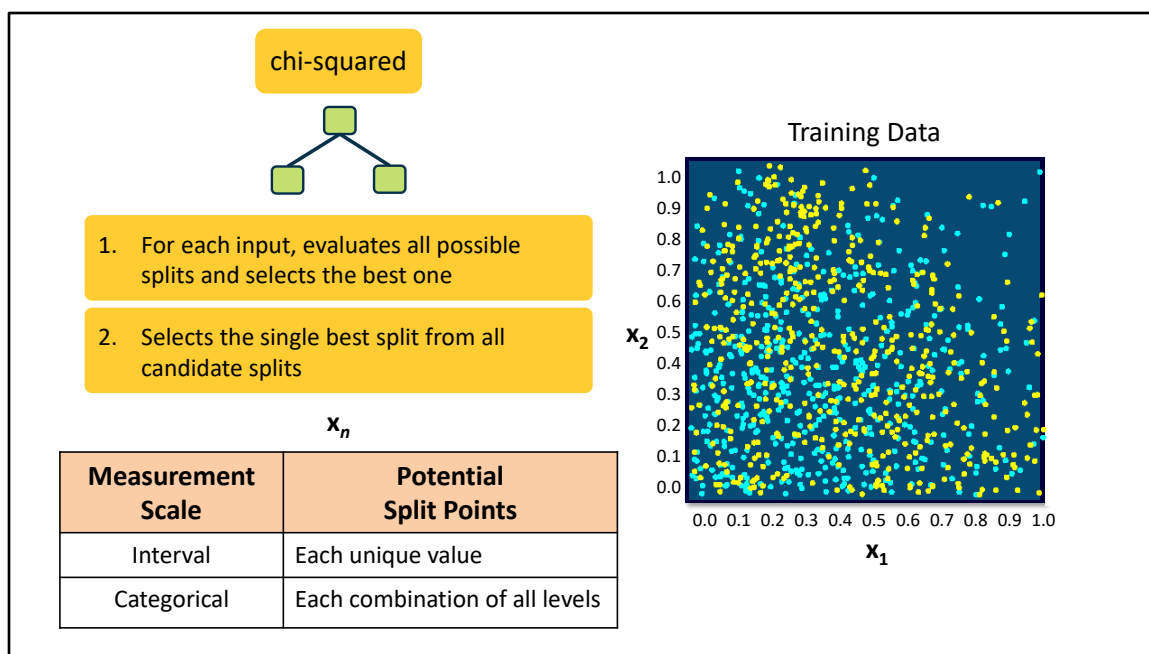


Let's look at a typical split-search process for growing a decision tree.



In this example, the training data is shown in a scatter plot. The target is binary and the two outcomes are represented as blue and yellow dots. There are two interval inputs (x_1 on the x axis, and x_2 on the y axis). The values of each input range from zero to 1. Note that the algorithm for interval targets is similar to the algorithm for binary targets.

The algorithm for categorical targets with more than two outcomes is more complicated and is beyond the scope of this course.



In this example, the split search algorithm uses chi-squared as the splitting criterion to grow a tree with 2-way splits.

For each input, the split search evaluates all possible splits and selects the best one for that input. Inputs whose splits do not meet the predefined threshold for the splitting criterion might be ignored. Then, the algorithm selects the single best split from all the candidate splits. In this example, the split search starts with the input x_1 .

An input's measurement scale determines how the algorithm selects the split point.

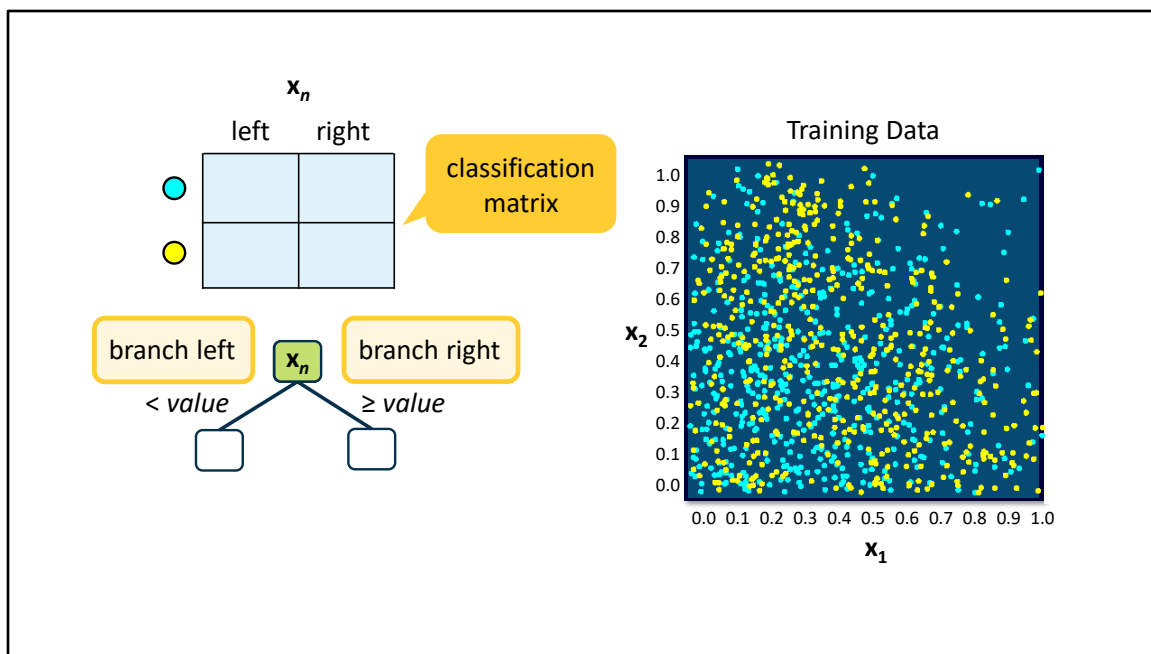
If the measurement scale of the selected input is *interval*, each unique value is a potential split point.

Interval variables with millions of values can be computationally expensive with regard to the number of split points and thus the decision tree in SAS Viya bins the numerical variables.

If the input is *categorical*, each combination of all levels is a potential split point.

If the input is ordinal, the order of the levels must be maintained when they are combined to form the split.

For now, we focus on interval inputs.

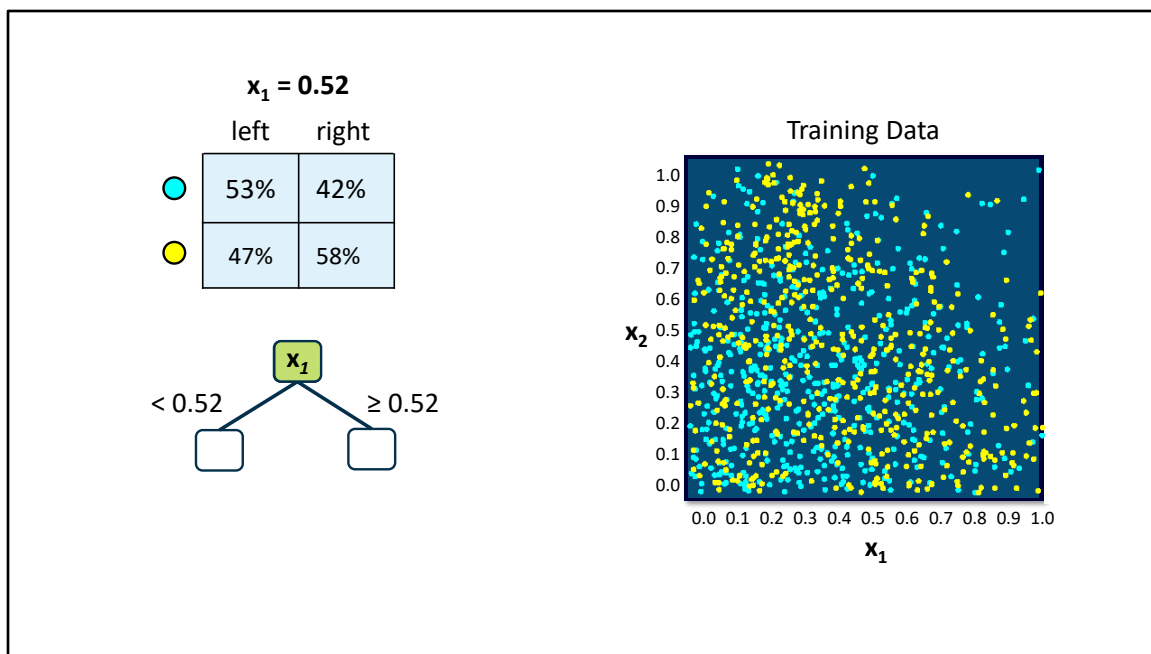


Splitting the data generates two groups, one for each branch direction.

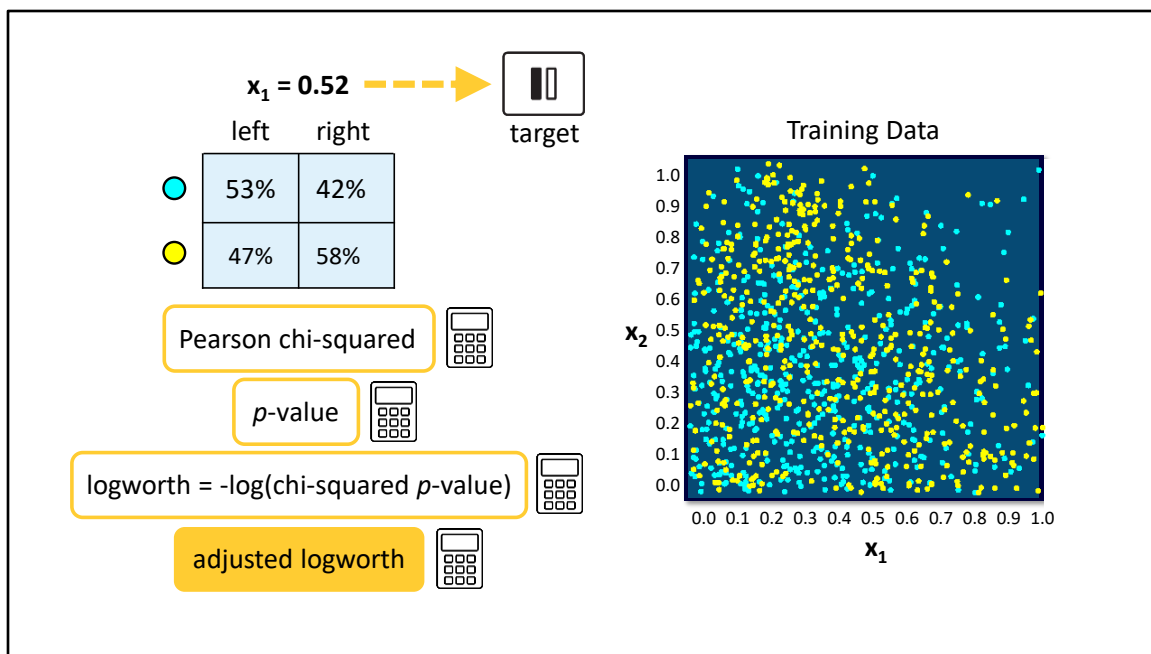
Cases with input values less than the split point are said to *branch left*.

Cases with input values greater than or equal to the split point are said to *branch right*.

The groups, combined with the target outcomes, form a two-by-two classification matrix. The columns specify the two groups defined by the split: left and right. The rows specify the target value.



Each cell contains a frequency count of the cases based on the split point and the outcome level. In the classification matrix shown here, the counts have been converted to percentages. These percentages are for one particular split based on one value of the input x_1 . The algorithm will calculate a different set of percentages for every possible split point (that is, every value of x_1).

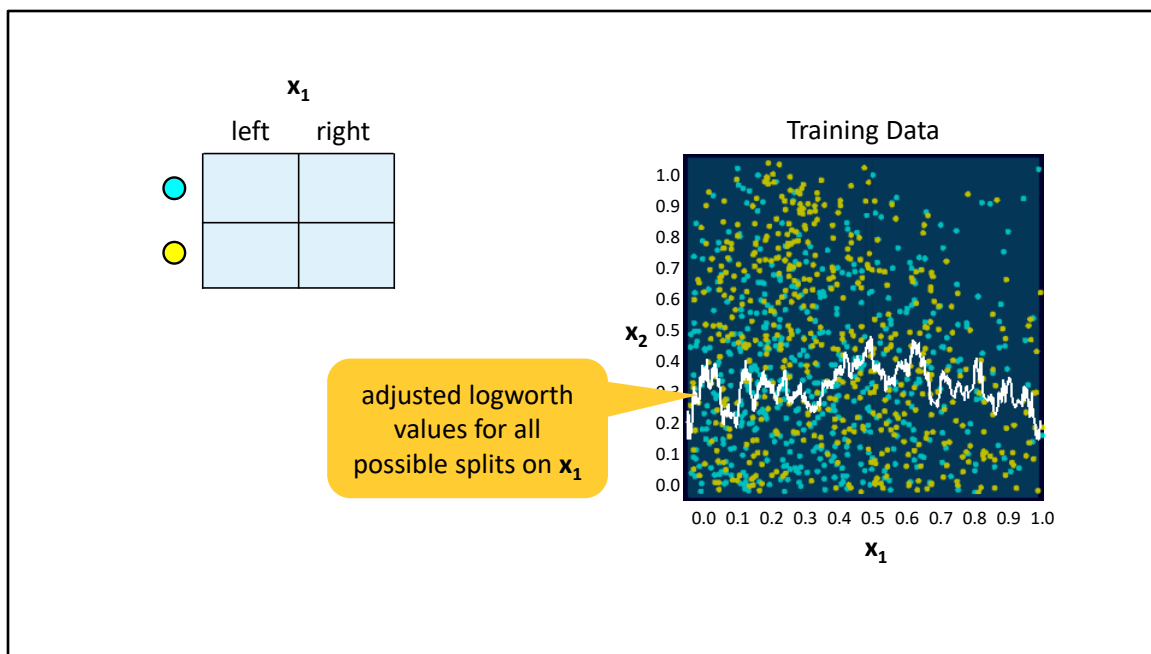


Next, for each possible split based on x_1 , the algorithm quantifies the strength of the relationship between the input and the target by using the frequencies to calculate the Pearson chi-squared statistic. Large values of the chi-squared statistic suggest that the proportion of zeros and ones in the left branch is different from the proportion in the right branch. A large difference in outcome proportions indicates a strong relationship (or dependency) and, thus, a good split.

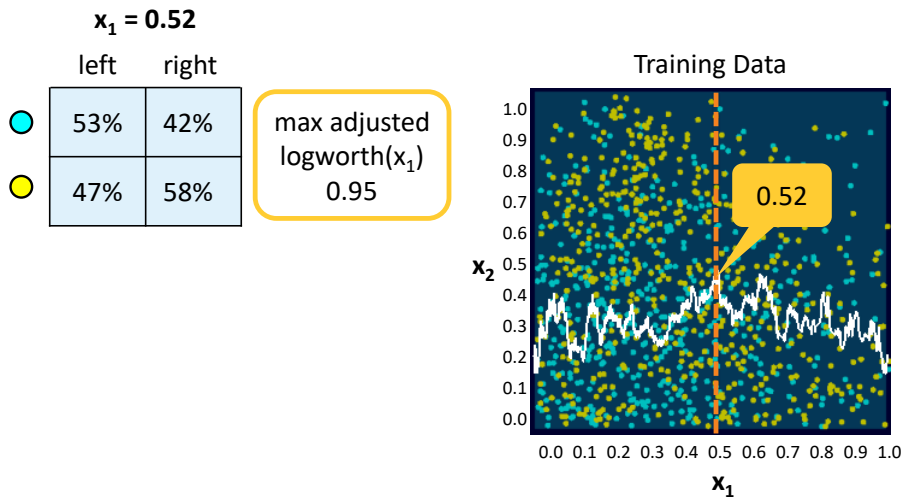
The Pearson chi-squared statistic, along with its degrees of freedom, has an associated probability value, or p -value. The p -value indicates the likelihood of obtaining the observed value of the Pearson chi-squared statistic assuming identical target proportions in each branch direction. For a large data set, this p -value can be very close to zero.

For this reason, the algorithm uses the logworth of the p -value instead to determine the quality of the split. The logworth is equal to negative log of the chi-squared p -value.

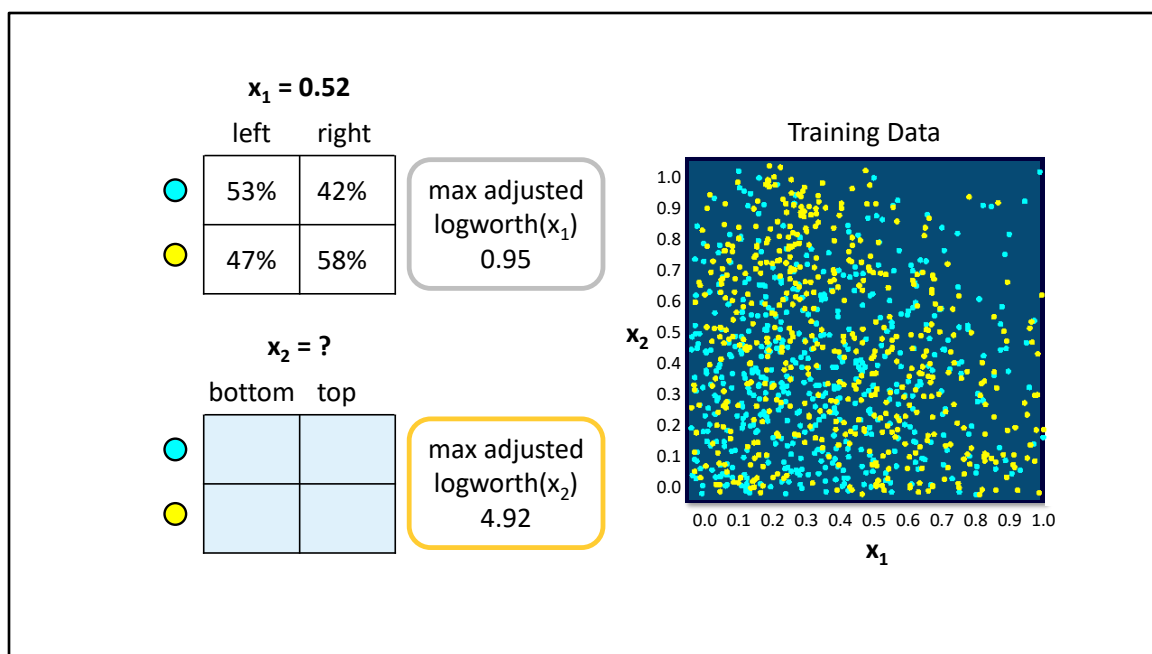
To account for issues that arise when multiple tests are performed, the algorithm adjusts the logworth values. This adjustment is similar to the Bonferroni adjustment that statisticians use to overcome the issues that arise when performing multiple tests. You learn more about the Bonferroni adjustment later.



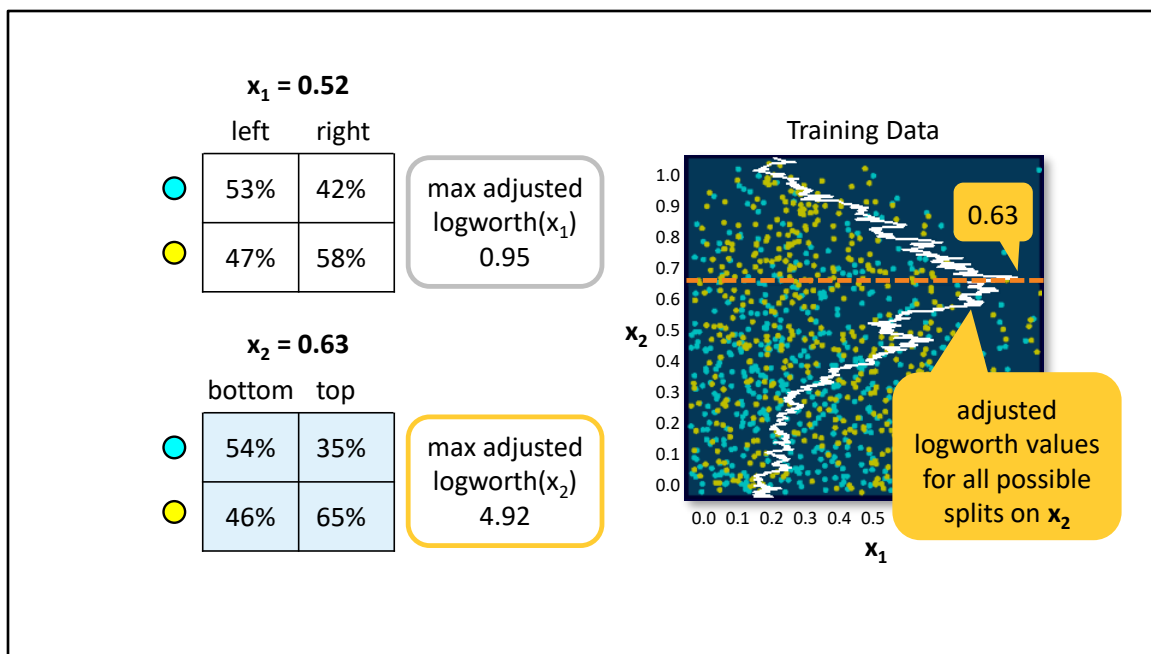
In the scatter plot of the training data, the horizontal white jagged line represents the adjusted logworth values for all possible splits on x_1 .



The best split for an input is the split that yields the maximum adjusted logworth value. Here, the vertical line in the scatter plot shows that the maximum adjusted logworth for x_1 , which is 0.95, is based on the input value 0.52.

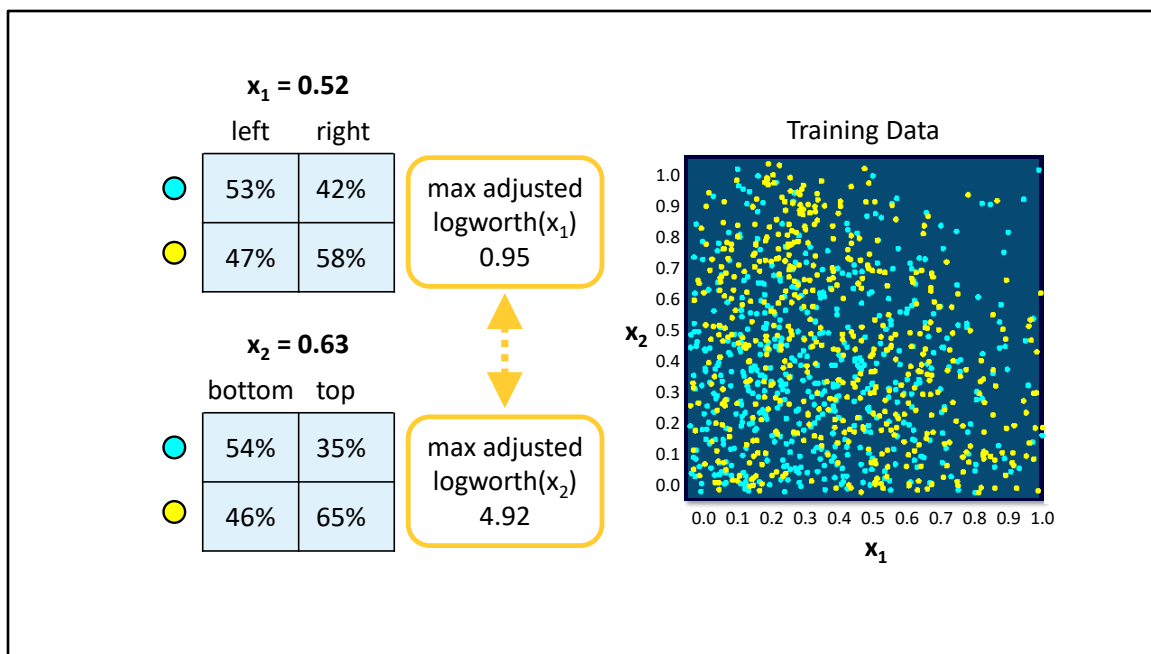


The split search process is repeated for every input in the training data. Inputs whose adjusted logworth fails to exceed the splitting criterion's threshold are excluded from consideration.

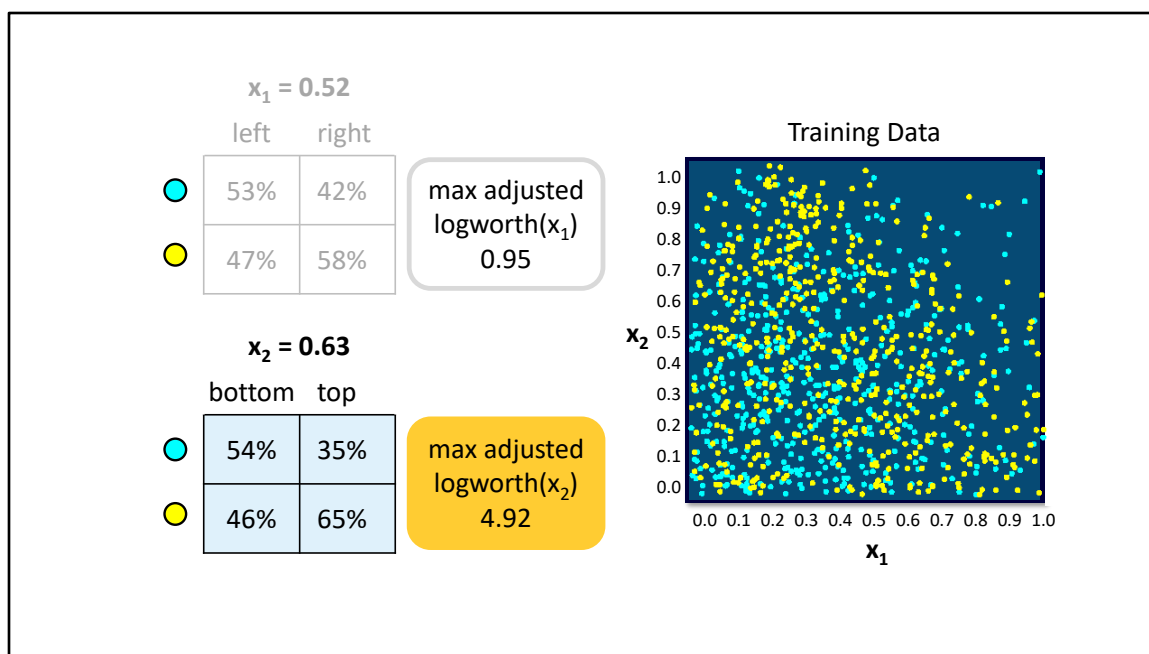


In the scatter plot for this example, the vertical white jagged line represents the adjusted logworth values for all possible splits on x_2 .

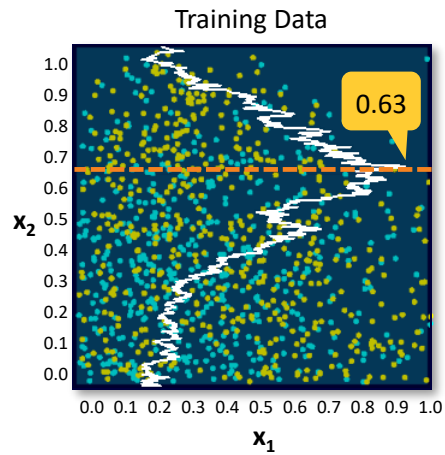
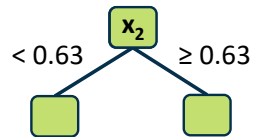
The horizontal line in the scatter plot shows that the maximum adjusted logworth for x_2 , which is 4.92, is based on the input value 0.63.



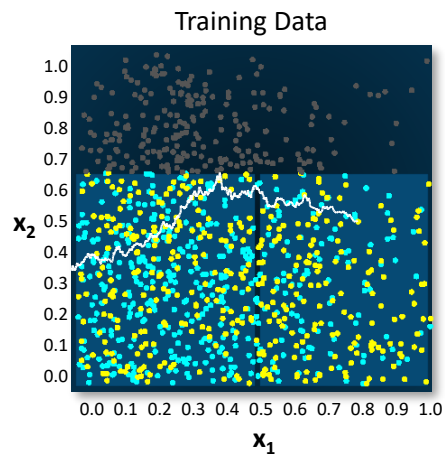
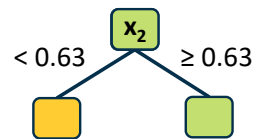
After you determine the best split for every input, the split search algorithm compares the maximum adjusted logworth values across all of the inputs. The split with the highest adjusted logworth becomes the splitting rule.



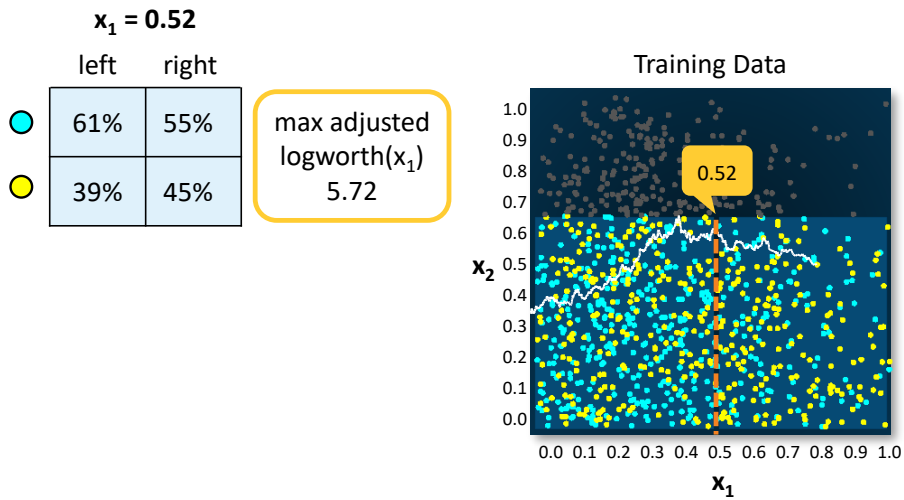
In this example, the split on x_2 has the highest maximum adjusted logworth value, so it becomes the basis for the splitting rule.



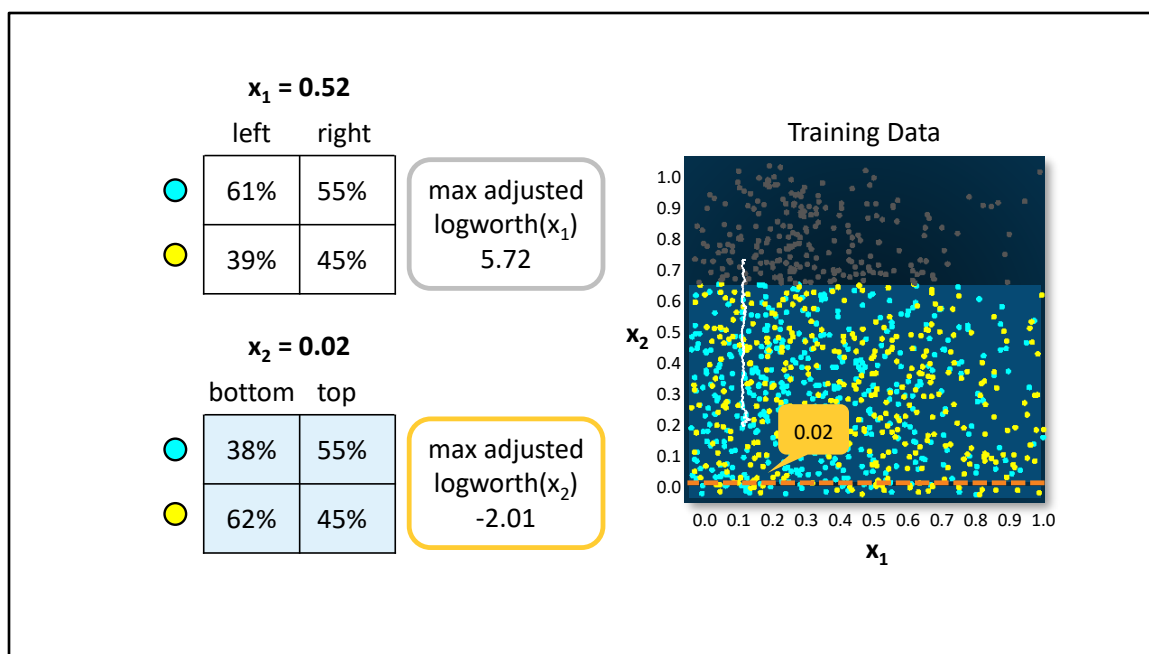
The algorithm uses the splitting rule to separate the training data into two groups. In this example, if x_2 is less than 0.63, then the case branches left. Else if x_2 is greater than or equal to 0.63, the case branches right.



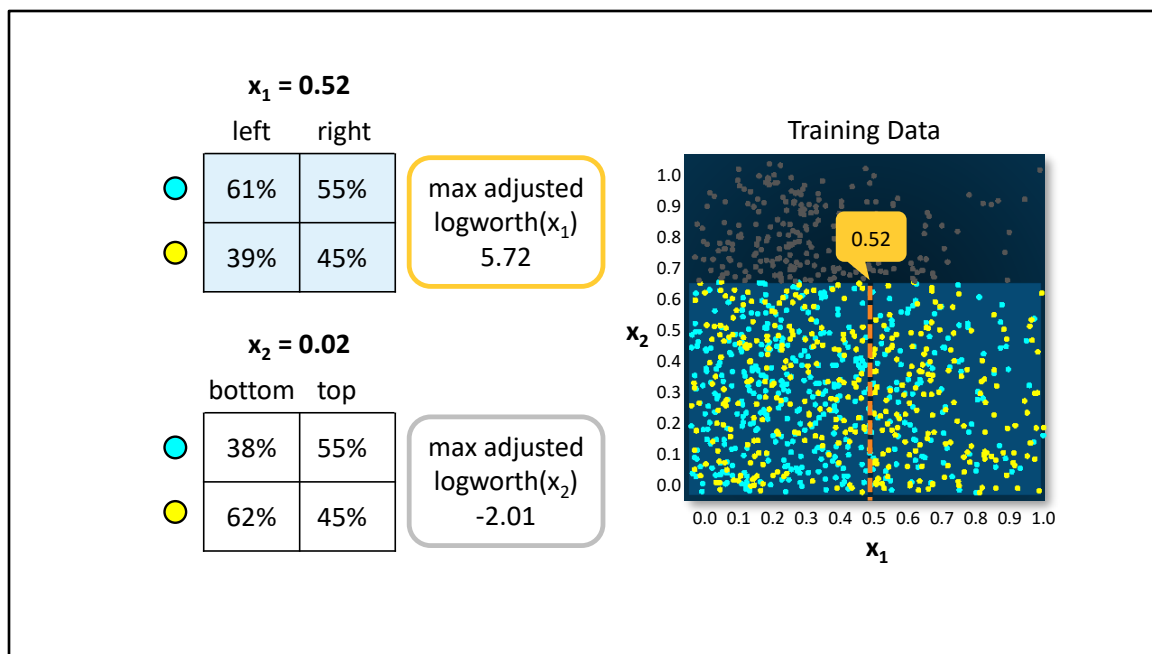
The algorithm moves on to one of the groups in the first generation, and repeats the split search process for that group.



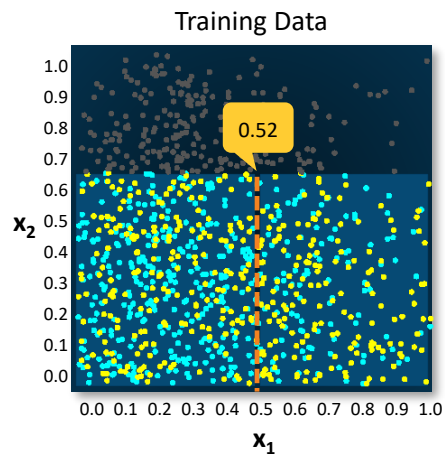
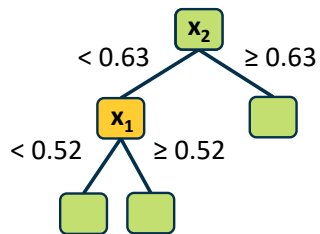
In this scatter plot, the horizontal white jagged line shows the adjusted logworth values of x_1 . The vertical line shows the value of x_1 , which is 0.52, that corresponds with the maximum adjusted logworth value 5.72. This is the optimal split point for x_1 .



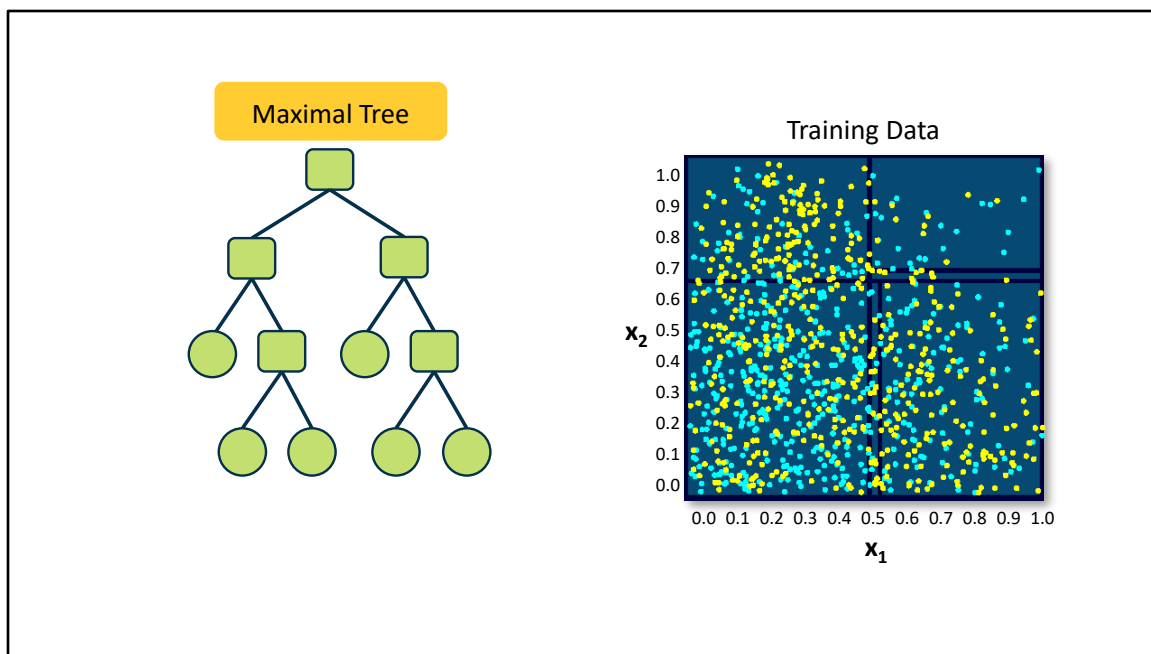
The optimal split point for x_2 is likewise determined. The maximum adjusted logworth, which is -2.01, is based on the x_2 value of 0.02. You might be surprised that the logworth of the x_2 split is negative. However, this results from several adjustments made to the logworth calculation, which you learn about later.



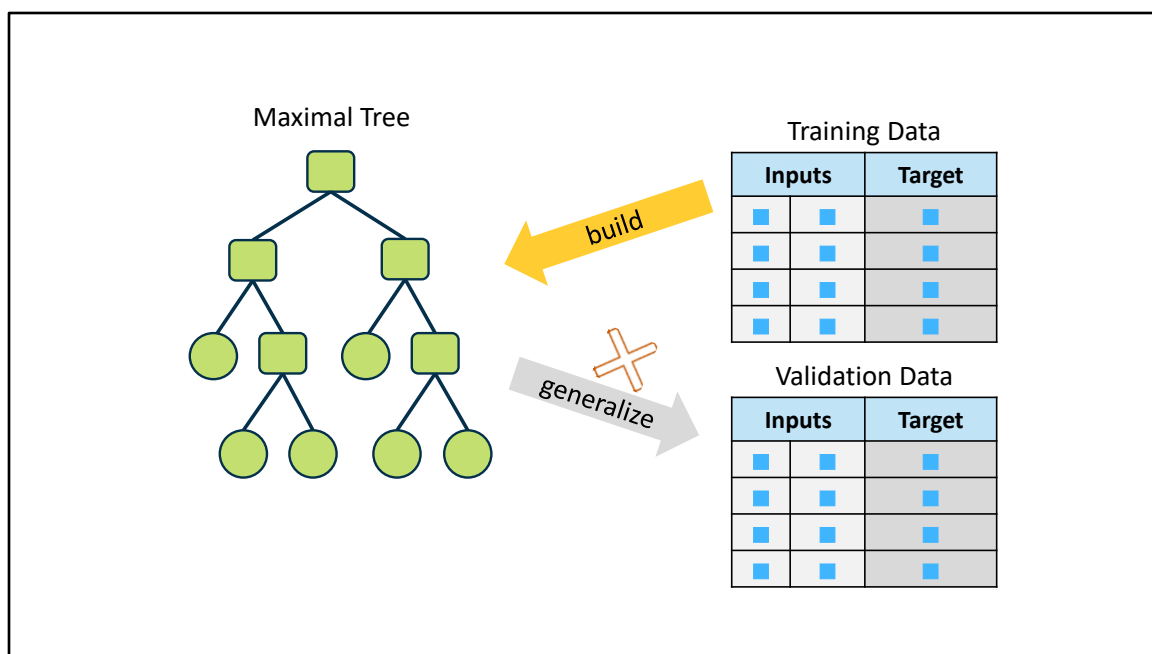
The x_1 maximum adjusted logworth, which is 5.72, is higher than the maximum adjusted logworth for x_2 , so the selected splitting rule is based on the associated value of x_1 .



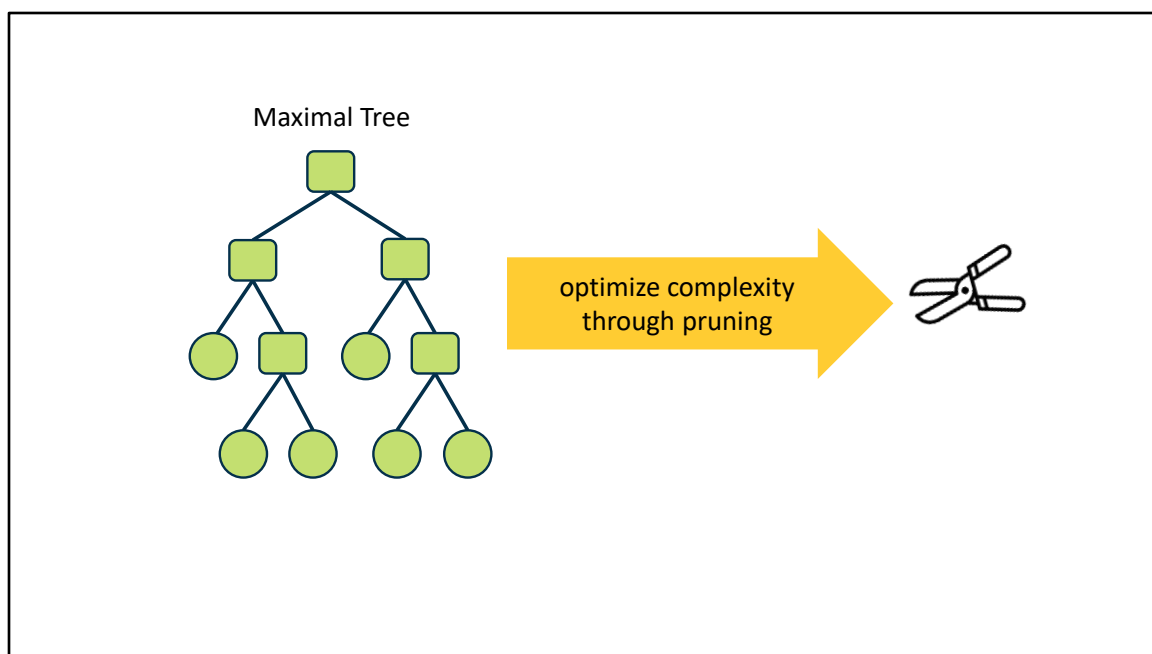
Now the algorithm uses the adjusted splitting rule to separate the data into two groups.



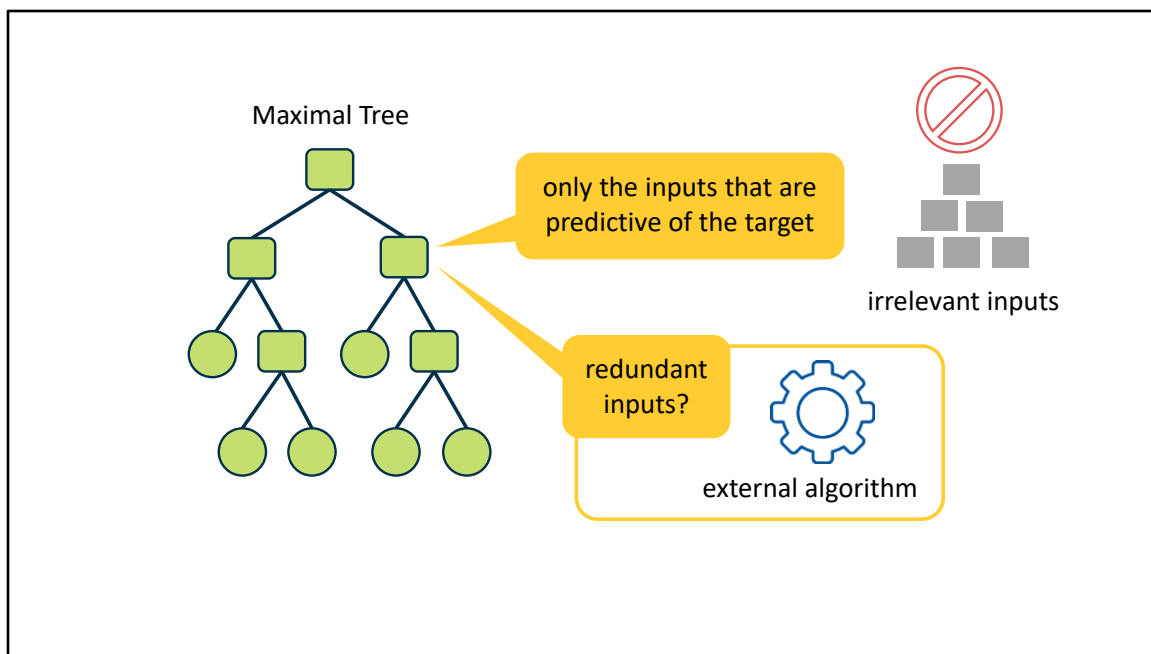
Unless another stopping rule is triggered first (such as a limit on the tree's depth), the split-search process repeats in each leaf until there are no more splits whose adjusted logworth exceeds the adjusted threshold. This stops the tree's growth. The result of the split-search process is known as the *maximal tree*.



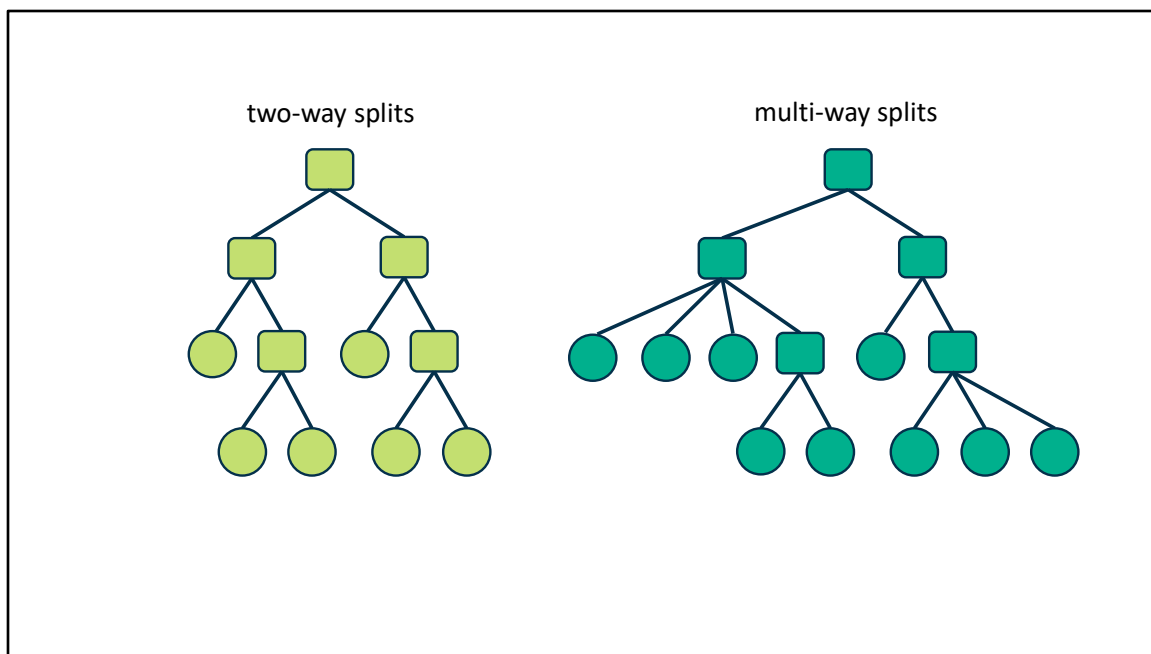
The maximal tree is built exclusively on the training data so it likely will not generalize well on the validation data.



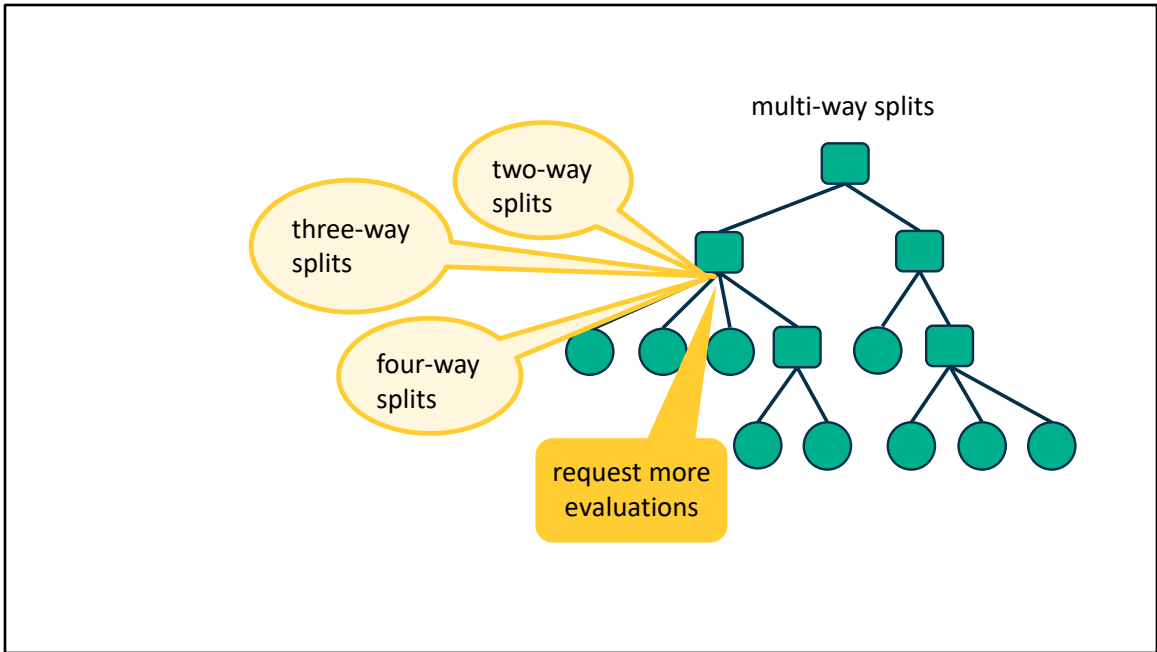
The maximal tree is the starting place for optimizing the complexity of the model, through a process called pruning. You learn about pruning later.



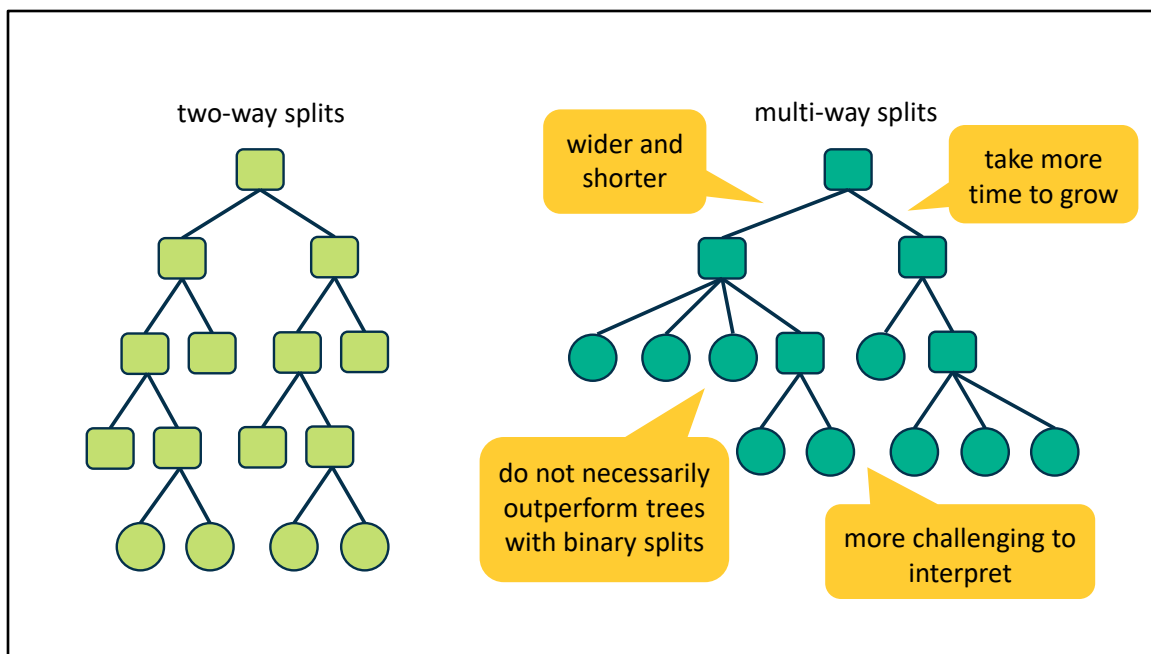
The maximal tree contains only the inputs that are predictive of the target (that is, the inputs that appear in splitting rules). This is the first point in the process of building a decision tree where irrelevant inputs are excluded. However, trees have no built-in method for ignoring redundant inputs. To manage input redundancy, you must use an algorithm that is external to the tree. For example, you can use the Variable Selection node.



Although this example shows a decision tree with two-way splits, we could have created a decision tree with multi-way splits where the values of the inputs are partitioned into disjoint ranges.



Multi-way splits request more evaluations for the candidate splits because all inputs in all n -way splits must be considered. For example, in 4-way splits, all possible candidates for two-way splits, three-way splits, and four-way splits are evaluated.



Trees with multi-way splits tend to be wider and shorter than trees with binary splits. However, research has shown that trees with multi-way splits do not necessarily outperform trees with binary splits. Trees with multi-way splits also take more time to grow, and they can be more challenging to interpret.

Question: Number of Splits



Copyright © SAS Institute Inc. All rights reserved.

Question: Number of Splits

If you build a decision tree model to predict a binary target, the rules in that decision tree are limited to two-way splits.

- a. True
- b. False

Answer: Number of Splits

If you build a decision tree model to predict a binary target, the rules in that decision tree are limited to two-way splits.

- a. True
- ☒ b. False

CA: b

FB: This statement is false. The number of splits per rule does not depend on the type of target. If the split is based on binary input (for example, whether a person owns a home), a two-way split is the only possibility. As you learn later, you can specify the maximum number of splits that Model Studio uses to build a decision tree.

Think About It



Copyright © SAS Institute Inc. All rights reserved.

Discussion

How do decision trees address the curse of dimensionality?



FB: The split search process reduces the number of inputs in the model by eliminating irrelevant inputs. Irrelevant inputs do not appear in any splitting rules in the decision tree.

Adjustments in a Split Search

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Missing Values in Decision Trees in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Modifying the Recursive Partitioning Parameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Surrogate Splits

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Calculating Variable Importance for Surrogate Splits

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



3.4 Modifying the Model: Pruning



Optimizing the Complexity of a Decision Tree Model



Copyright © SAS Institute Inc. All rights reserved.

Optimizing the Complexity of a Decision Tree Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- ✓ Improve the model.
 - Optimize the complexity of the model.
 - Regularize and tune the hyperparameters of the model.
 - Build ensemble models.

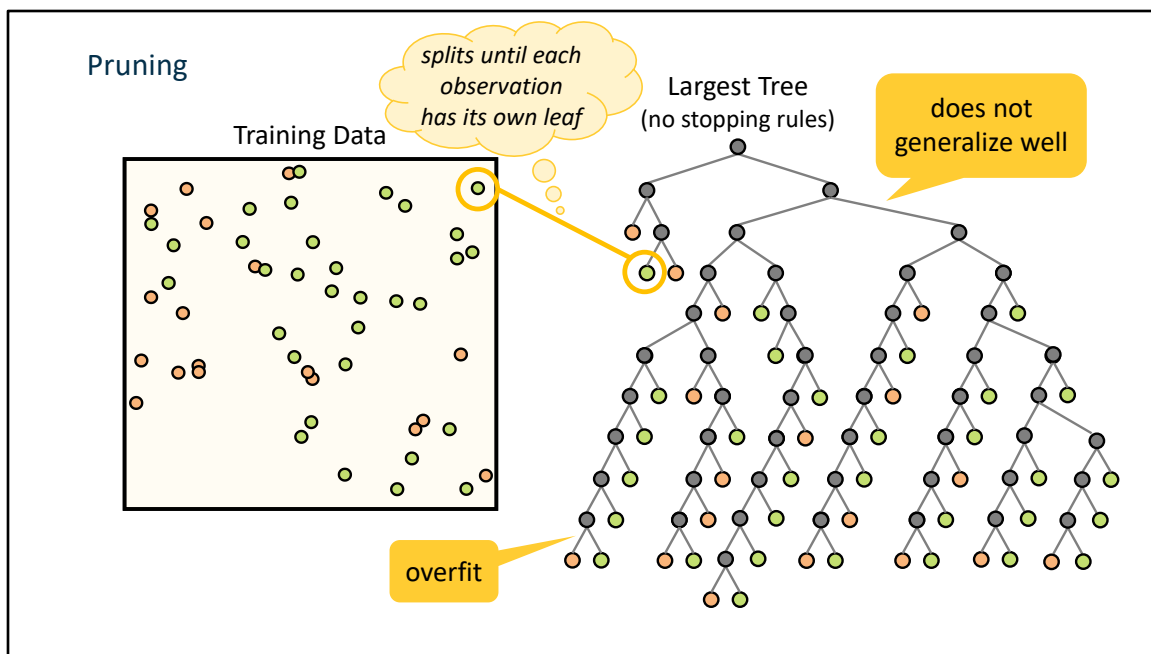
pruning

Remember that the goal of predictive modeling is to build models that can be applied to future cases so that you can make business decisions. With this goal in mind, the next task in the discovery phase for decision trees is optimizing complexity. Optimizing complexity starts with the maximal tree and then removes branches using a process called pruning. The optimal model is the model that generalizes to new data and is a subset of the maximal tree. The optimal tree has the best balance between bias and variance.

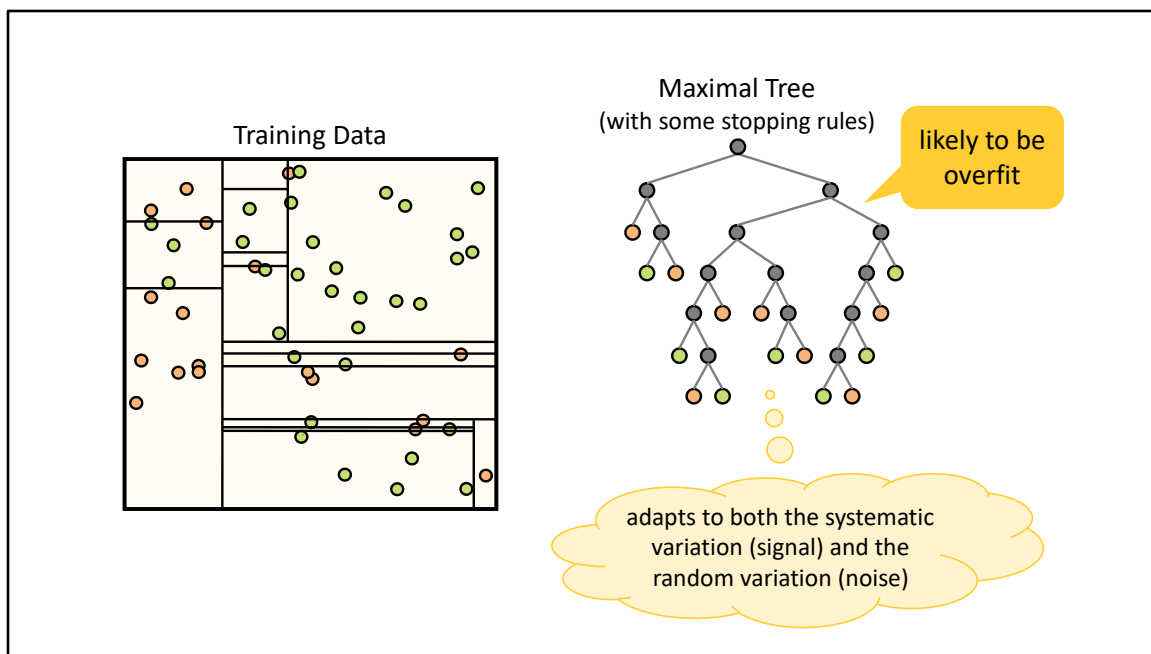
Pruning



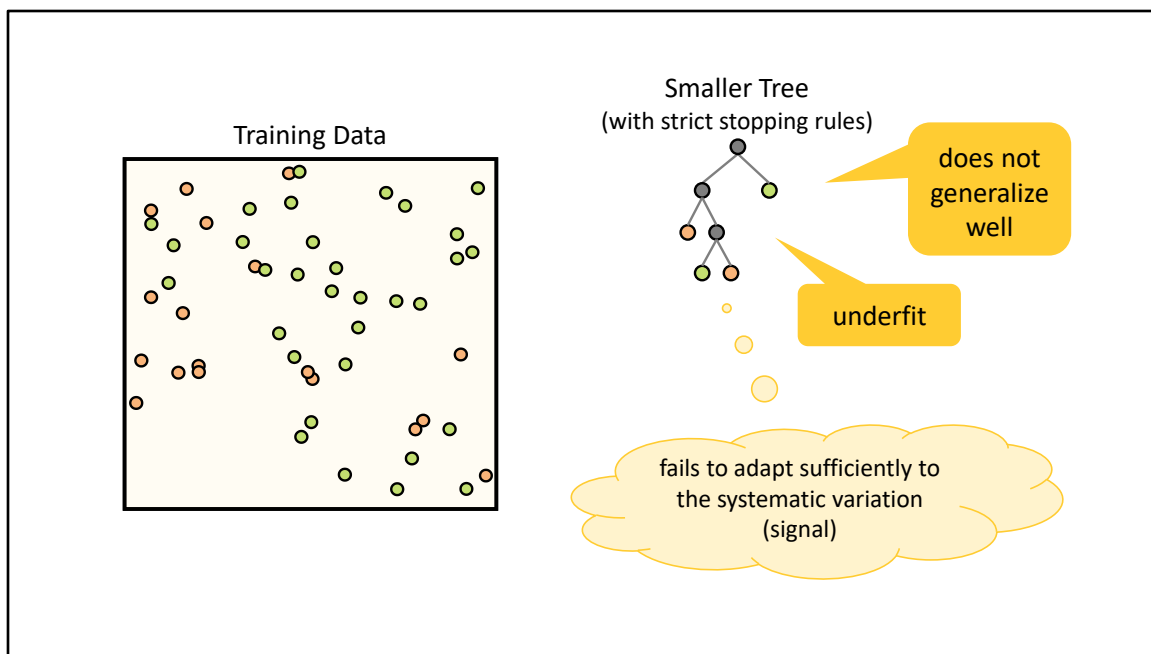
Copyright © SAS Institute Inc. All rights reserved.



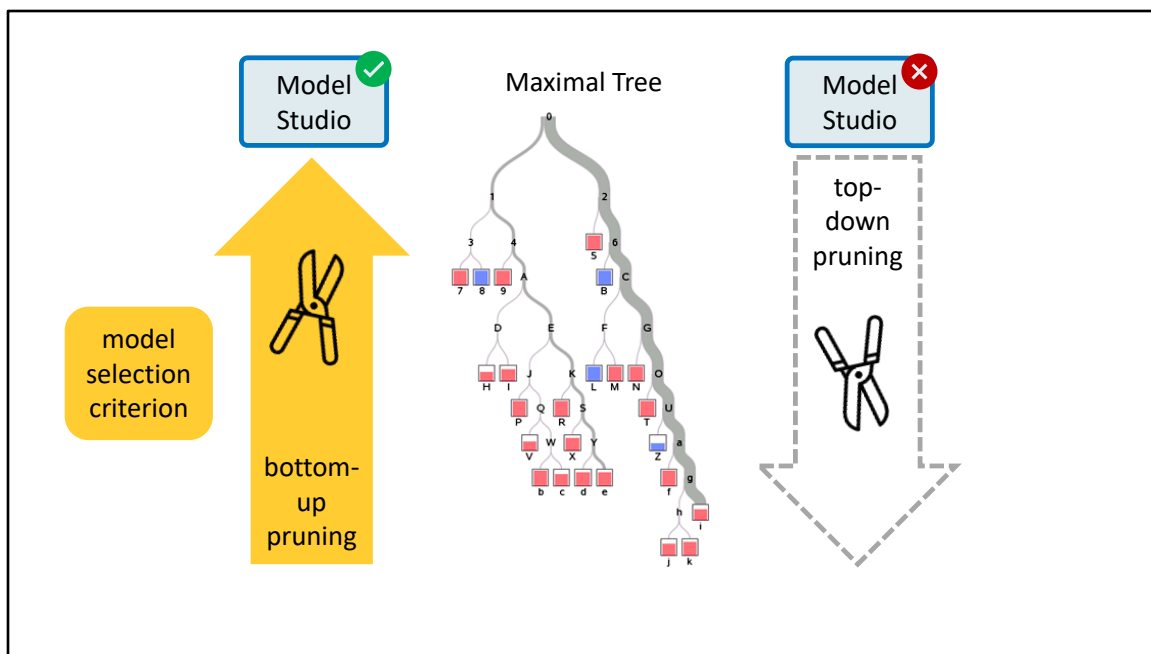
A decision tree algorithm can easily overfit the data. If recursive partitioning continues to split the data until each observation has its own leaf (that is, if there are no stopping rules), it will be 100% accurate for every observation in the training data. This tree does not generalize well to new data.



The maximal tree that results from recursive partitioning is smaller than the largest possible tree but is still likely to be overfit. The maximal tree adapts to both the systematic variation of the target (or the signal) and the random variation (or the noise).

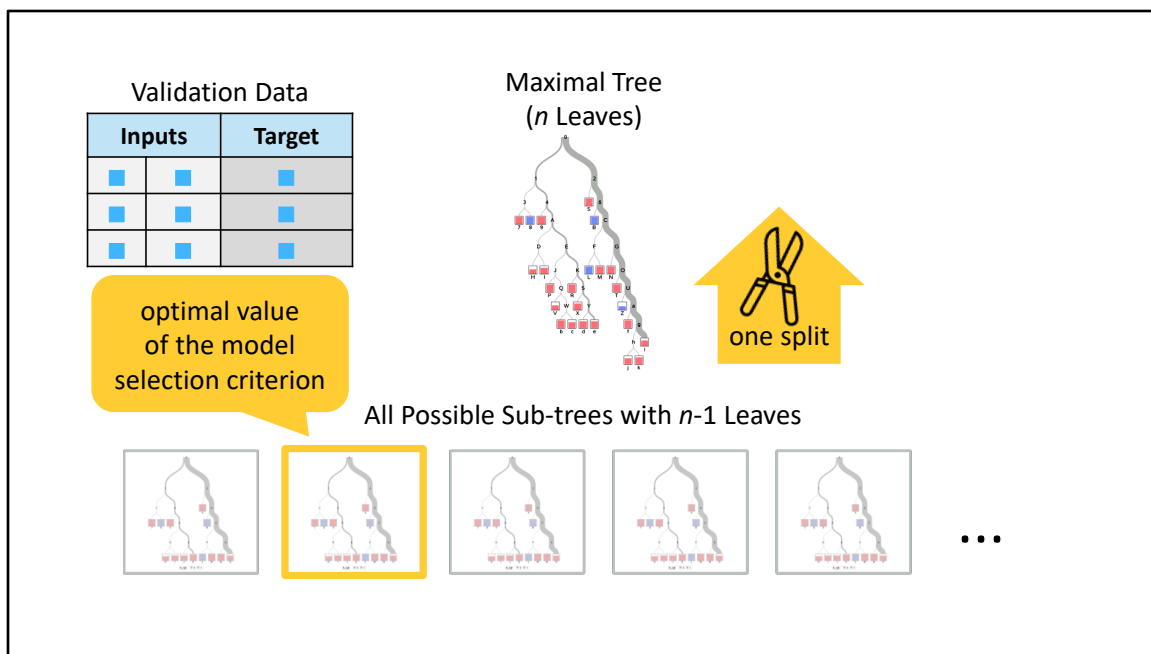


At the other extreme, a small tree with only a few branches might underfit the data. It might fail to adapt sufficiently to the signal, which usually results in poor generalization.



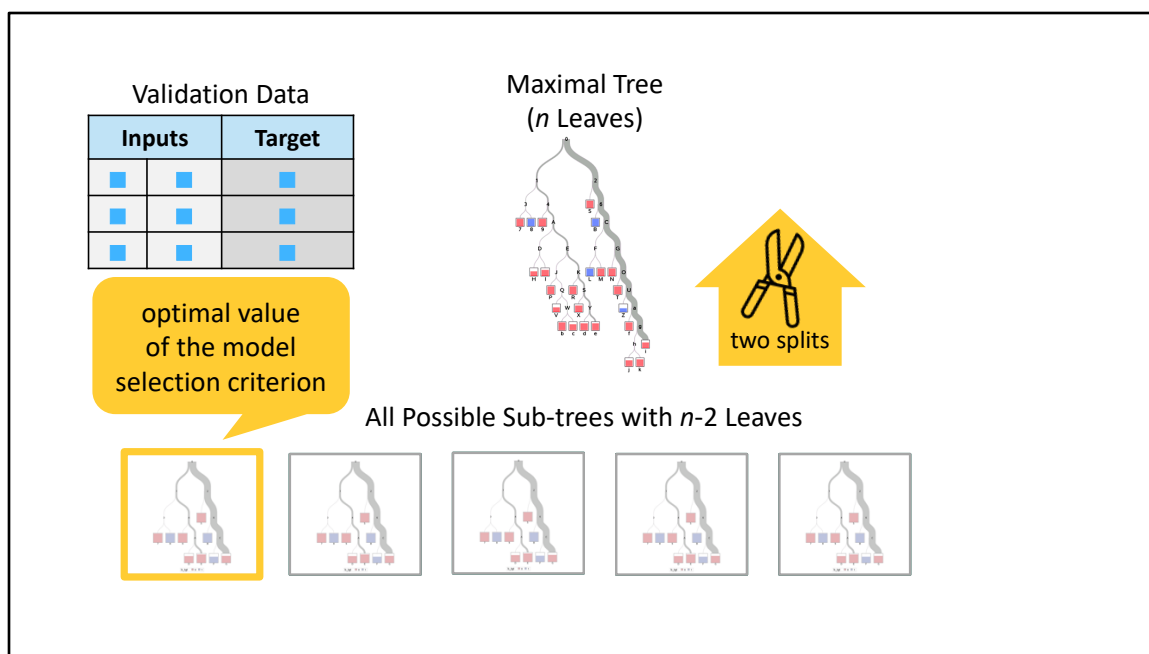
To build the optimal tree, which is neither too large nor too small, Model Studio starts with the maximal tree and uses a process called bottom-up pruning to remove branches. Bottom-up pruning is also known as post pruning or retrospective pruning. The other type of pruning, which is top-down pruning, is not available in Model Studio. Top-down pruning is usually faster but is considered less effective than bottom-up pruning.

Bottom-up pruning starts at the leaf level and removes branches in a backward fashion using a model selection criterion.



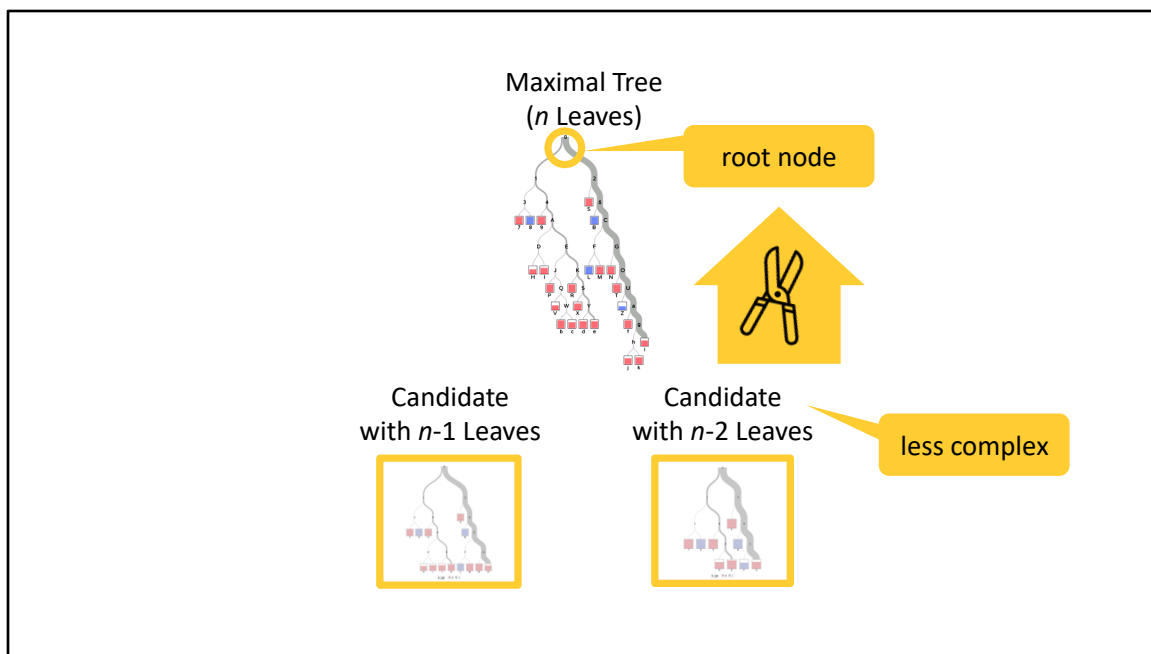
Suppose the maximal tree has n leaves. First, the pruning algorithm evaluates all possible sub-trees consisting of $n-1$ leaves. To do this, the algorithm removes (or prunes) one split.

Out of all possible sub-trees with $n-1$ leaves, the algorithm selects the sub-tree with the optimal value of the model selection criterion on the validation data. The selected sub-tree is the candidate from all sub-trees with $n-1$ leaves.



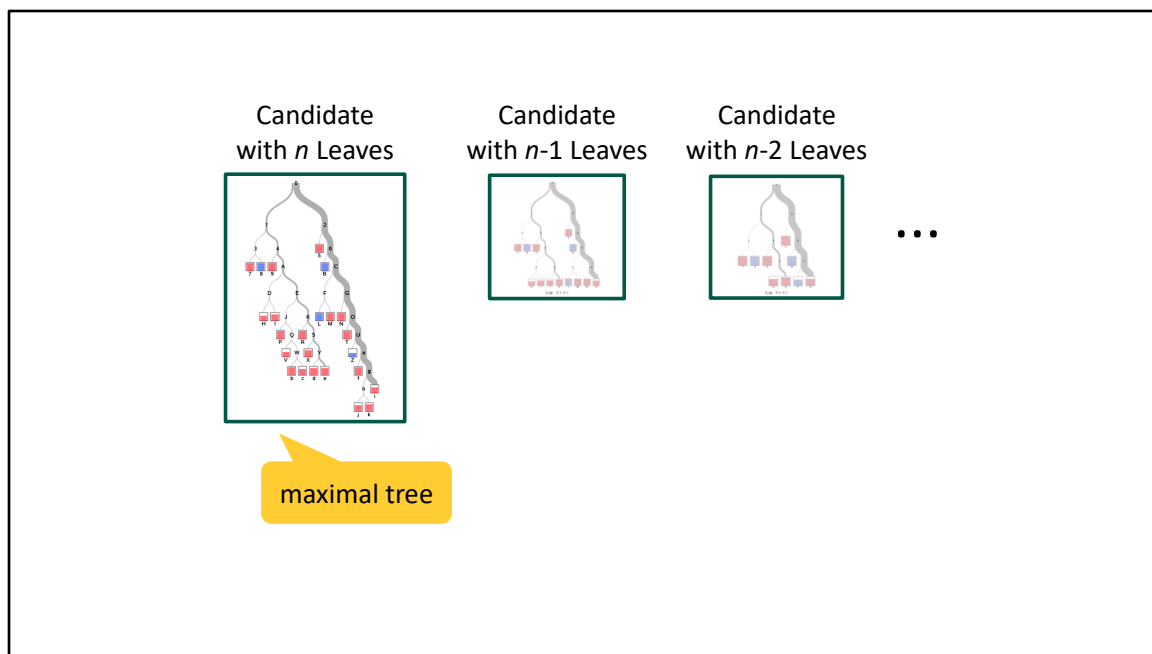
Then the pruning algorithm returns to the maximal tree.

This time, the algorithm evaluates all possible sub-trees consisting of $n-2$ leaves by pruning two splits and selecting the candidate from all sub-trees with $n-2$ leaves.

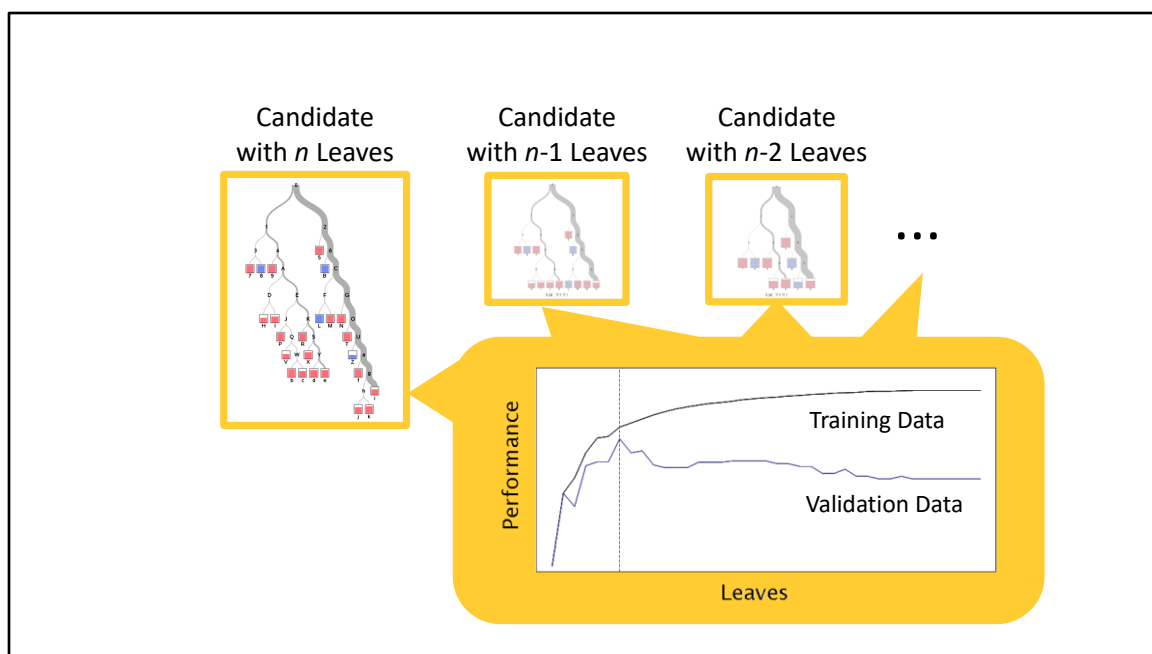


The candidate with $n-2$ leaves is less complex than the candidate with $n-1$ leaves.

The pruning algorithm continues in this way until it reaches the root node.



At this point, the algorithm has created a sequence of candidate sub-trees, ranging from the most complex (that is, the maximal tree) to the least complex (that is, the root node). The candidate sub-trees are not necessarily nested within each other.



Finally, from the sequence of candidate sub-trees, the pruning algorithm selects the sub-tree with the best performance on the model selection criterion based on the validation data. If multiple sub-trees have the same performance, the algorithm selects the least complex sub-tree from this group.

Bottom-Up Pruning Requirements

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Pruning Options in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Modifying the Pruning Parameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Pruning



Copyright © SAS Institute Inc. All rights reserved.

Question: Pruning

Which of the following statements is true about pruning decision trees?

- a. The goal of pruning is a tree with low bias and high variance.
- b. Pruning starts by identifying a sequence of candidate sub-trees, one for each possible number of leaves, and then selects the best of the candidates.
- c. In bottom-up pruning, the sub-tree with the best performance on training data is selected.
- d. Top-down pruning is usually more time-consuming but is considered more effective than bottom-up pruning.

Answer: Pruning

Which of the following statements is true about pruning decision trees?

- a. The goal of pruning is a tree with low bias and high variance.
- ☒ b. Pruning starts by identifying a sequence of candidate sub-trees, one for each possible number of leaves, and then selects the best of the candidates.
- c. In bottom-up pruning, the sub-tree with the best performance on training data is selected.
- d. Top-down pruning is usually more time-consuming but is considered more effective than bottom-up pruning.

CA: b

FB: Accuracy is obtained by multiplying the proportion of observations falling into each leaf by the proportion of those correctly classified in the leaf and then summing across all leaves.

Regularizing and Tuning the Hyperparameters of a Machine Learning Model



Copyright © SAS Institute Inc. All rights reserved.

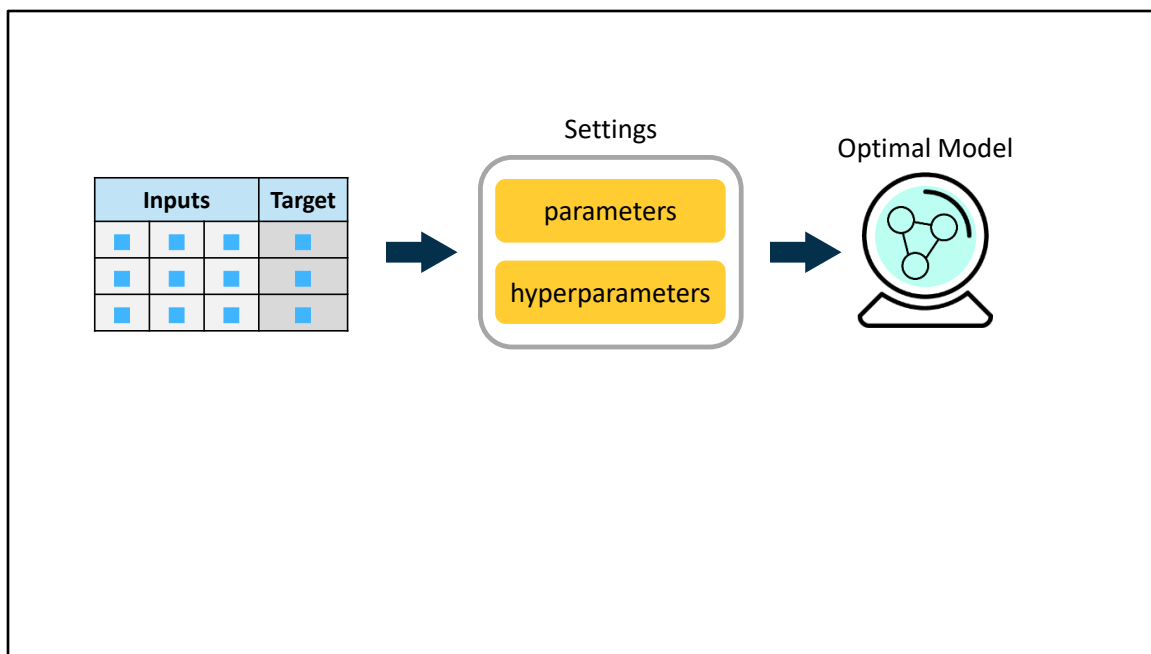
Regularizing and Tuning the Hyperparameters of a Machine Learning Model



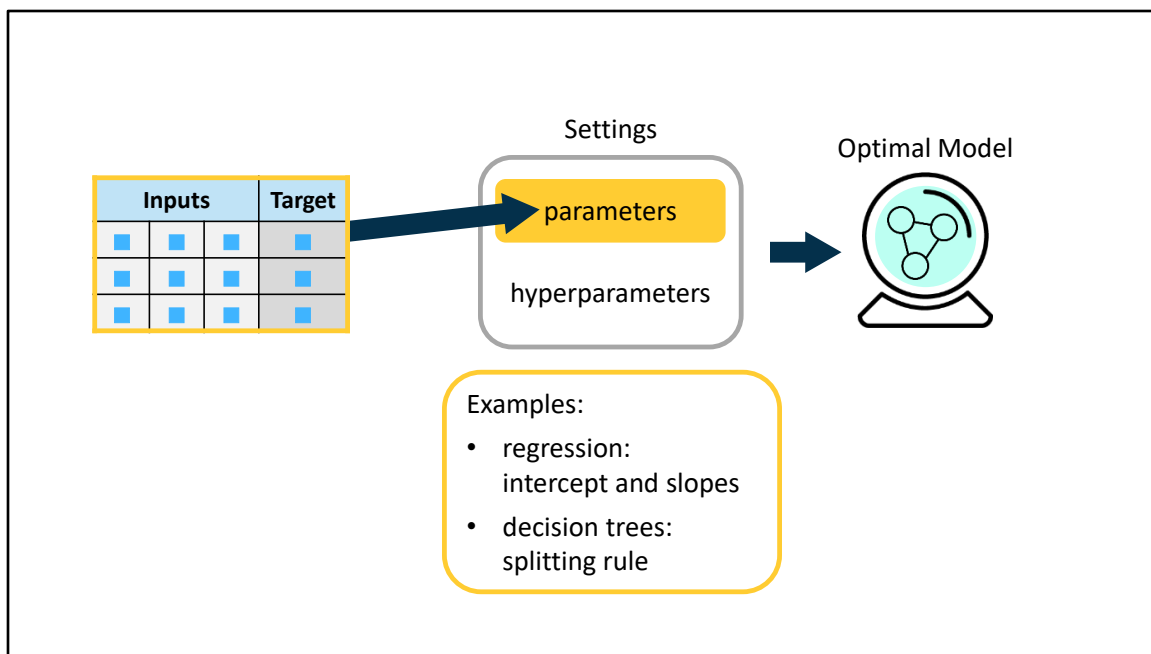
Essential Discovery Tasks

- ✓ Select an algorithm.
- ✓ Improve the model.
- ✓ Optimize the complexity of the model.
 - Regularize and tune the hyperparameters of the model.
 - Build ensemble models.

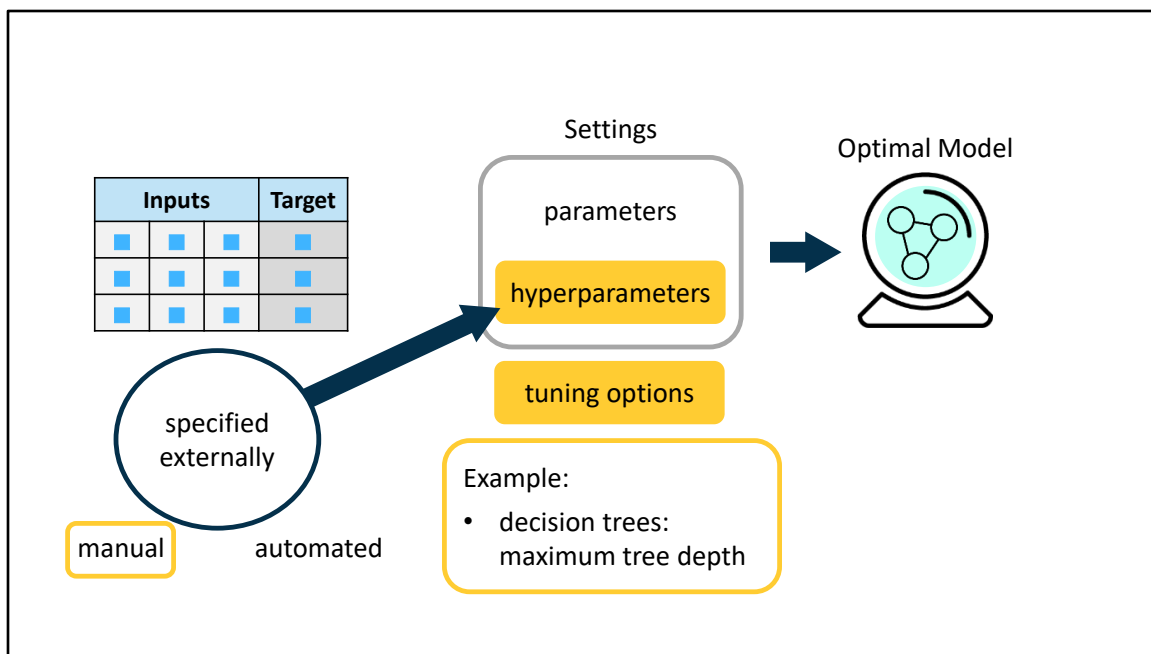
Regularizing and tuning the hyperparameters of the model is an important discovery task.



As with any predictive model, when you build a machine learning model, you want to find the model settings that result in the best model performance and generalization. Settings include both the parameters and the hyperparameters of the model. So, what is the difference?

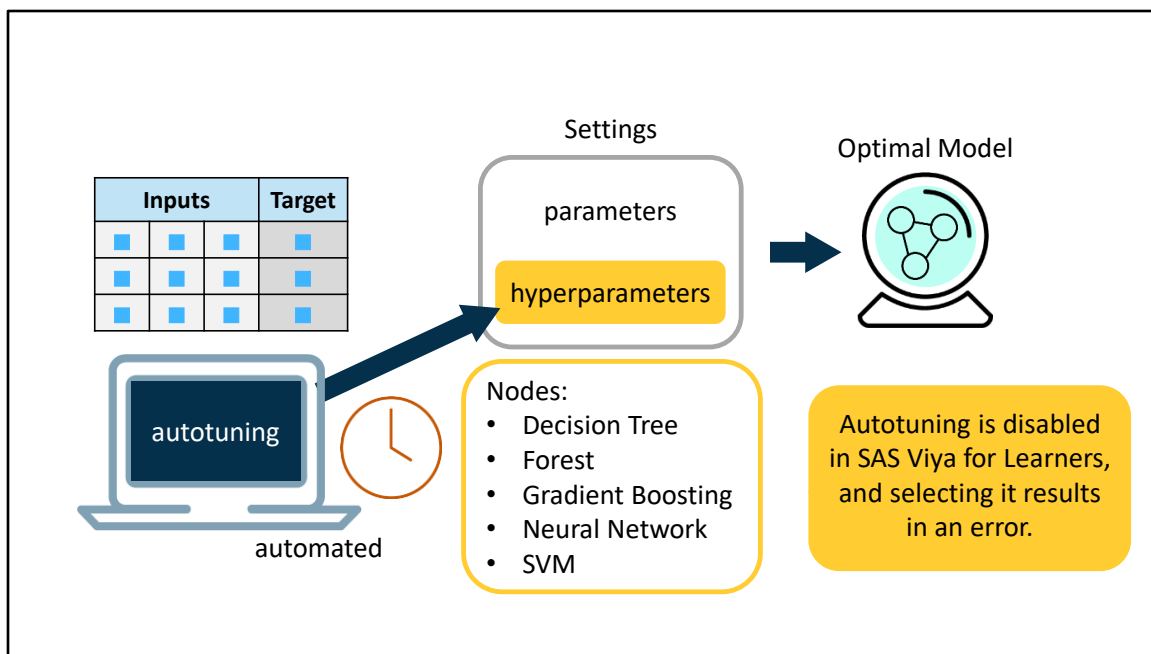


A model *parameter* is a part of the model whose value can be estimated from the data during model training. For models that can be defined by a mathematical equation that provides a prediction, a parameter is typically a variable in that equation. For example, in a regression model, the intercept and the slopes are parameters that come from the associated mathematical equation. For decision trees, a splitting rule is an example of a parameter.



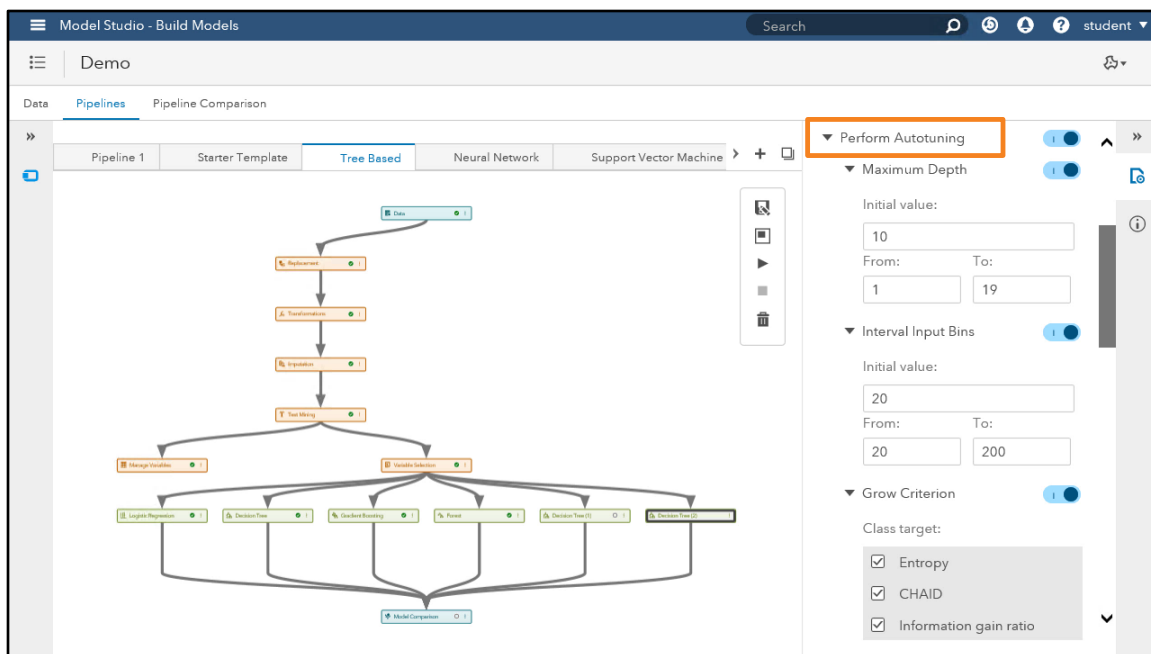
A model *hyperparameter* is a variable that is used to find the optimal model but whose value cannot be estimated from the data. Instead, these values must be specified externally, either manually or through an automated process. Hyperparameters are sometimes called tuning options. For decision trees, maximum tree depth is an example of a hyperparameter.

Settings for the hyperparameters can significantly influence the resulting accuracy of the predictive models. However, there are no clear defaults that work well for different data sets. It is possible to discover the best hyperparameters manually (for example, based on experience, trial and error, and heuristics), but this is time-consuming.



As an alternative, machine learning functionality in SAS Viya provides an automated option for hyperparameter optimization, which is called autotuning. Autotuning capabilities are available for the following nodes: Decision Tree, Forest, Gradient Boosting, Neural Network, and SVM, as well as other modeling nodes that are not discussed in this course. In general, autotuning searches for the best combination of hyperparameters specific to each modeling algorithm.

When you decide whether to perform autotuning, keep in mind that autotuning can substantially increase run time.



To turn on autotuning in Model Studio, you select the **Perform Autotuning** option in the properties pane for the selected modeling node. You can then specify the hyperparameters that you want to tune. The available hyperparameters are specific to each model. Autotuning involves building and evaluating models based on a range of values for each of the selected hyperparameters.

There is no guarantee that autotuning will arrive at the best model. However, autotuning can provide a good starting place for you to use your knowledge and domain expertise to improve the model.

Hyperparameter Optimization Approaches

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Autotuning Options for Decision Trees in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Using the Best Hyperparameter Values for a Model

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



(Optional) Practice: Build a Decision Tree Model Using Autotuning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



3.5 Building and Modifying Ensembles of Trees



Building Ensemble Models



Copyright © SAS Institute Inc. All rights reserved.

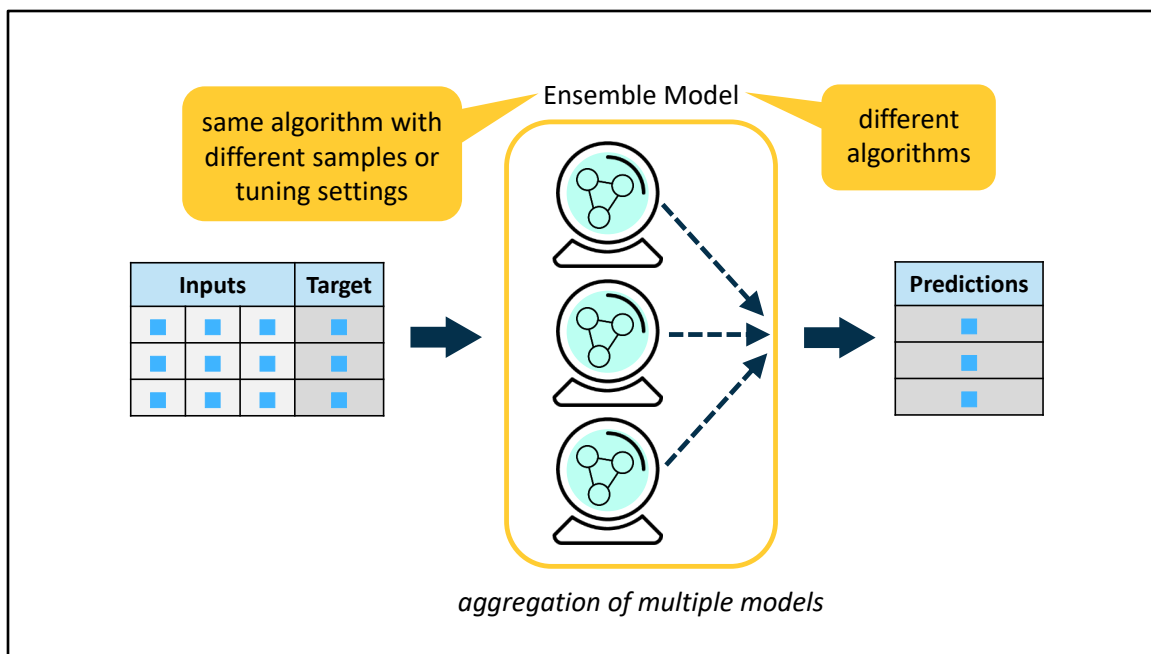
Building Ensemble Models



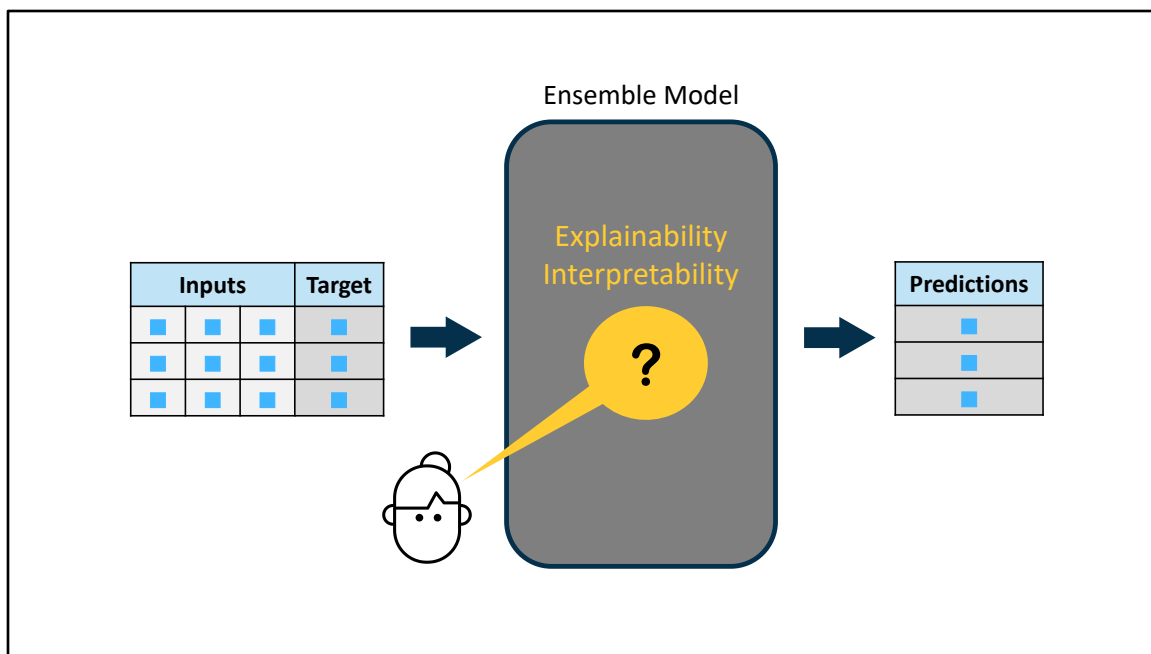
Essential Discovery Tasks

- ✓ Select an algorithm.
- ✓ Improve the model.
- ✓ Optimize the complexity of the model.
- ✓ Regularize and tune the hyperparameters of the model.
- Build ensemble models.

You can spend a fair amount of time building, improving, optimizing, and tuning a single machine learning model for each algorithm. However, any one of these individual models might not be the most effective solution for a specific problem and data set. In this case, you can consider building an ensemble model.



An ensemble model is an aggregation of multiple models. The final prediction from the ensemble model is a combination of the predictions from the component models. These component models can be trained from different algorithms that approach the problem from different perspectives, or from the same algorithm applied to different samples or using different tuning settings, or both.



It is typically not possible to interpret an ensemble model.

The screenshot displays the SAS Model Studio - Build Models interface. The top navigation bar includes a menu icon, the title 'Model Studio - Build Models', a search bar, and user information 'student'. Below the navigation bar, the 'Demo' tab is active, and the 'Pipeline Comparison' view is selected. The main workspace shows a 'Tree Based' ensemble pipeline with the following steps: Data, Explainer, Anomaly Detection, Imputation, Test Mining, Merge Variables, Variable Selection, Logistic Regression, Decision Tree, Gradient Boosting, and XGBoost. The right-hand panel is titled 'Forest' and contains the following configuration options:

- Description:** Fits a forest model, which consists of multiple decision trees based on different samples of the data and
- Number of trees:** 100
- Class target voting method:** Probability
- Tree-splitting Options:**
 - Class target criterion:** Information gain ratio
 - Interval target criterion:** Variance
 - Maximum number of branches:** 2

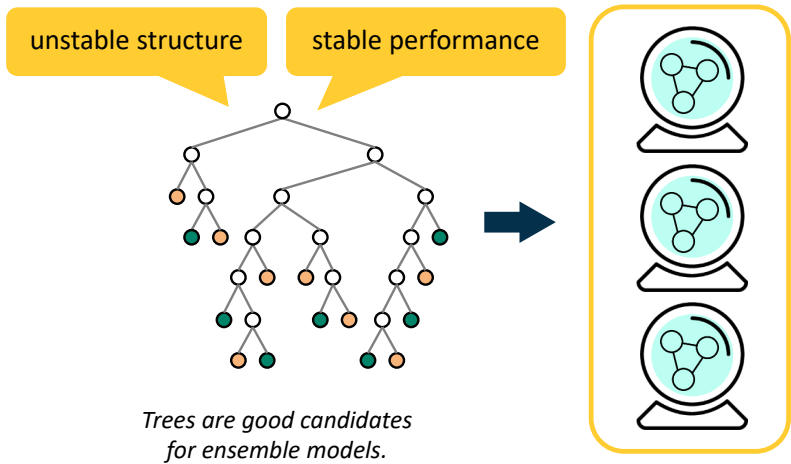
When you build an ensemble model in Model Studio, you can choose to include models based on a variety of algorithms or on a single algorithm. In this lesson, you learn to build two single-algorithm ensemble models: gradient boosting and forest.

Perturb and Combine Methods



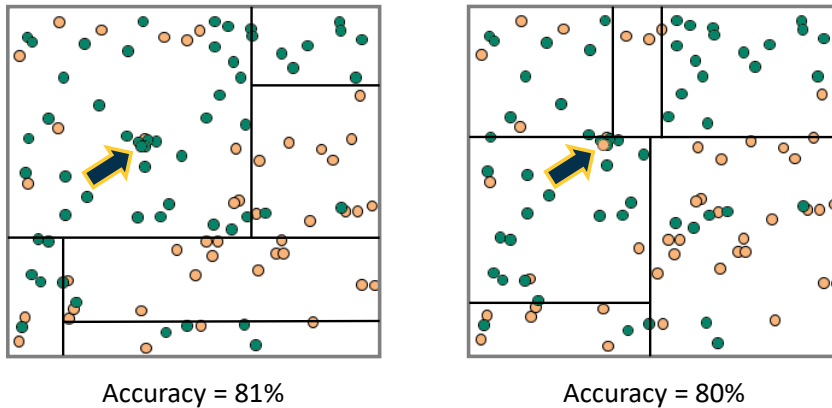
Copyright © SAS Institute Inc. All rights reserved.

Perturb and Combine Methods



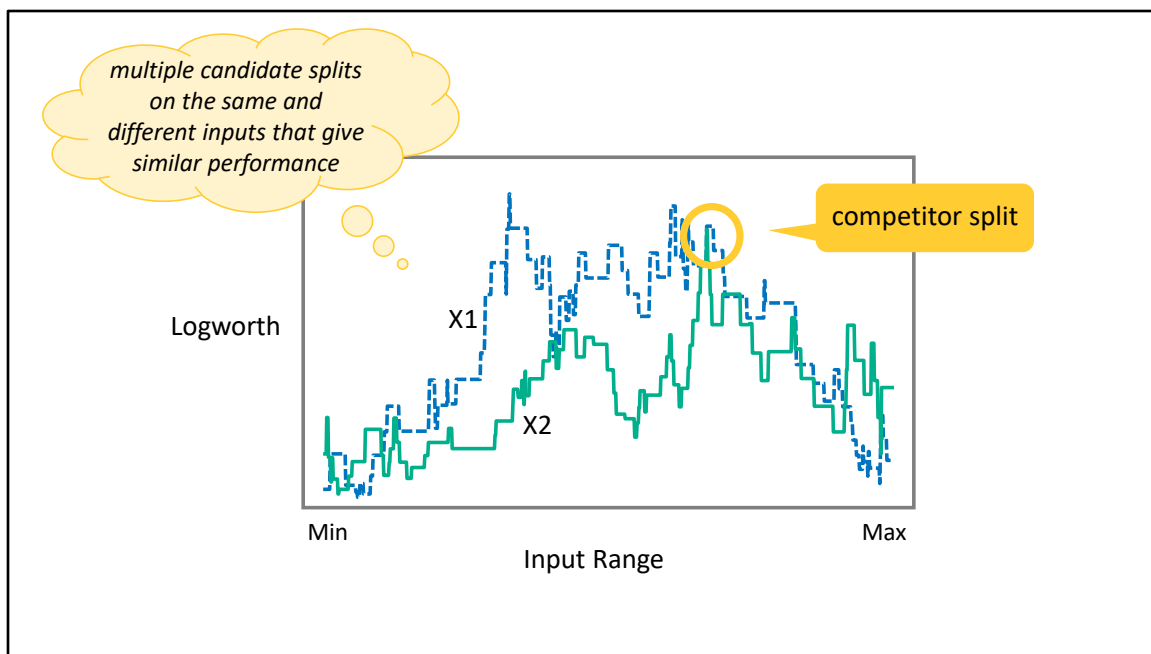
Decision tree models have a very unstable structure, so they are a good candidate for ensemble models. Even small changes in the training data can cause large changes in the tree structure and other aspects of the tree model. However, despite these changes, the overall performance of the trees usually remains stable.

Instability in a Single Tree



This example shows a scatter plot of two inputs and a binary target with the two outcomes green and gold. The plot also contains lines showing the results of recursive partitioning. This tree has an accuracy of 81%.

Changing the outcome of one case from green to gold results in a completely different tree, as shown by the change in the results of recursive partitioning. However, the accuracy is almost identical: 80%. So, what causes this instability?



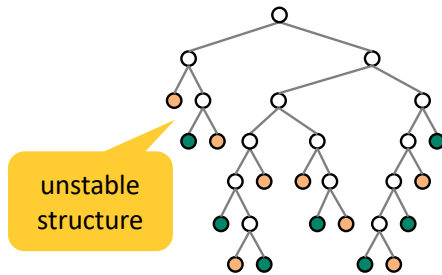
This plot shows the logworth for all possible values of two input variables, X1 and X2, which are candidates for a single split. The input range is on the x axis and logworth is on the y axis.

The instability of trees results from the large number of univariate splits considered during recursive partitioning and the increasing fragmentation of the data. At each split point, there are typically multiple candidate splits on the same and different inputs that give similar performance. These are called competitor splits. A small change in the data can easily result in the selection of a different competitor split, which produces different subsets in the child nodes. The changes in the resulting subsets increase with each new generation of the tree.

P & C Methods

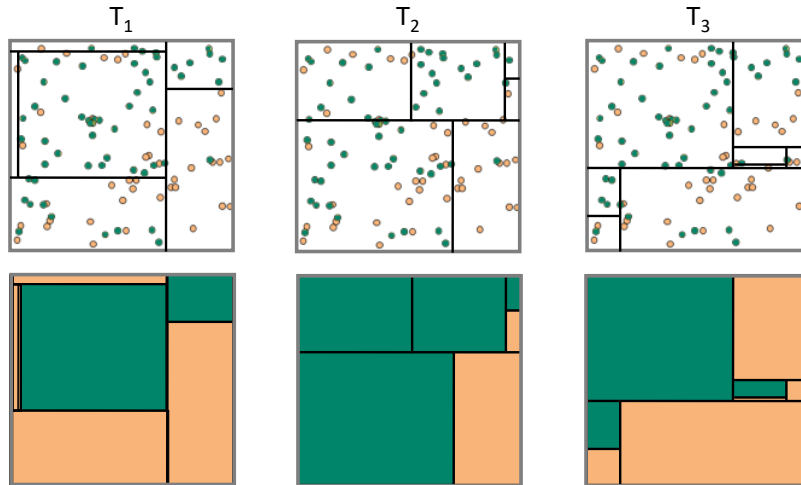
1. **Perturb:** Create different models.

2. **Combine:** Create a single prediction.



Fortunately, it is possible to take advantage of the instability of trees to create more powerful models. *Perturb and combine* (or *P & C*) methods are used to create ensemble models in two steps. The perturb step creates different models by manipulating the distribution of the data or altering the construction method. The combine step then uses the predictions of the models built in the perturb step to create a single prediction. Perturb and combine methods can be used with any unstable algorithm but they are mostly used with trees.

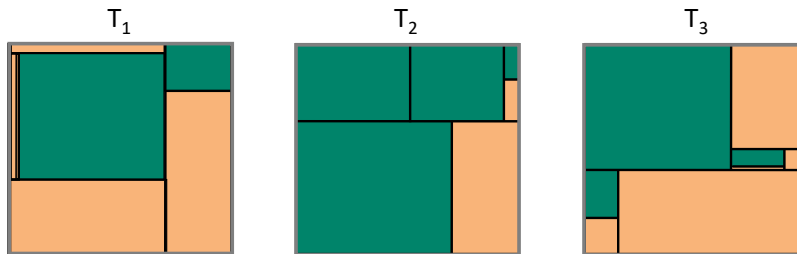
1. **Perturb:** Create different models.



The three pairs of plots shown here represent three different trees created during the perturb step. These trees are based on data shown earlier, which has a binary target (with green and gold outcomes) and two inputs. For each tree, the top plot is a scatterplot of the data with lines showing the splits. The bottom plot shows the decision predictions from the tree. Notice that the data points are identical in all three scatterplots. From this, we can infer that the perturb method in this example involves altering the construction method, such as changing the splitting criterion, instead of manipulating the data distribution.

1. **Perturb:** Create different models.

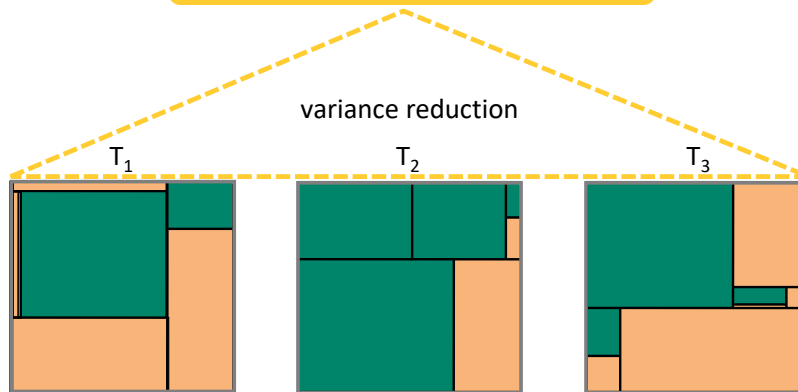
- resampling
- subsampling
- adding noise
- adaptively reweighting
- randomly choosing from the competitor splits



Commonly-used perturbation methods include resampling, subsampling, adding noise, adaptively reweighting, and randomly choosing from the competitor splits.

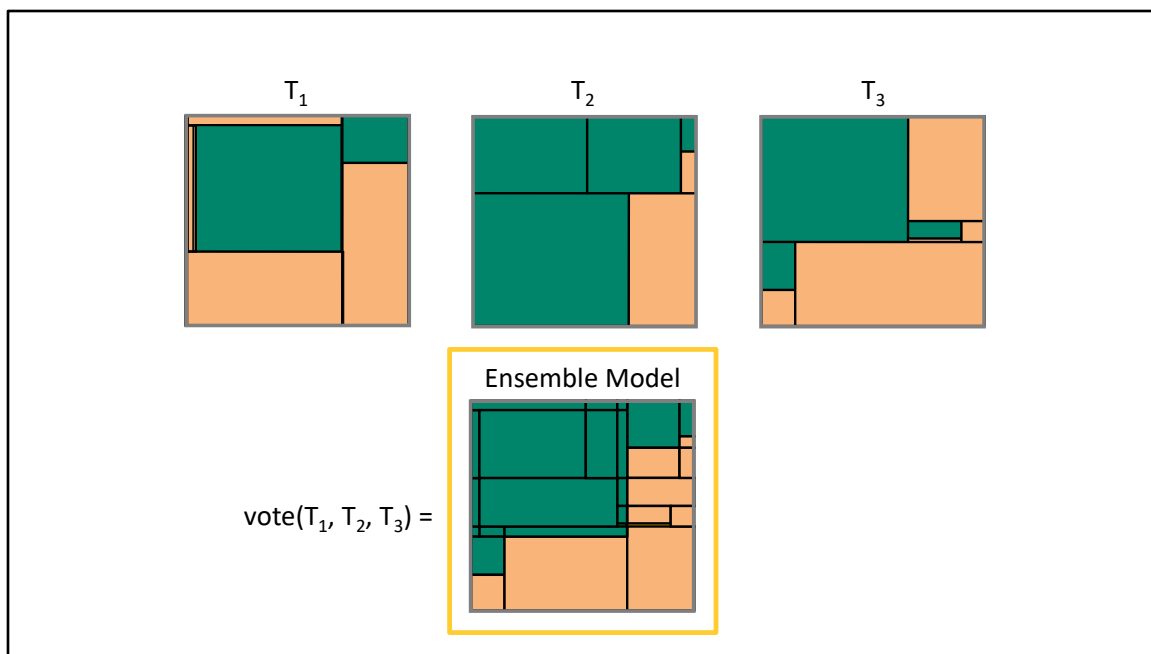
2. **Combine:** Create a single prediction.

- voting
- weighted voting
- averaging

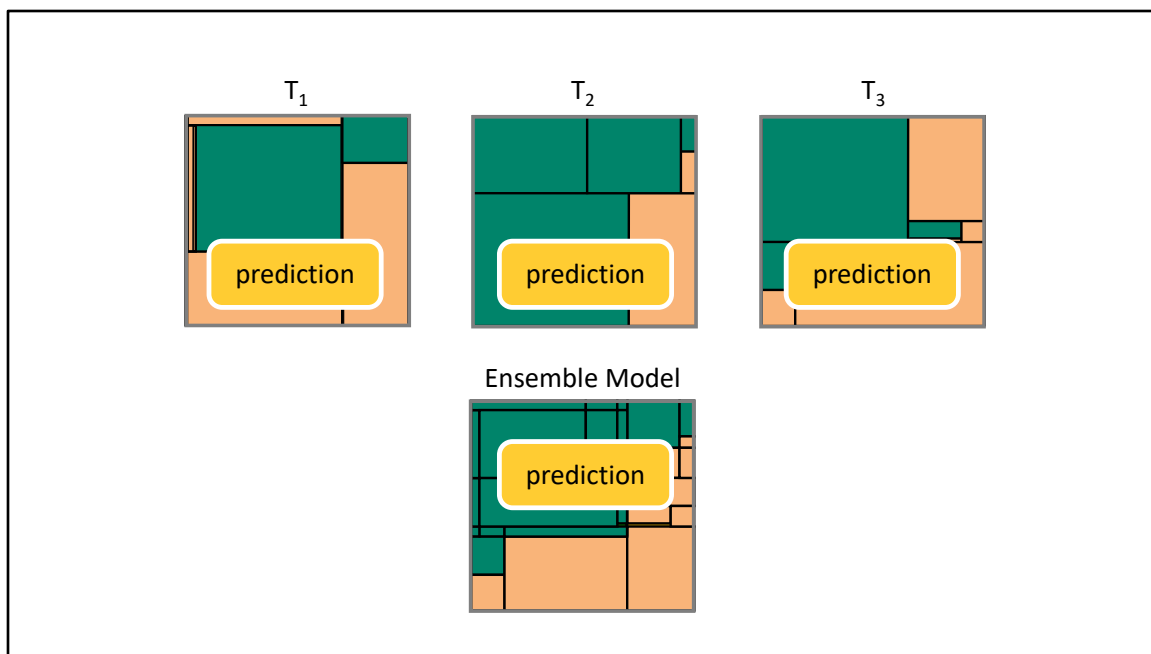


To create the ensemble model, the individual models can be combined in various ways, such as voting on the classifications, using weighted voting (where some models have more weight), and averaging the predicted values (either weighted or unweighted). The attractiveness of P & C methods is their improved performance over single models.

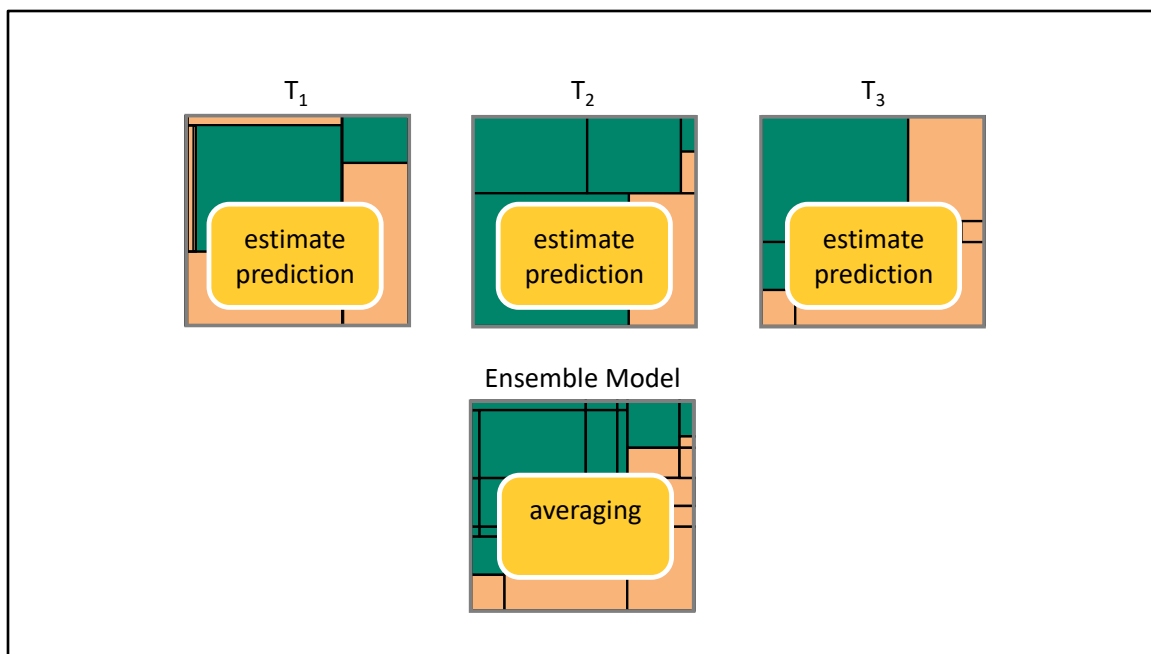
One reason why simple P & C methods give improved performance is variance reduction. If the base models have low bias and high variance, then averaging decreases the variance. In contrast, combining stable models can negatively affect performance. P & C methods create effective ensemble models for reasons beyond simple variance reduction, and these are the topic of much research.



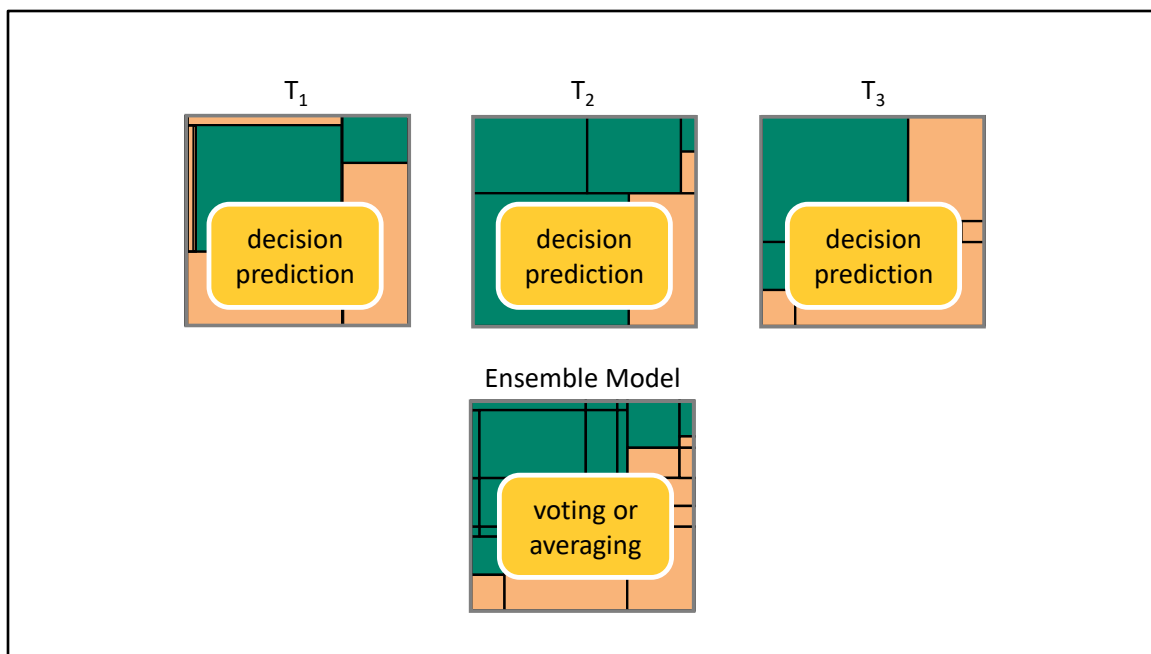
Here is a graph of the ensemble model for this example. Graphical representations show that ensembles of trees have decision boundaries of much finer resolution than would be possible with a single tree. The decision boundaries of a single tree can be viewed as step functions. The benefit of ensembling many trees together is that, by adding more steps, the steps themselves are essentially smoothed out.



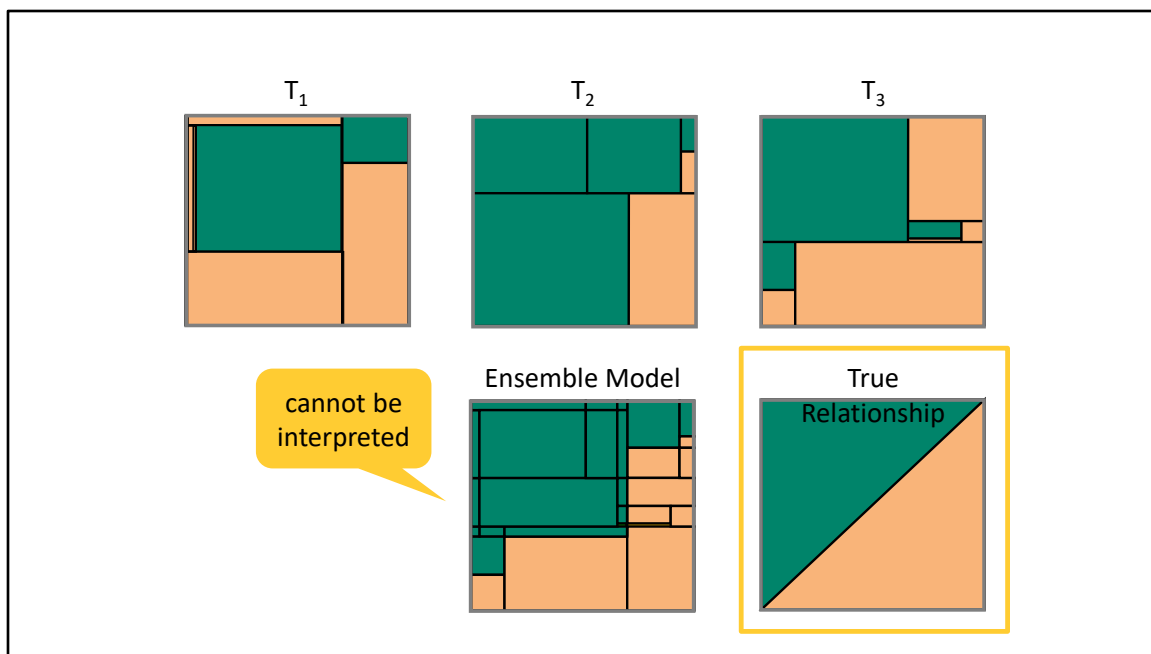
A new case is scored by obtaining a prediction from each individual tree and then combining the results.



If the predictions are estimates, the results are averaged.



If the predictions are decisions, results can be combined by voting or averaging the posterior probabilities as the basis for the final decision.



The main drawback of ensemble models is that they cannot be interpreted, unlike a single tree.

Here is a representation of the true relationship between the target and the inputs. This relationship is defined by a straight line. All cases above that line have a green outcome and all cases below the line have a gold outcome. Obviously, this is a simplified example. Real data are not likely to be linearly separable.

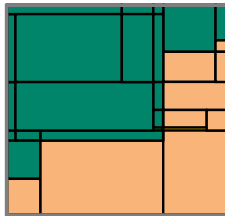
P & C Methods

bagging

boosting

...

Ensemble Model



In this course, we look at two P & C methods: bagging and boosting. Ensemble methods are a very active area of research in the fields of machine learning and statistics, so there are many additional P & C methods.

Question: Perturb and Combine Methods



Copyright © SAS Institute Inc. All rights reserved.

Question: Perturb and Combine Methods

Perturb and combine methods are used only with decision trees.

- a. True
- b. False

Answer: Perturb and Combine Methods

Perturb and combine methods are used only with decision trees.

- a. True
- ☒ b. False

CA: b

FB: Perturb and combine methods can be used with any unstable algorithm but they are mostly used with trees.

Bagging



Copyright © SAS Institute Inc. All rights reserved.

Bagging

P & C Methods

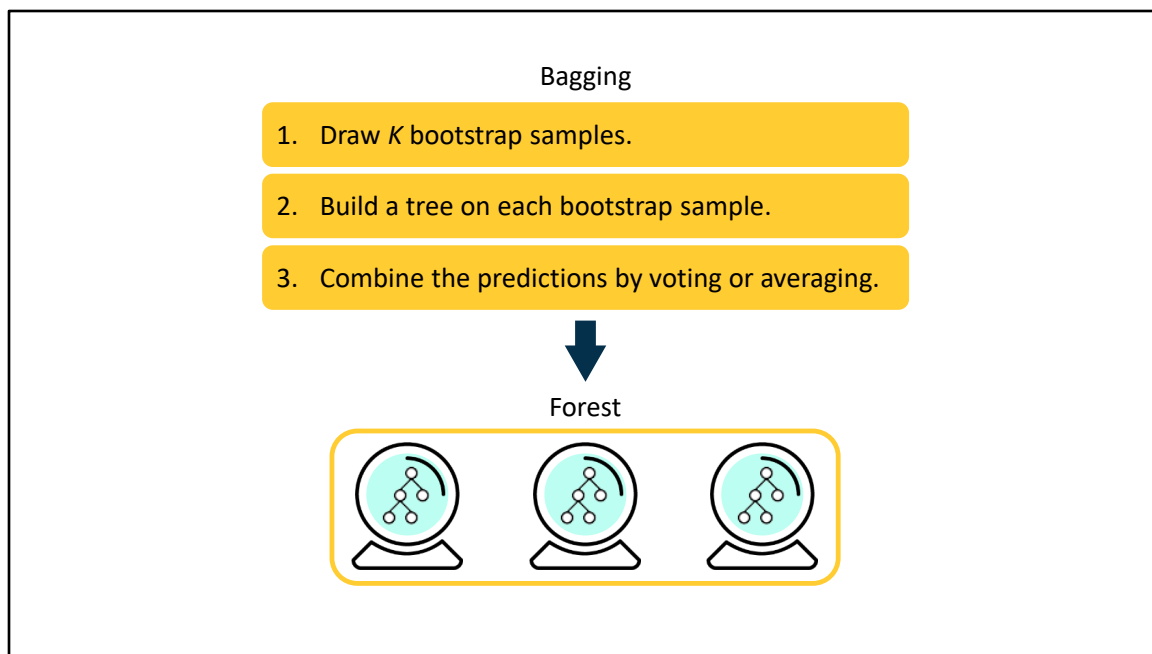
bagging

(bootstrap aggregation)

boosting

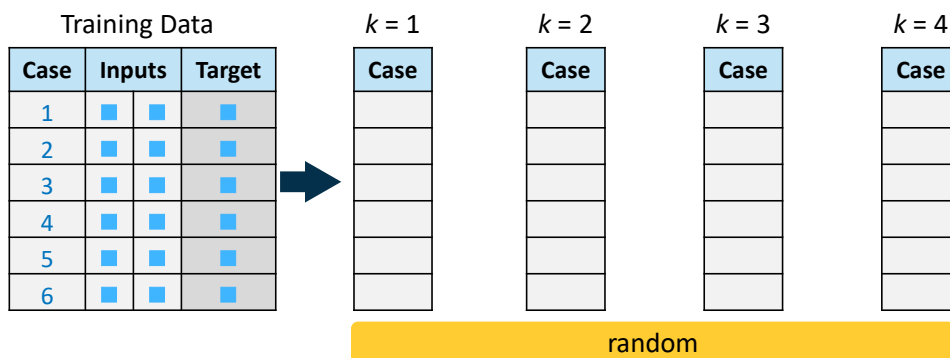
...

Bagging (or *bootstrap aggregation*) is the original perturb and combine method. It was developed by Leo Breiman in 1996.



Let's look at the three steps of the original bagging algorithm. As you'll learn later, Model Studio uses a variation of this algorithm to construct a forest. The three steps are as follows: Step 1. Draw k bootstrap samples. Step 2: Build a tree on each bootstrap sample. And step 3: Combine the predictions by voting or averaging.

1. Draw K bootstrap samples.



Let's look at a simple example. From a training data set that has six cases, the first step is to draw four bootstrap samples. Each sample has six cases. A *bootstrap sample* is a random sample from a training data set.

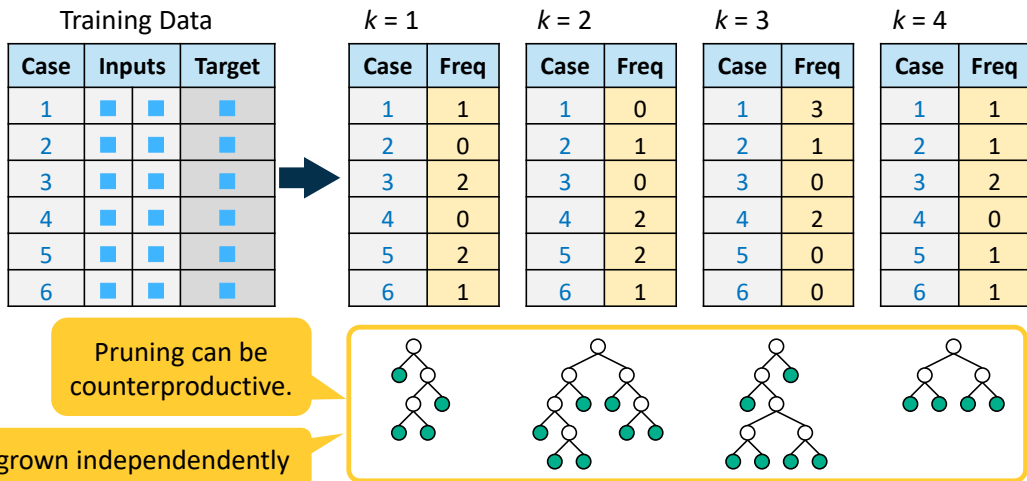
1. Draw K bootstrap samples.

Training Data				$k = 1$		$k = 2$		$k = 3$		$k = 4$	
Case	Inputs		Target	Case	Freq	Case	Freq	Case	Freq	Case	Freq
1	■	■	■	1	1	1	0	1	3	1	1
2	■	■	■	2	0	2	1	2	1	2	1
3	■	■	■	3	2	3	0	3	0	3	2
4	■	■	■	4	0	4	2	4	2	4	0
5	■	■	■	5	2	5	2	5	0	5	1
6	■	■	■	6	1	6	1	6	0	6	1

random with replacement

For bagging, a bootstrap sample is drawn with replacement. This means that some of the cases might be left out of the sample, and some cases might be represented more than once. For example, in the first bootstrap sample, cases 3 and 5 appear twice, cases 1 and 6 appear once, and cases 2 and 4 do not appear at all.

2. Build a tree on each bootstrap sample.



The second step is to build a tree on each bootstrap sample. When you create an ensemble of bagged trees, large trees with low bias and high variance are ideal, so pruning can be counterproductive. In the bagging method, trees are grown independently of each other, so they can be built in parallel in a distributed computing environment like SAS Viya. This makes bagging a relatively fast method compared to other P&C methods.

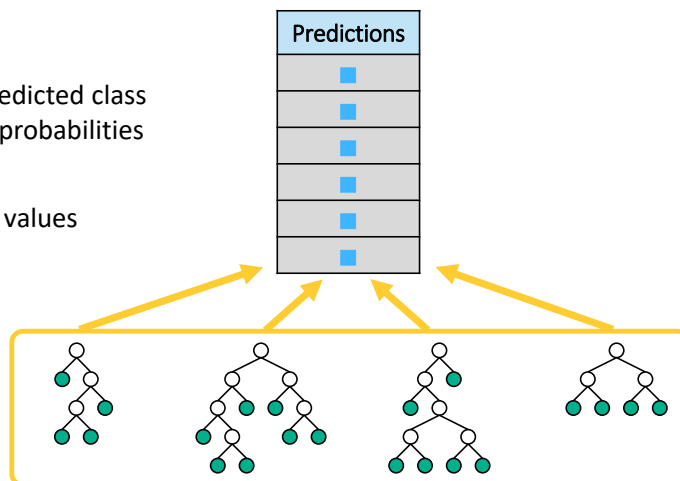
3. Combine the predictions by voting or averaging.

Classification:

- plurality vote of the predicted class
- mean of the posterior probabilities

Interval:

- mean of the predicted values



The third step is to combine the predictions by voting or averaging. For classification problems, you take the plurality vote of the predicted class or the mean of the posterior probabilities. Averaging the posterior probabilities often gives slightly better performance than voting. If you have an interval target, you take a mean of the predicted values.

Boosting



Copyright © SAS Institute Inc. All rights reserved.

Boosting

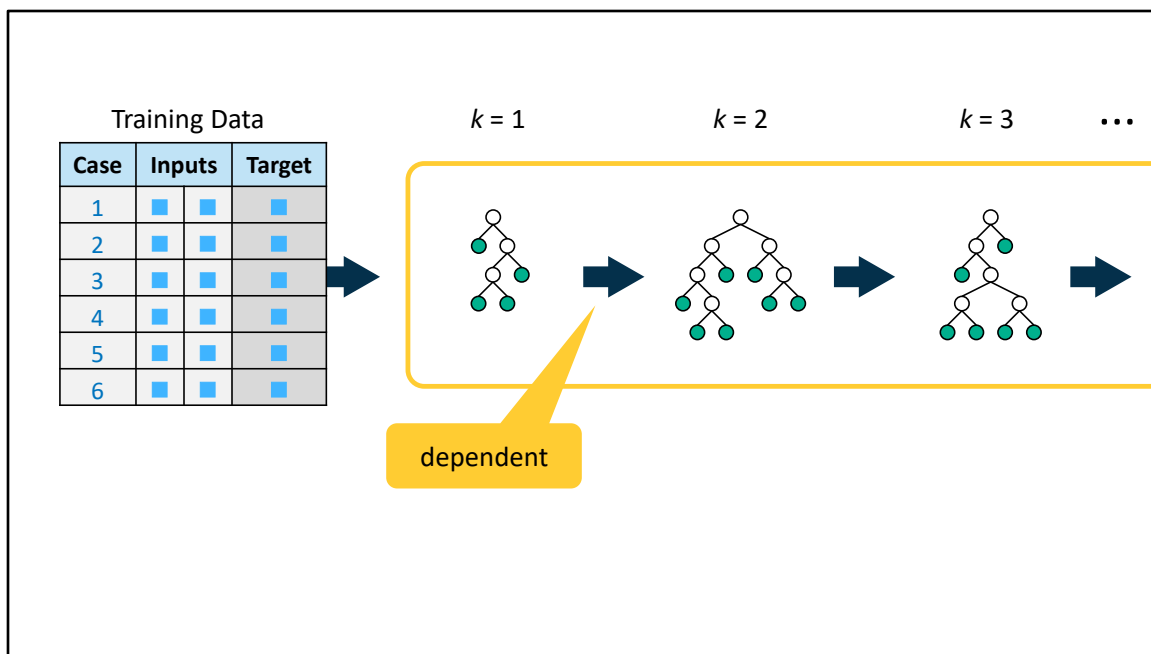
P & C Methods

bagging

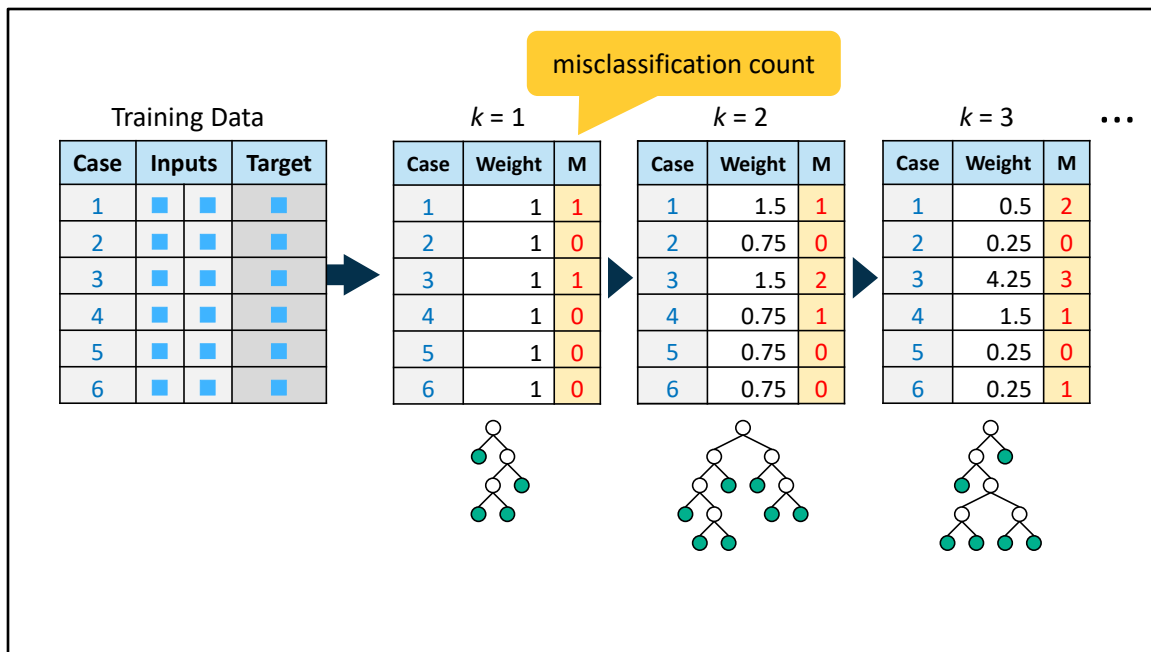
boosting

...

Boosting is a perturb and combine method for creating tree-based ensemble models for categorical targets.

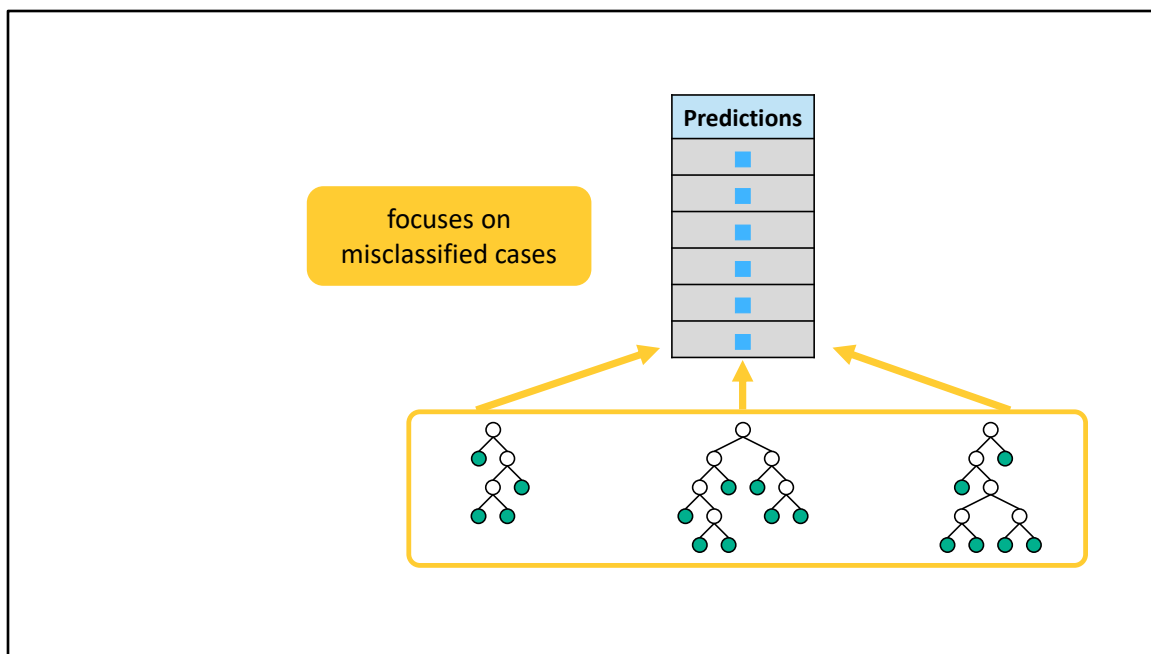


Unlike the bagging method, each tree that boosting creates is dependent on the tree from the previous iteration.



Across iterations, the algorithm keeps a cumulative count of misclassifications. Each case is then weighted based on whether its misclassification count increases in the current iteration. If the misclassification count increases, the weight increases. If the misclassification count remains the same, the weight decreases. The weights influence the likelihood that the case is selected in the sample for the next iteration.

The boosting process continues in this fashion for a predetermined number of iterations. The number of iterations equals the number of trees in the series.



The main advantage of boosting is that the algorithm focuses on the misclassified cases. This improves the performance of the model.

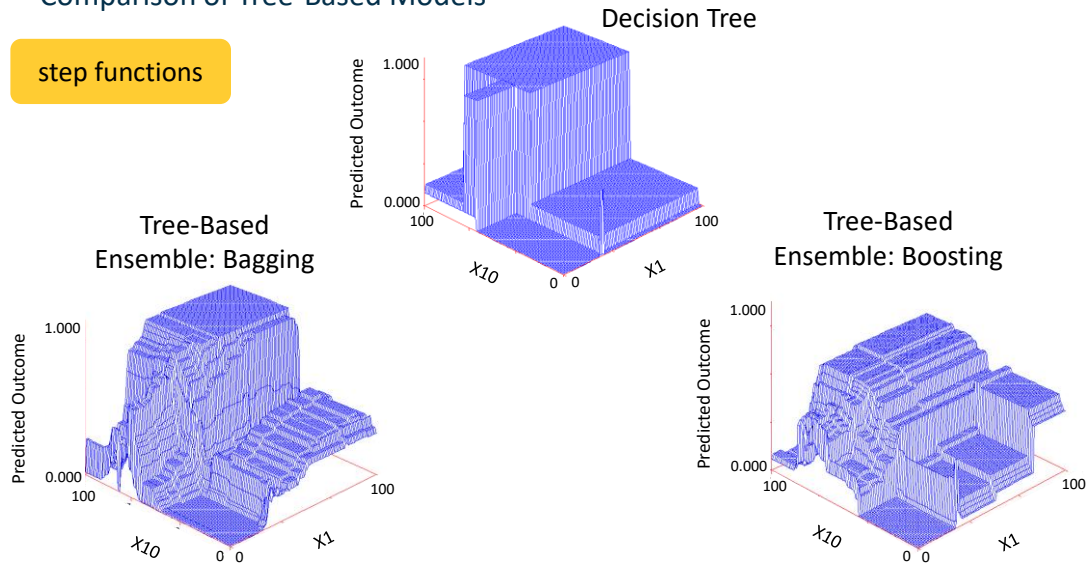
Comparison of Tree-Based Models



Copyright © SAS Institute Inc. All rights reserved.

Comparison of Tree-Based Models

step functions



Let's compare the accuracy of three tree-based models: a single decision tree, a tree-based ensemble model based on bagging, and a tree-based ensemble model based on boosting. The three plots shown here are based on a data set with two inputs.

A decision tree model is created through recursive partitioning, so all tree-based models use step functions to split the input space.

The plot for the single decision tree shows that there are few steps and those steps are large. Therefore, relatively few steps are used to classify the target, which can decrease the accuracy.

The plots for the two ensemble models have more steps and those steps are smaller. Because more steps are used to classify the target, accuracy can be increased compared to a single tree.

Notice that boosting smooths the prediction surface more than bagging because boosting emphasizes the misclassified cases during the training.

Gradient Boosting Models

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Building a Gradient Boosting Model

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Autotuning Options for Gradient Boosting in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



(Optional) Practice: Build a Gradient Boosting Model Using Autotuning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Tree-Based Models



Copyright © SAS Institute Inc. All rights reserved.

Question: Tree-Based Models

Which of the following statements is true regarding tree-based models?

- a. Small changes in the training data can cause large changes in the structure of a tree.
- b. Ensemble models are used only with decision trees.
- c. In the boosting algorithm, cases that are correctly classified are given more weight in subsequent models.
- d. In the bagging algorithm, the training data are resampled without replacement.

Answer: Tree-Based Models

Which of the following statements is true regarding tree-based models?

- ☒ a. Small changes in the training data can cause large changes in the structure of a tree.
- ☐ b. Ensemble models are used only with decision trees.
- ☐ c. In the boosting algorithm, cases that are correctly classified are given more weight in subsequent models.
- ☐ d. In the bagging algorithm, the training data are resampled without replacement.

CA: a

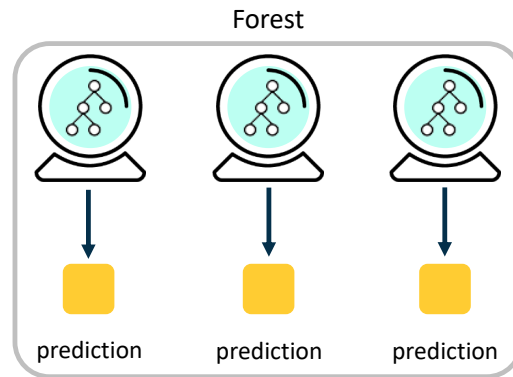
FB: Even small changes in the training data can cause large changes in the tree structure. However, despite these changes, the overall performance of trees usually remains stable.

Forest Models

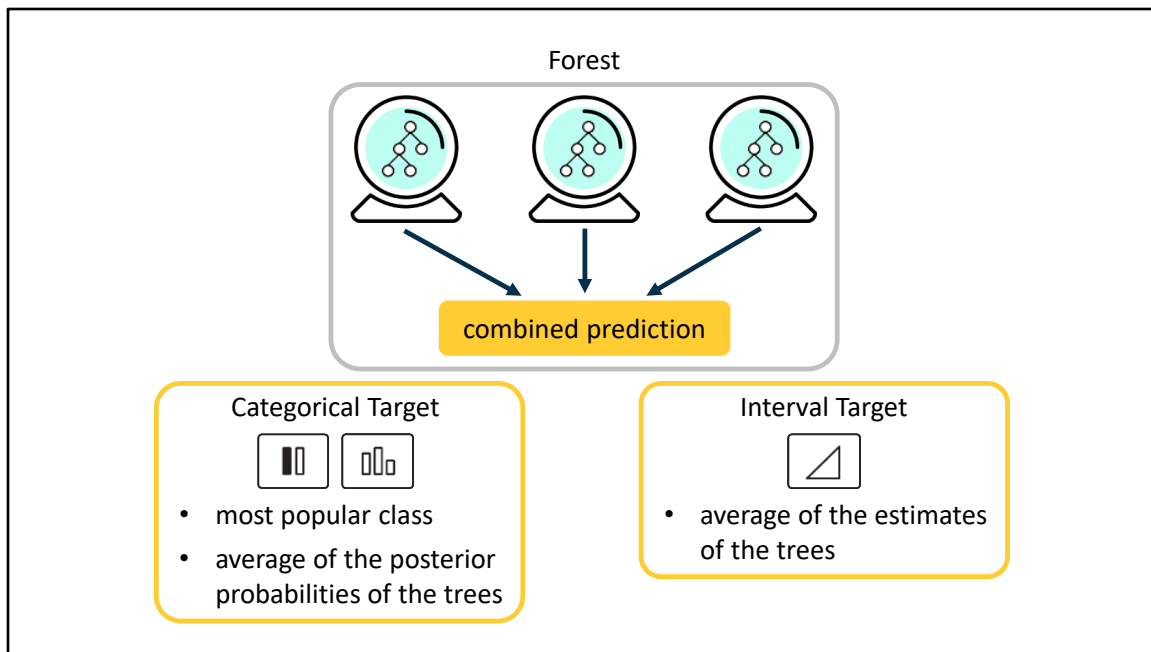


Copyright © SAS Institute Inc. All rights reserved.

Forest Models



A forest is an ensemble of decision trees, each one able to predict its own response to a set of input variables.



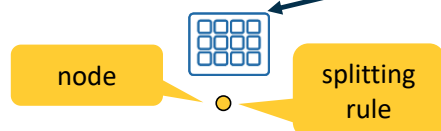
The results from the individual trees are combined to provide the final prediction. For a categorical target, the forest model's prediction is either the most popular class (as determined by a vote) or the average of the posterior probabilities of the individual trees. For an interval target, the forest model's prediction is the average of the estimates from the individual decision trees.

1. Select a sample of cases, with replacement.

2. Select a sample of input variables from all available inputs.

Training Data

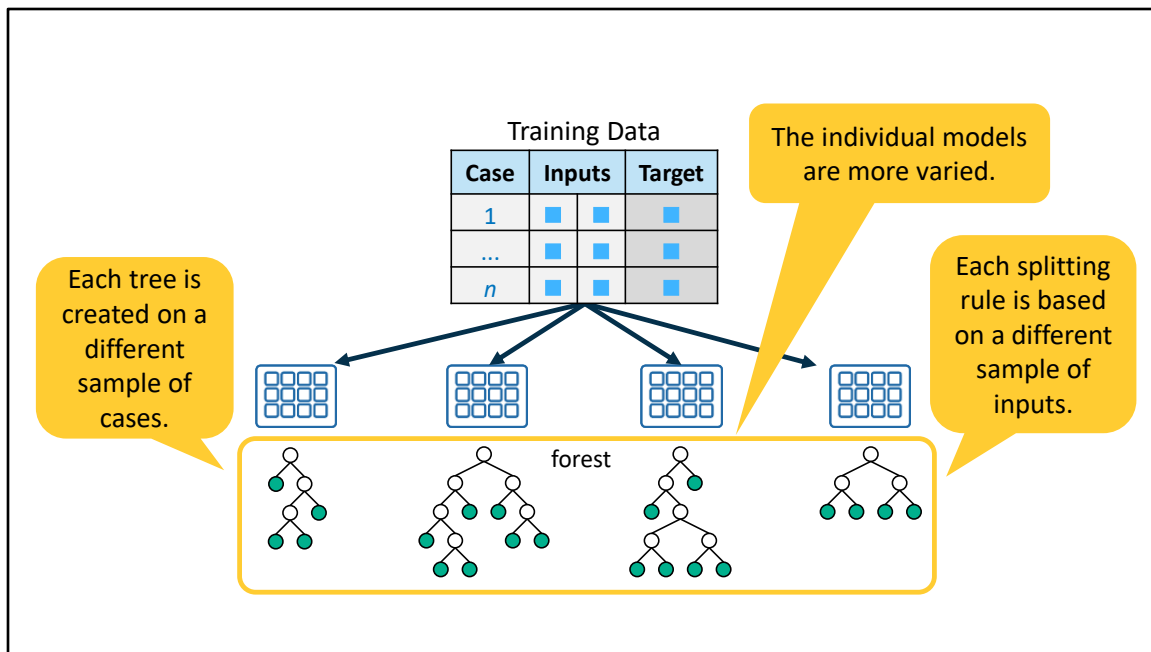
Case	Inputs		Target
1	■	■	■
...	■	■	■
n	■	■	■



The input that has the ***strongest association with the target*** is used in the splitting rule for that node.

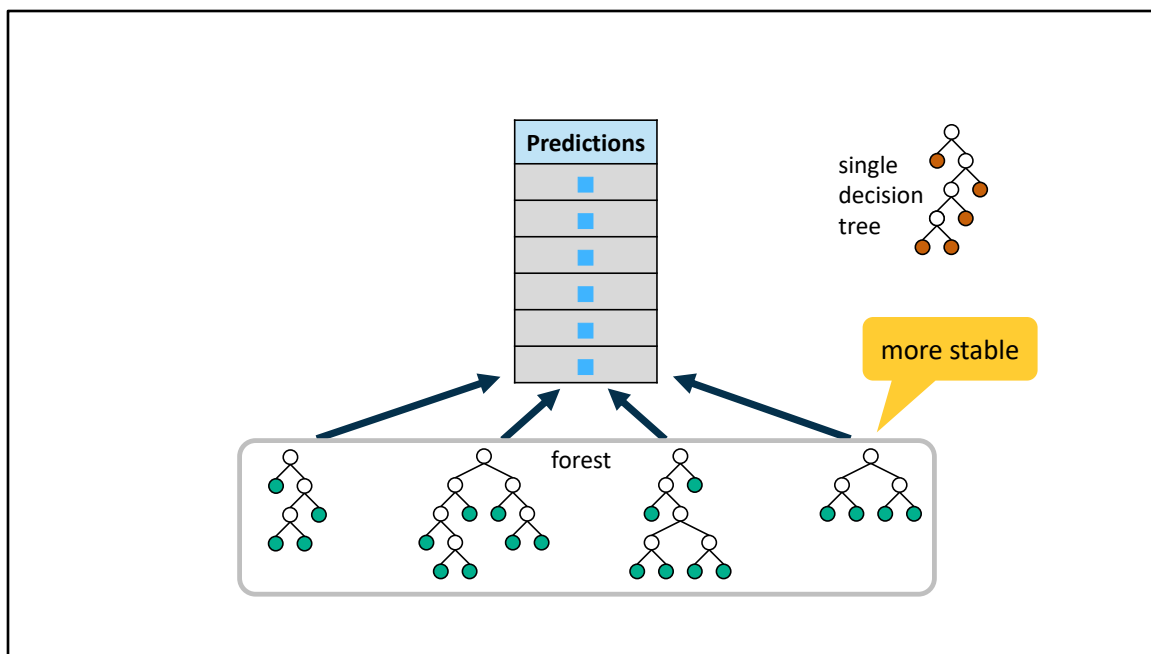
In Model Studio, the forest algorithm uses the following process to build each tree: The algorithm selects a sample of cases, with replacement, from the original training data. Then, for each node, the algorithm selects a sample of input variables from all available inputs.

From this sample, the input that has the strongest association with the target is used in the splitting rule for that node. Therefore, the method of selecting the input variable for a splitting rule is different for a forest than it is for the split-search process used to build an individual tree.

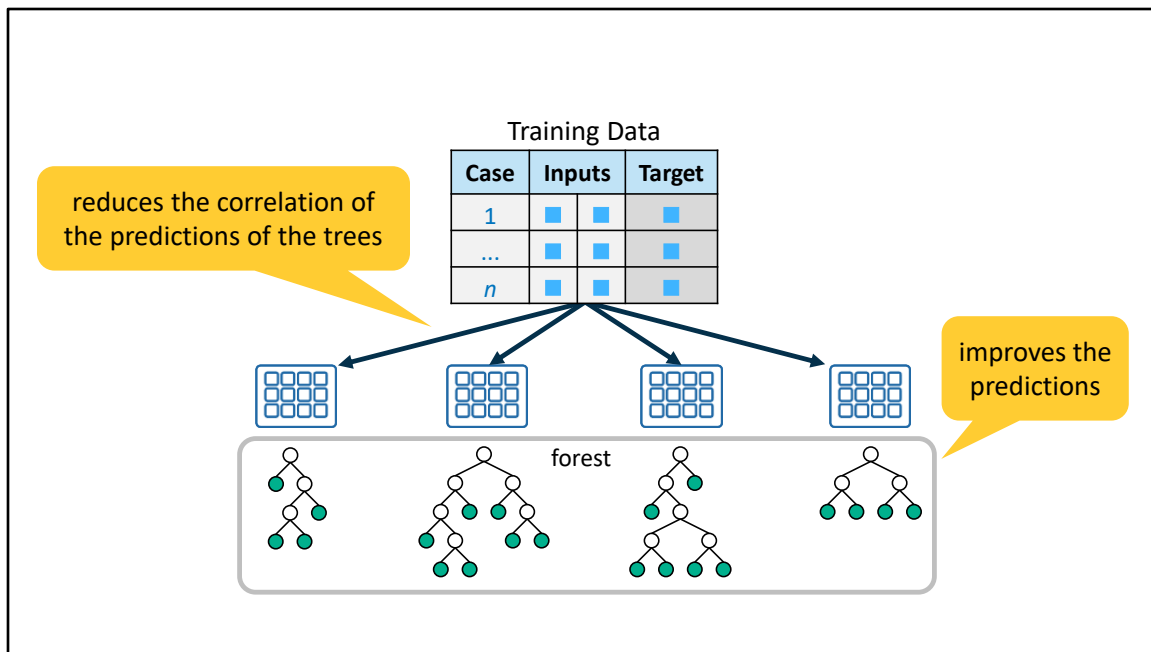


As you can see, the trees in a forest created in Model Studio differ from each other in two ways.

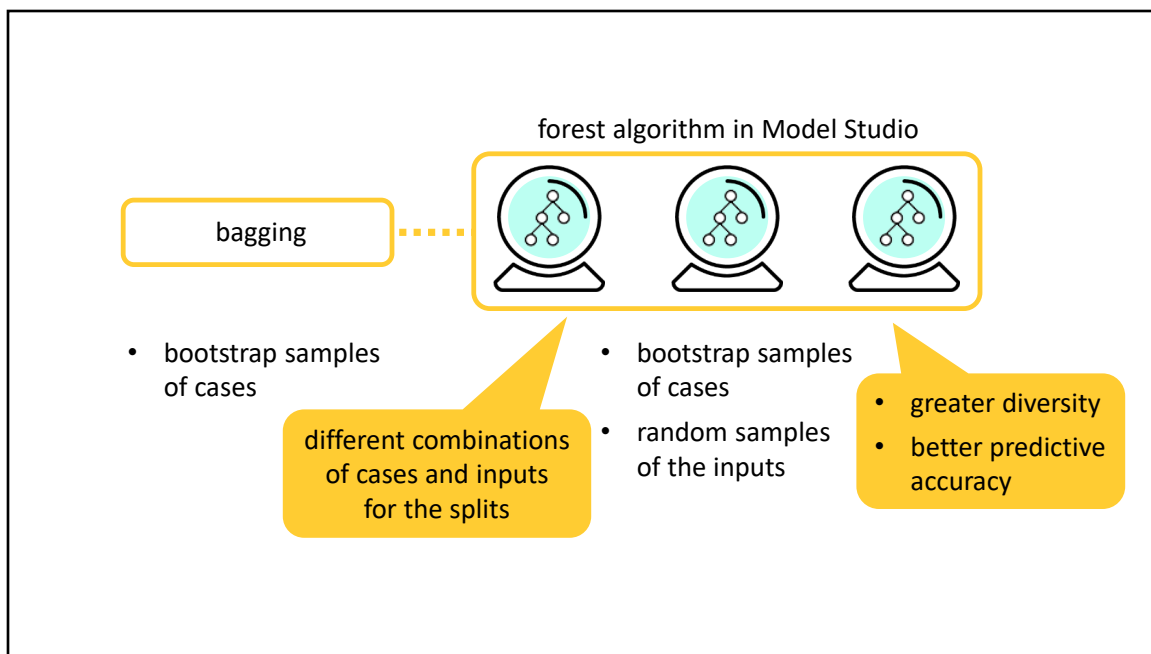
Each tree is created on a different sample of the cases, and each splitting rule is based on a different sample of the inputs. This process ensures that the individual models in the ensemble are more varied.



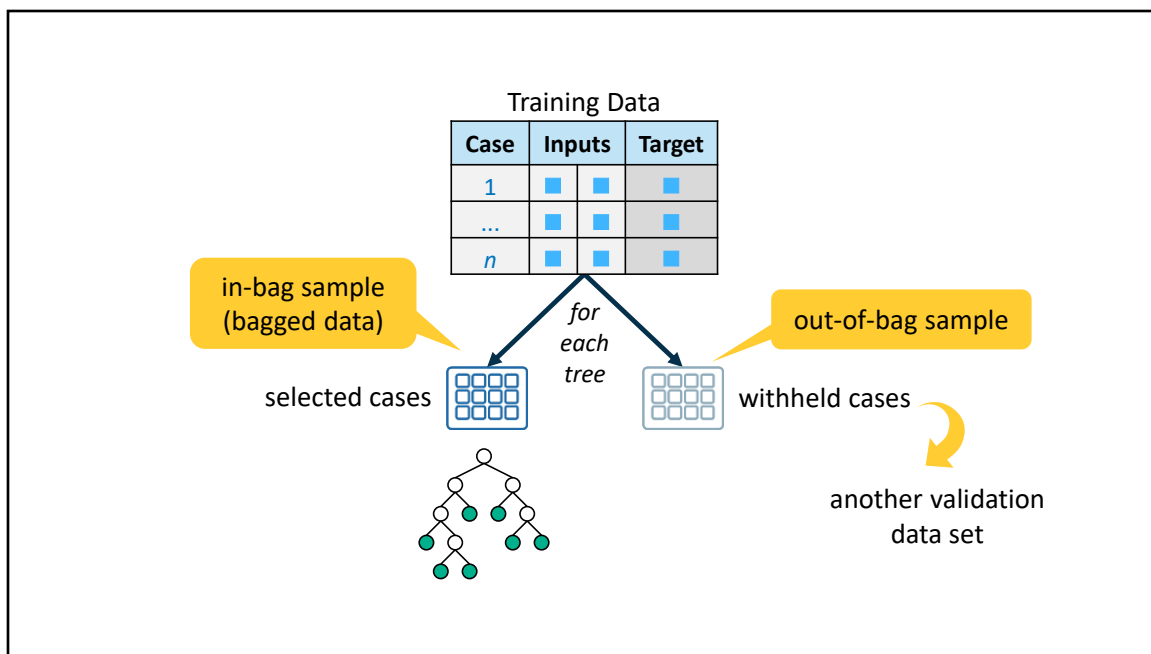
The process that the forest algorithm uses to build the individual trees and then combine the predictions results in a more stable model than a single tree.



Training each tree with different data reduces the correlation of the predictions of the trees. This, in turn, is likely to improve the predictions of the forest as compared to the naïve method of using the same data to build all the trees in a forest.



Although the forest algorithm in Model Studio is based on bagging, the sampling method is not entirely the same. Like bagging, Model Studio's forest algorithm takes bootstrap samples of the cases. However, this forest algorithm also takes random samples of the inputs. Therefore, the trees in the forest use different combinations of cases and inputs to determine the splits. This additional perturbation (beyond bagging) leads to greater diversity in the trees, and often better predictive accuracy than with bagging.



Each sample of the original training data that is selected to train a specific decision tree is called *bagged data* or *in-bag sample*. For each tree in the forest, the data that are withheld from training are called an *out-of-bag sample*. Model assessment measures (such as misclassification rates and average squared error) and iteration plots are constructed on both the entire training data set and the out-of-bag sample. So, you can think of the out-of-bag sample as another validation data set.

Demo: Building a Forest Model

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Gradient Boosting versus Forest Models



Copyright © SAS Institute Inc. All rights reserved.

Question: Gradient Boosting versus Forest Models

Which of the following is a difference between gradient boosting and forest models in Model Studio?

- a. Forests are based on bagging. Gradient boosting models are based on boosting.
- b. The gradient boosting method samples the inputs at each iteration. The forest method does not.
- c. In a forest, each tree's target is the residual from the previous tree. In a gradient boosting model, the trees are independent of each other.

Answer: Gradient Boosting versus Forest Models

Which of the following is a difference between gradient boosting and forest models in Model Studio?

- a. Forests are based on bagging. Gradient boosting models are based on boosting.
- b. The gradient boosting method samples the inputs at each iteration. The forest method does not.
- c. In a forest, each tree's target is the residual from the previous tree. In a gradient boosting model, the trees are independent of each other.

CA: a

FB: Forests and gradient boosting models are based on different P&C methods.

Autotuning Options for Forests in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



(Optional) Practice: Build a Forest Model Using Autotuning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



3.6 Lesson Conclusion



Lesson Quiz

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.

